Semester 3 — Lab ETE pack

Three labs in one file. Page numbers below are PDF pages; use the bookmarks panel to jump.

2	**ECE2130 Electronic Devices Lab** LTspice from zero, Exp 0–12, design problem, revision set	83 pp.
85	**ECE2131 Digital Electronics Lab** Pin-to-pin wiring for every experiment, designs checked, revision set	47 pp.
132	**ECE2132 Data Structures & Algorithms Lab** C++ from zero, Exp 1–12 (corrected, tested code), exam day, revision set	91 pp.

Same method in every pack: guess first, then read; type or build it yourself; check boxes close each chapter with answers at its end; the revision set at the end of each lab is the re-test to do on a later day.

00 · Start here: the whole lab on one page

ECE2130 / VDT2130 Electronic Devices Lab-I · source: the lab manual (session 2025-26), 39 pp. · every expected number in this pack comes from running the manual's own circuits in LTspice

The lab as one path

Instruments (Exp 0) → Diodes: 1 zener curve · 2 regulator · 3 clipper · 4 clamper →
BJT: 5 curves · 6 CE amp · 7 follower · 8 feedback → JFET: 9 curves · 10 CS amp → 11 oscillator →
12 MOSFET inverter → Extra: two-stage amp + design problem

Each block builds on the one before it:

- **Diodes first.** A diode is one junction. Exp 1 measures it, and Exps 2–4 use it in three circuits.
- **Then the BJT.** A transistor is two junctions. Exp 5 measures its curves, and Exps 6–8 turn it into amplifiers.
- **Then FETs.** The same "measure the curve, then build the amplifier" pattern, with a field-controlled device.
- **Exp 11 and Exp 12** take an amplifier somewhere new: positive feedback makes an oscillator, and a transistor used as a switch makes a logic gate.

How each experiment is run and marked

Exp	Topic	Done in	Marked by
0	DC and AC measurement	hands-on	not assessed
1	Zener characteristics, dynamic resistance	hands-on	viva, ETE
2	Zener line and load regulation	LTspice / Virtual Lab	viva, ETE
3	Clipper	hands-on	viva, ETE
4	Clamper	LTspice	viva, ETE
5	BJT CE characteristics, h-parameters	LTspice / Virtual Lab	viva, ETE
6	CE amplifier frequency response	hands-on	viva, ETE
7	Emitter follower gain	hands-on	viva, ETE
8	Negative feedback (bypass capacitor)	LTspice	viva, ETE
9	JFET drain and transfer characteristics	LTspice	viva, ETE
10	CS amplifier frequency response	LTspice	viva, ETE
11	RC phase-shift oscillator	LTspice	viva, ETE
12	MOSFET inverter	LTspice	viva, ETE

The manual does not give this lab's marks split. The Digital Electronics lab manual gives 100 = internal 60 + external 40, with a 3-hour exam. Expect the same shape here, but it is not confirmed.

ETE (end-term exam) for a lab usually means: you are given one experiment, you build or simulate it, you show the result, and you answer **viva** (oral) questions on it. So for every experiment you need to be able to:

1. draw the circuit from memory,
2. build or simulate it without help,
3. get the expected readings and calculate the result,
4. answer "why" questions about it.

Each chapter below gives you those four things, in that order.

What goes in the record (lab file)

The manual (p. 3) asks for six headings per experiment:

Heading	What to write
Aim	as in the list above
Circuit diagram	the circuit, labelled with values
Theory	about 15 lines. Each chapter has a ready "Theory for the record" block.
Observations	the table or a well-labelled waveform
Calculations	if any, with the formula first
Result	the parameters, **with % error**

$$\% \text{ error} = \frac{|\text{theoretical} - \text{observed}|}{\text{theoretical}} \times 100$$

How a chapter works

> **THE QUESTION**
> Each chapter opens with the experiment's aim, quoted from the manual, plus a real viva-style question.

> **GUESS FIRST**
> Answer it on paper before reading on, even if your answer is a wild guess. Guessing first makes the explanation stick better than reading cold. This line is the reason the pack works, not decoration.

Then come **the ideas needed**, **the circuit**, a **Do it** box with the exact steps, **what you should see** (simulated curves and numbers), the worked **calculation**, a **Write in your record** box, **traps**, and **checks**. The answers to the checks are on the last page of each chapter.

Errors in the manual (know these for the viva)

Where	The manual says	Correct
Exp 0	$V_{rms} = V_p/\sqrt{2}$, with V_p read as peak-to-peak	For a sine, $V_{rms} = V_{peak}/\sqrt{2} = V_{pp}/(2\sqrt{2})$
Exp 1	Requirement: diode 1N4007	The experiment needs a **zener** diode. A 1N4007 is an ordinary rectifier and will not show a sharp breakdown below about 1000 V.
Exp 5	h_{21} = output admittance, h_{22} = forward current gain	Swapped. $h_{21} = h_{fe}$ (forward current gain), $h_{22} = h_{oe}$ (output admittance).
Exp 5	$1/h_{oe} = \Delta I_C/\Delta V_{CE}$	$h_{oe} = \Delta I_C/\Delta V_{CE}$. The ratio is a conductance, so $1/h_{oe}$ is the output resistance.
Exp 5	"Keep emitter current I_b = 20 A"	**Base** current I_B = 20 **µA**
Exp 5	Fig 5.1 with 62 kΩ in the collector	With 62 kΩ, I_C can never exceed 10 V / 62 kΩ = 0.16 mA, so the I_B = 20/40 µA table cannot be filled. See Exp 5.
Exp 6	Input 500 mV	This clips the output badly. Use about 20 mV (Exp 6 shows both).
Exp 7	Input impedance "of the order of hundred ohms"	Input impedance is **high**: about 3 kΩ here (set by the bias resistors), and about 600 kΩ looking into the base alone. The **output** impedance is the low one (about 70 Ω).
Exp 9	$r_d = (7 - 6)/(0.1183 - 0.1180) = 3.3$ kΩ	Decimal slip. The manual's own data (11.83 − 11.80 mA) gives 1 V / 0.03 mA = **33 kΩ**. LTspice gives 38 kΩ.
Exp 12	"V_{th} is the inverter threshold voltage, which is $V_{dd}/2$"	Two different things. V_{TH} is the MOSFET's turn-on voltage. V_M is where $V_{out} = V_{in}$. $V_M = V_{DD}/2$ only for a symmetric CMOS inverter, not for this resistor-load circuit ($V_M \approx 0.76$ V here).

01 · LTspice from zero

Needed for Exps 2, 4, 5, 8, 9, 10, 11, 12 and the extra experiment · LTspice is free (Analog Devices) and already installed on your laptop

Where this sits

LTspice basics → the 4 analyses → drawing a circuit → reading the result → exporting data

What you already have: circuits from C&S (KVL, KCL, sources, R, C), and the idea that a graph of one quantity against another describes a device.

Three terms first:

- **Simulator:** a program that solves the circuit's KVL/KCL equations for you, using a mathematical **model** of each part.
- **Model:** the set of numbers that describes a real part. For example, "BC547B" tells LTspice the β, the junction capacitances, and so on.
- **Directive:** a text command starting with a dot (`.dc` , `.ac` , `.tran` , `.op` , `.step`). It tells LTspice *which* analysis to run.

MANUAL EXP 2, FIG 2.1

The circuit has `.dc V1 0 20 1` written under it. What does that line make LTspice do?

GUESS FIRST

Guess what each of the four words means: `.dc` , `V1` , `0 20` , `1` .

The four analyses (every software experiment uses one or more)

Directive	What it does	Gives you	Used in
`.op`	finds the DC operating point (all capacitors open)	one number per node	Exp 2 (load), checking a Q-point
`.dc V1 0 20 1`	sweeps source V1 from 0 to 20 V in 1 V steps, solving at each step	a curve: anything against V1	Exps 2, 5, 9
`.ac dec 10 200 1Meg`	small-signal sine at each frequency, 10 points per decade, 200 Hz to 1 MHz	gain (dB) and phase against frequency	Exps 7, 8, 10, extra
`.tran 10m`	runs time forward for 10 ms	waveforms against time	Exps 4, 7, 10, 11, 12
`.step param R 100 1100 100`	reruns the analysis for R = 100, 200, … 1100	a family of curves	Exp 2 (load)

So `.dc V1 0 20 1` = "DC sweep of V1, start 0, stop 20, step 1". A **nested** sweep has a second source at the end: `.dc Vce 0 12 0.05 Ib 0 60u 20u` sweeps V_{ce} for each of I_b = 0, 20, 40, 60 µA, which gives one curve per I_b.

Units: the trap that costs marks

SPICE ignores case, so `M` **means milli, not mega.**

You want	Type	Not
1 MΩ	`1Meg`	~~`1M`~~ (that is 1 mΩ)
10 µF	`10u`	
0.5 nF	`0.5n` or `500p`	
2.2 kΩ	`2.2k`	

Drawing a circuit

DO IT: DRAW, SET, RUN

1. **File → New Schematic.**
2. Place parts using the keyboard: **R** resistor, **C** capacitor, **D** diode, **G** ground. **F2** opens the full component list (`voltage`, `npn`, `njf`, `nmos`, `zener`).
3. **F3** draws a wire: click at the start, click at each corner, right-click to stop. **Ctrl+R** rotates a part before you place it.
4. **Every circuit needs a ground (G).** Without one, the run fails with a "singular matrix" or "floating node" error.
5. **Right-click a part** to set its value (e.g. `2.2k`).
6. To choose a **model** (BC547B, 2N3819, 1N914, BZX84C6V2L, FDR6580, 2N2222, 2N5434), right-click the diode or transistor and click **Pick New ...**. All the manual's parts are in LTspice's library.
7. To set up a **source**, right-click it and click **Advanced**: - **SINE**: DC offset, amplitude, frequency. `SINE(0 5 1k)` = 0 V offset, 5 V peak, 1 kHz. - **Small signal AC amplitude**: type `1` (or `0.5`) here for an `.ac` run. - **PULSE**: V1, V2, delay, rise, fall, on-time, period, cycles.
8. To add a directive, press **S** (or **Edit → SPICE Directive**), type it, and place the text anywhere on the sheet. Alternatively, **Simulate → Edit Simulation Cmd** fills the directive in for you.
9. **F4** puts a label on a wire. Name the output `Vo` so it is easy to plot.
10. Click **Run** (the running-man icon).

Reading the result

You want	Do this
a node voltage	after the run, click on the wire (the probe turns into a red voltage probe)
a current	click on the part's body (the probe turns into a current clamp)
a different x-axis (e.g. V(base) instead of V_{be})	right-click the x-axis, then set **Quantity plotted** to `V(base)`
an exact value	click the trace's name above the plot. A cursor appears, and its readout box shows x and y. A second click adds cursor 2; the box then also shows the difference.
gain in plain ratio, not dB	right-click the y-axis and choose **Linear**
y-axis limits	right-click the y-axis and set **Manual limits**

In an `.ac` plot, the **solid line is the gain in dB** and the **dashed line is the phase**.

OBSERVATION TABLES FROM A SIMULATION

The manual says: "Paste data points from simulations". With the plot window active, use **File → Export data as text**. The file has one row per point. Copy only the rows the table asks for (5–10 points), and mark on the graph the points used in calculations.

TRAP: THE DEFAULT TIME STEP

`.tran 20m` lets LTspice pick its own step, which can be coarse. Edges then look slanted, or tall false spikes appear. Write `.tran 0 20m 0 1u` instead: the fourth number caps the step at 1 µs.

CHECK 1

(a) Write the directive that sweeps V_{gs} from −5 V to 0 V in 10 mV steps. (b) Your 1 MΩ resistor shows `1M`. What value is LTspice actually using? (c) Which analysis gives a frequency response?

Answers

Check 1. (a) `.dc Vgs -5 0 0.01` (or `10m`). (b) 1 milliohm, a short circuit. Write `1Meg`. (c) `.ac`, with the source's AC amplitude set.

Opening question. `.dc` = DC sweep, `V1` = the source being swept, `0 20` = from 0 V to 20 V, `1` = in 1 V steps. LTspice solves the circuit 21 times and plots any node against V1.

Exp 0 · Measuring DC and AC: power supply, function generator, DSO, multimeter

Manual pp. 4–6 · hands-on · not assessed on its own, but every hands-on experiment (1, 3, 6, 7) depends on it

0 Instruments → 1 Zener curve → 2 Regulator → 3 Clipper → 4 Clamper

What you already have: voltage, current, and a sine wave $v(t) = V_p \sin(2\pi ft)$ from C&S.

Three terms first:

- **Peak V_p:** the sine's height above its centre line.
- **Peak-to-peak V_{pp}:** top to bottom, $= 2V_p$.
- **RMS:** the steady DC value that would heat a resistor equally. For a sine, $V_{rms} = V_p/\sqrt{2}$.

MANUAL EXP 0, AIM

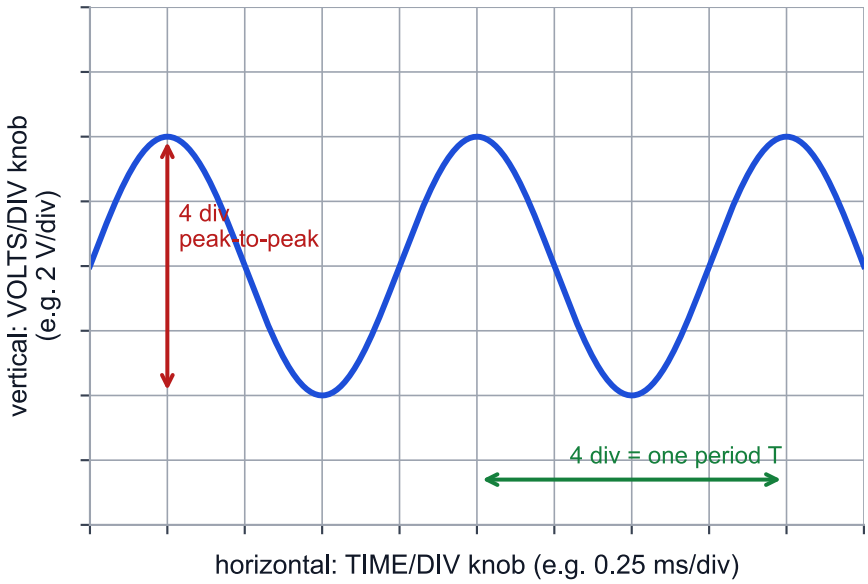
Generation and measurement of the DC voltage and AC signals. Viva form: "The DSO shows 4 divisions peak-to-peak at 2 V/div, and one cycle spans 4 divisions at 0.25 ms/div. Give V_{pp}, V_{rms} and f."

GUESS FIRST

Write your three numbers before reading on.

The four instruments

Instrument	What it does	The knobs that matter
DC power supply	gives a steady voltage	voltage knob; **current-limit** knob (set it low to protect the circuit); + and − terminals
Function generator	gives a sine, square or triangle wave	waveform buttons, frequency range + fine knob, **amplitude**, **DC offset** (keep at 0 unless asked)
DSO (digital storage oscilloscope)	draws voltage against time	**VOLTS/DIV** (vertical scale), **TIME/DIV** (horizontal scale), **coupling DC/AC**, trigger, **Auto-set**, **Measure** menu
DMM (digital multimeter)	gives one number: V, A or Ω	knob position; black lead in **COM**; red lead in **VΩmA** (or **10A** for currents above 200 mA)



Reading a DSO screen: count divisions, then multiply by the knob setting.

Reading the screen (the answer to the question)

- **Volts:** V_{pp} = (divisions top to bottom) × (VOLTS/DIV) = 4 × 2 = **8 V**. So V_p = 4 V.
- **RMS of a sine:** $V_{rms} = V_p/\sqrt{2} = 4/1.414 =$ **2.83 V**. This is what a DMM on AC volts should show.
- **Time:** T = (divisions per cycle) × (TIME/DIV) = 4 × 0.25 ms = 1 ms, so **f = 1/T = 1 kHz**.

Wave	V_{rms}
sine	$V_p/\sqrt{2} = 0.707\ V_p$
triangle	$V_p/\sqrt{3} = 0.577\ V_p$
square	V_p

TRAP: THE MANUAL'S RMS FORMULA

The manual writes $V_{rms} = V_p/\sqrt{2}$ but has you read "V_p" as **peak-to-peak**. Divide **peak** by √2, or peak-to-peak by $2\sqrt{2}$. Using $V_{pp}/\sqrt{2}$ gives an RMS twice too large.

TRAP: THE METER AND THE WAVE SHAPE

A basic DMM is calibrated for **sine waves**. On a square or triangle wave it can read wrong unless it says "True RMS". Basic meters also lose accuracy at higher frequencies, so keep the generator at about 1 kHz or lower for the meter comparison.

DO IT (HANDS-ON)

DC:

1. Set the supply to a value (e.g. 5 V), with the current limit low.
2. Connect the DSO probe tip to + and the probe's ground clip to −. Coupling **DC**.
3. Count the divisions the trace rises from the 0 V line, and multiply by VOLTS/DIV.
4. Read the same output with the DMM on DC volts.

AC:

1. Connect the generator output to the DSO (probe tip to signal, ground clip to ground).
2. Press Auto-set, or set VOLTS/DIV and TIME/DIV so that 1–2 cycles fill the screen.
3. Count V_{pp} divisions and the divisions in one period T. Calculate V_{rms} and f.
4. Measure the same output with the DMM on AC volts.
5. Repeat for the square and triangle waves.

WRITE IN YOUR RECORD

Observation table columns: wave shape · divisions for V_{pp} · VOLTS/DIV · V_{pp} · divisions for T · TIME/DIV · T · f = 1/T · V_{rms} (calculated) · V_{rms} (DMM).

Result: average error of the DSO reading = (|DMM − DSO| / DMM) × 100 averaged over the readings.

Theory for the record: a function generator produces sine, square and triangle voltages of set amplitude and frequency. A DSO samples the input, stores it digitally and draws voltage against time on a grid. Amplitude = divisions × VOLTS/DIV, and period = divisions × TIME/DIV, with f = 1/T. A DMM measures DC/AC voltage, current and resistance. On AC it displays RMS, which for a sine equals $V_{peak}/\sqrt{2}$. A regulated DC supply holds its output constant against changes in load and line.

CHECK 1

(a) A triangle wave is 6 V peak-to-peak. What is its RMS? (b) The probe's ground clip is left unconnected. What does the trace typically look like? (c) Why must the DMM be on the mA port, and in series, to measure a current?

Answers

Opening question. V_{pp} = 8 V, V_{rms} = 2.83 V, f = 1 kHz.

Check 1. (a) V_p = 3 V, so V_{rms} = 3/√3 = **1.73 V**. (b) A noisy, fuzzy trace (often showing 50 Hz mains pickup), because the probe has no reference point. (c) An ammeter must carry the same current as the branch, so it goes in series. It has a very low resistance, so it must never be placed across a source: that is a short circuit, and it blows the meter's fuse.

Exp 1 · Zener diode characteristics and dynamic resistance

Manual pp. 7–8 · hands-on · CO1, CO2 · theory: ECE2101 unit 2 (pn junctions)

0 Instruments → **1 Zener curve** → 2 Regulator → 3 Clipper → 4 Clamper

What you already have: Ohm's law, and the meters from Exp 0.

Three terms first:

- **Diode:** a p-type and an n-type piece of silicon joined together (a *pn junction*). The p side is the **anode** and the n side is the **cathode** (the bar on the symbol). Current flows easily from anode to cathode.
- **Forward / reverse bias:** anode more positive than cathode = forward. The reverse = reverse.
- **Breakdown:** under enough reverse voltage, a junction suddenly conducts heavily.

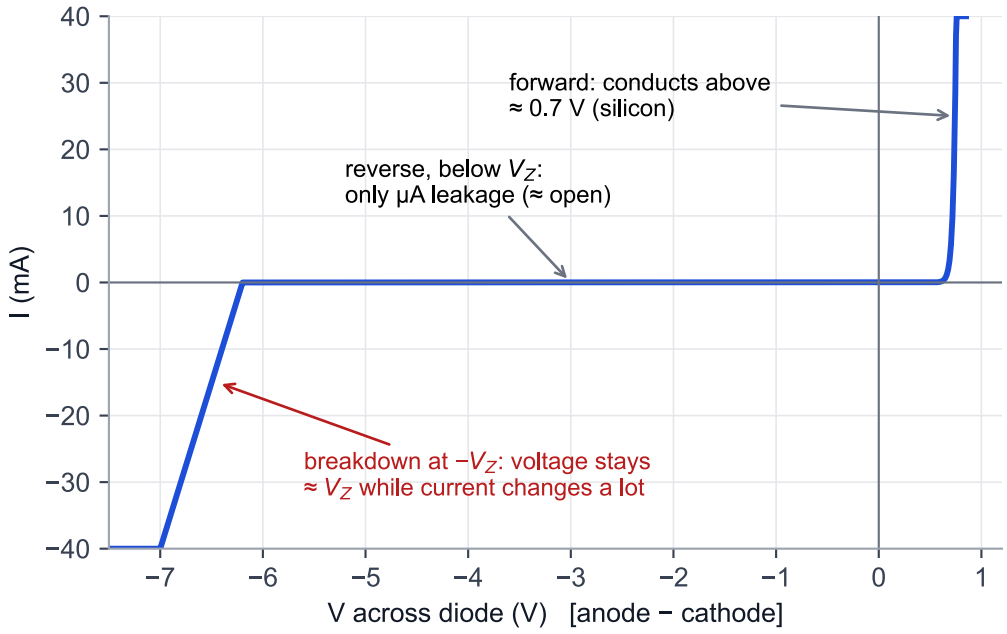
MANUAL EXP 1, AIM

To plot the characteristics of a zener diode and find its dynamic resistance under reverse-biased condition. Viva form: "Why does a zener's reverse current rise sharply at one voltage, and what is its *dynamic* resistance?"

GUESS FIRST

Sketch I against V for an ordinary diode, both directions, before reading on. Where does it conduct?

The idea: a diode that is meant to break down



One curve, three regions. A zener is built so the left-hand breakdown happens at a chosen, sharp voltage V_Z.

1. **Forward:** almost no current until about 0.6–0.7 V (silicon), then current rises steeply. This is the same as any diode.
2. **Reverse, below V_Z:** only a tiny leakage current (μA). The diode is close to an open circuit.
3. **Reverse, at V_Z:** the junction **breaks down**. The current can rise from 1 mA to 10 mA while the voltage barely moves from V_Z.

Region 3 is what makes a zener useful: **a voltage that stays put while the current changes**, which is a voltage reference. Exp 2 uses exactly that.

Why breakdown happens (viva staple; from ECE2101 unit 2):

	Zener breakdown	Avalanche breakdown
doping	heavy, so the junction is thin	light, so the junction is wide
mechanism	the field pulls electrons straight across (tunnelling)	fast carriers knock out new ones (chain reaction)
V_Z	below about 5 V	above about 6 V
temperature coefficient	negative	positive

A 6.2 V zener is near the crossover and uses both mechanisms. That is why ~5–6 V zeners drift least with temperature. "Zener diode" is the trade name for any breakdown diode, whichever mechanism it uses.

Dynamic resistance r_z: in breakdown, how much the voltage moves per unit change of current:

$$r_z = \frac{\Delta V_Z}{\Delta I_Z}$$

A small r_z (tens of ohms or less) means a better voltage reference. It is called *dynamic* because it is the slope of the curve, not V/I at one point. V_Z/I_Z would be a static resistance, and a meaningless one here.

Limits:

- **$I_{Z(min)}$:** the least current that keeps the zener in breakdown.
- **$I_{Z(max)} = P_Z/V_Z$:** the largest current before it overheats. A 0.5 W, 6.2 V zener allows $I_{Z(max)} = 0.5/6.2 = 81$ mA.

The series 1 kΩ resistor in the circuit limits the current.

The circuit

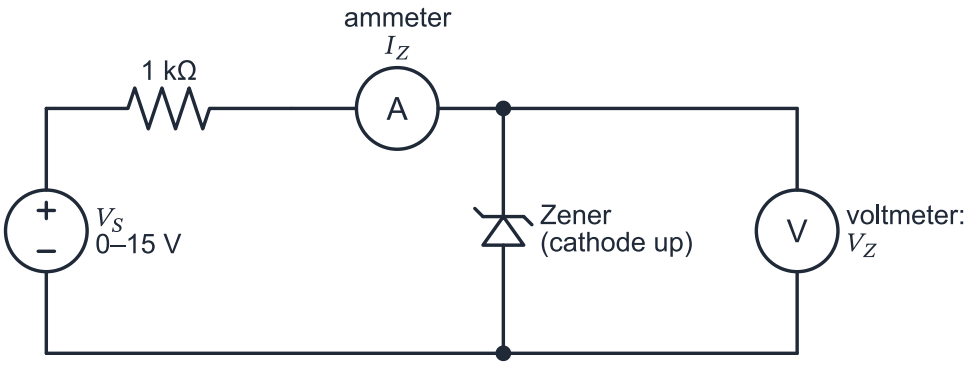


Fig 1.1, redrawn. The zener's cathode faces the + side, so it is reverse-biased. The ammeter is in series; the voltmeter (or DSO) is across the zener.

TRAP: THE MANUAL'S PARTS LIST

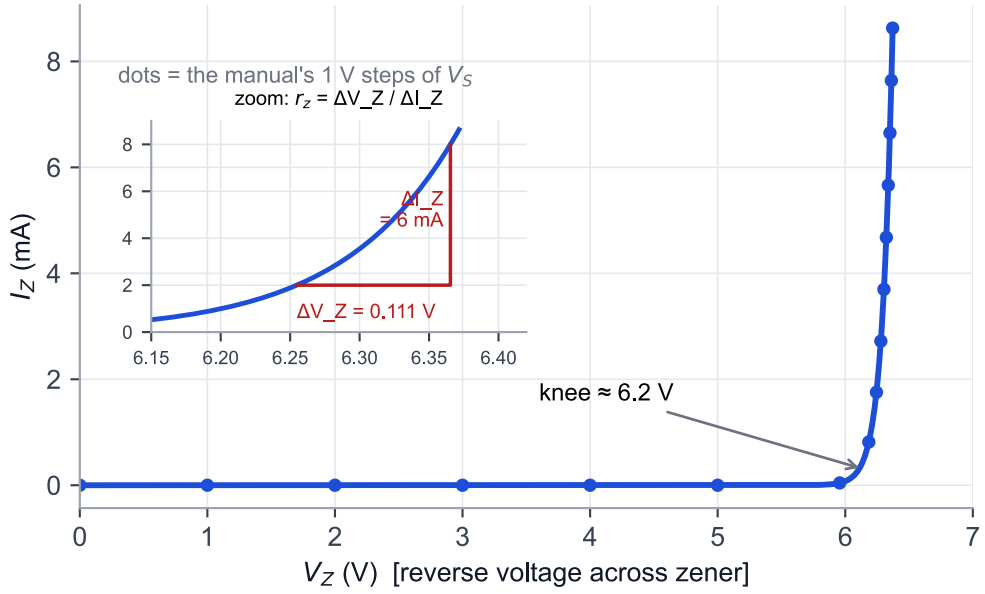
It lists a **1N4007**. That is an ordinary rectifier with a breakdown near 1000 V, so it will never break down on a 0–30 V supply. Ask for a **zener** (e.g. 6.2 V). A 1N4007 placed reverse-biased just shows µA leakage at every V_S.

DO IT (HANDS-ON, BREADBOARD)

1. Supply **off**, voltage knob at 0, current limit low.
2. Wire the 1 kΩ from supply + to the ammeter. Ammeter (DMM on mA DC, red lead in the mA port) to the zener's **cathode** (the banded end). Zener's anode to supply −.
3. Put the voltmeter (DMM on DC V, or the DSO) across the zener.
4. Switch on. Raise V_S in **1 V steps from 0 to 15 V**, reading V_Z and I_Z at each step.
5. Plot I_Z (y) against V_Z (x). Mark the knee: that is V_Z.
6. Take two points well into breakdown, read ΔV_Z and ΔI_Z, and calculate r_z.

What you should see

Simulated with a 6.2 V zener (BZX84C6V2L, the part Exp 2 uses). A different zener shifts the knee to its own V_Z, but the shape is identical.



LTspice, 1 kΩ series, V_S 0–15 V. Flat (open) until about 6 V, then nearly vertical. The zoom shows the r_z triangle.

V_S (V)	0–5	6	7	8	9	10	11	12	13	14	15
V_Z (V)	= V_S	5.957	6.185	6.245	6.279	6.303	6.322	6.337	6.350	6.362	6.372
I_Z (mA)	≈ 0 (µA)	0.04	0.82	1.76	2.72	3.70	4.68	5.66	6.65	7.64	8.63

Below breakdown, the zener takes almost no current, so no voltage drops across the 1 kΩ and $V_Z = V_S$. In breakdown, V_Z is pinned and the rest of V_S drops across the resistor: $I_Z = (V_S - V_Z)/1\ k\Omega$. Check: $V_S = 15$ gives $(15 - 6.372)/1000 = 8.63$ mA ✓.

Calculation

Using the 10 V and 15 V rows (both well into breakdown):

$$r_z = \frac{6.372 - 6.303}{8.63 - 3.70\ \text{mA}} = \frac{0.069\ \text{V}}{4.93\ \text{mA}} \approx 14\ \Omega$$

The zoom figure uses 2 → 8 mA and gives 0.111 V / 6 mA = 18 Ω. r_z shrinks as the current grows, so always **state the current range used**.

WRITE IN YOUR RECORD

Result: breakdown voltage $V_Z \approx$ **6.2 V** (at $I_Z \approx 1$ mA); dynamic resistance $r_z \approx$ **14 Ω** (between 3.7 and 8.6 mA). Use your own readings in the lab.

Theory for the record: a zener is a heavily doped pn diode designed to operate in reverse breakdown. Forward, it behaves like a normal diode (about 0.7 V). Reverse, only leakage flows until V_Z, where the current rises sharply while the voltage stays nearly constant. Breakdown is by tunnelling (below ~5 V) or avalanche (above ~6 V). Operation must stay between $I_{Z(min)}$ and $I_{Z(max)} = P_Z/V_Z$. The slope $\Delta V_Z/\Delta I_Z$ in breakdown is the dynamic resistance; a small value means good regulation, which Exp 2 uses.

TRAP: NO SERIES RESISTOR

Without the 1 kΩ (or with it shorted), the current in breakdown is limited only by the supply, and the zener burns out. Never connect a zener directly across a supply.

CHECK 1

(a) At $V_S = 12$ V, a student reads $V_Z = 6.34$ V. Predict I_Z without the ammeter. (b) Why is r_z not simply V_Z/I_Z? (c) A 3.3 V zener and a 12 V zener: which mechanism does each use, and which way does each one's V_Z move when heated?

Answers

Opening question. Reverse current rises sharply at V_Z because the junction breaks down (tunnelling or avalanche), producing many carriers. Dynamic resistance = $\Delta V_Z/\Delta I_Z$, the slope in breakdown. It is small (≈ 14–18 Ω here).

Check 1. (a) I_Z = (12 − 6.34)/1 kΩ = **5.66 mA**. (b) r_z describes how V changes when I changes (the slope), which is what matters for regulation. V_Z/I_Z at one point is a large, current-dependent number with no use here. (c) 3.3 V: Zener (tunnelling) mechanism, V_Z falls with heat (negative coefficient). 12 V: avalanche, V_Z rises with heat (positive coefficient).

Exp 2 · Zener diode as a voltage regulator: line and load regulation

Manual pp. 9–12 · LTspice / Virtual Lab · CO3 · the manual's printed data matches the simulation below digit-for-digit

What you already have: from Exp 1, in breakdown V_Z stays ≈ 6.2 V while I_Z changes. From C&S, voltage dividers and KCL.

Three terms first:

- **Regulator:** a circuit that keeps V_o constant.
- **Line regulation:** how much V_o moves when the **input** V_{in} changes. The "line" is the supply line.
- **Load regulation:** how much V_o moves when the **load current** changes, i.e. when R_L changes.

MANUAL EXP 2, AIM

To use a zener diode for line and load regulation. Viva form: "Input rises from 14 V to 17 V and V_o rises by only 0.08 V. Where did the other 2.92 V go?"

GUESS FIRST

Answer before reading on. Hint: there are only two other parts, R1 and the zener.

How a zener regulates

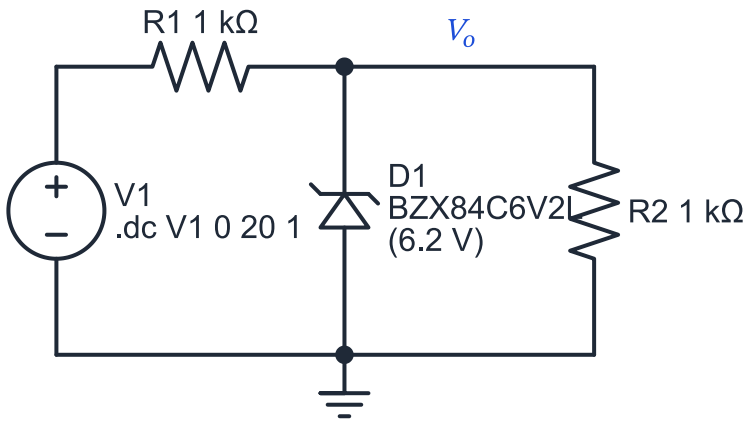


Fig 2.1 (line regulation): V1 swept 0→20 V, with a fixed 1 kΩ load.

KCL at the output node: the current through R1 splits between the zener and the load.

$$I_{R1} = \frac{V_{in} - V_Z}{R_1} = I_Z + I_L$$

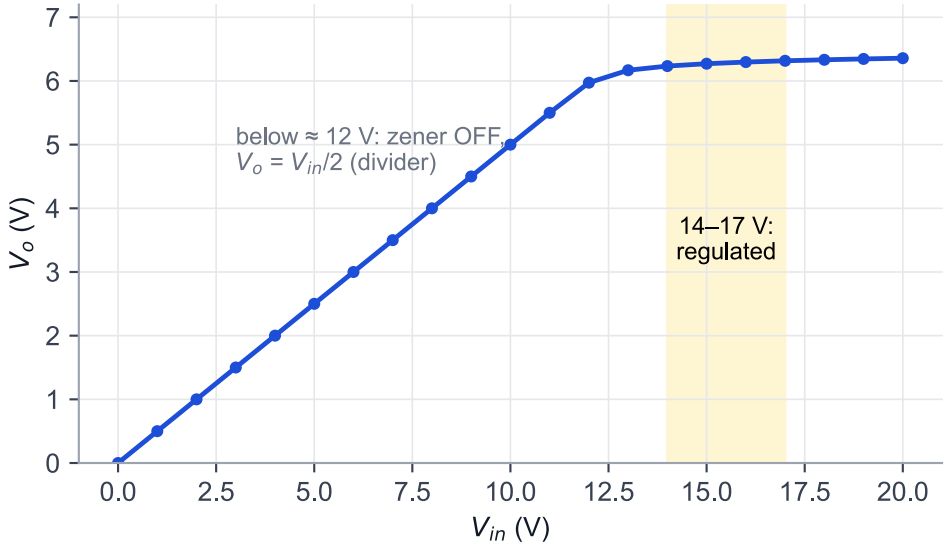
- **V_{in} rises:** the extra current all goes into the **zener**. V_Z barely moves (small r_z), so V_o stays put and **R1 absorbs the extra voltage**. That is the answer to the question.
- **R_L falls (more load current):** the zener gives up current to the load. V_o holds **as long as I_Z stays above $I_{Z(min)}$**.
- If the load demands more than R1 can supply, I_Z hits zero, the zener switches off, and V_o becomes an ordinary divider output that collapses.

Because the zener sits in parallel (shunt) with the load, this is called a **shunt regulator**.

Line regulation (Fig 2.1)

DO IT (LTSPICE)

1. Draw V1, R1 = 1k, zener **BZX84C6V2L** (cathode to the output node), R2 = 1k load, and ground.
2. Label the output wire `vo` (F4).
3. Directive: `.dc V1 0 20 1`. Run.
4. Plot V(vo) against V1. Use cursors to read V_o at V_{in} = 14 V and 17 V.



Two regions. Below about 12 V the zener is off and the output is half the input (1 kΩ:1 kΩ divider). Above it, V_o is pinned near 6.2–6.36 V.

Why 12 V? With the zener off, $V_o = V_{in}/2$. The zener turns on when $V_{in}/2$ reaches about 6.2 V, i.e. at $V_{in} \approx 12.4$ V.

V_{in} (V)	10	11	12	13	14	15	16	17	18	20
V_o (V)	5.000	5.500	5.973	6.168	**6.234**	6.271	6.297	**6.317**	6.333	6.358

$$\% \text{ line regulation} = \frac{\Delta V_o}{\Delta V_{in}} \times 100 = \frac{6.317 - 6.234}{17 - 14} \times 100 = \frac{0.083}{3} \times 100 = 2.8\ \%$$

Load regulation (Fig 2.2)

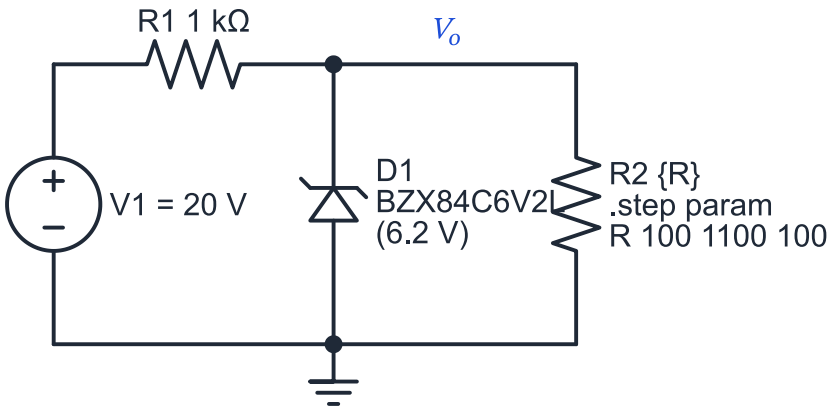
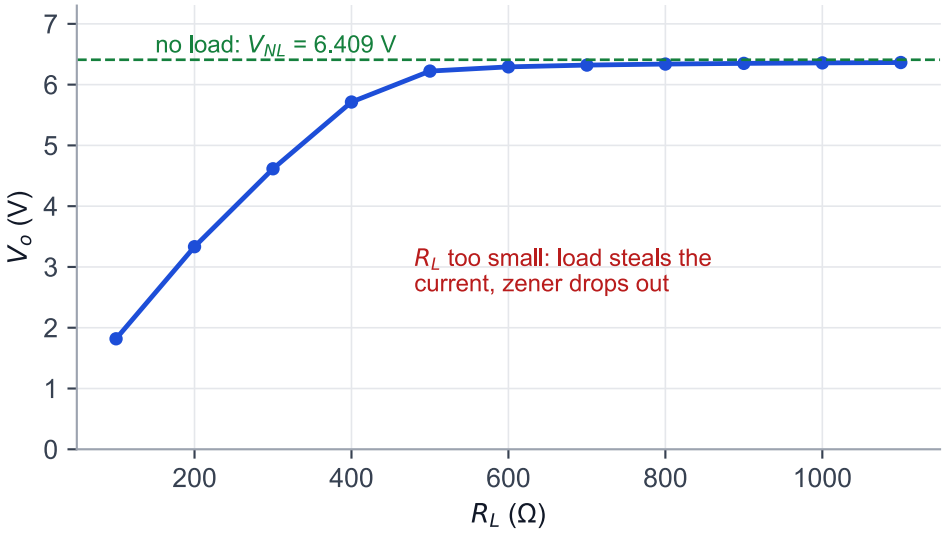


Fig 2.2 (load regulation): V1 fixed at 20 V, load stepped 100 Ω → 1100 Ω.

DO IT (LTSPICE)

1. Same circuit, but V1 = 20 V and the load value written as `{R}` (curly braces = a parameter).
2. Directives: `.step param R 100 1100 100` and `.op`. Run.
3. Plot V(vo). The x-axis is now R.
4. **No load:** delete R2, run `.op` again, and read V_{NL}.
5. Choose a full-load point (e.g. R = 1 kΩ) and read V_{FL}.



Above about 450 Ω the output holds near 6.3 V. Below that, the load takes more current than R1 can supply.

R_L (Ω)	100	200	300	400	500	600	800	1000	1100	no load
V_o (V)	1.818	3.333	4.615	5.714	6.223	6.292	6.338	**6.358**	6.364	**6.409**

$$\% \text{ load regulation} = \frac{V_{NL} - V_{FL}}{V_{FL}} \times 100 = \frac{6.409 - 6.358}{6.358} \times 100 = 0.81\ \%$$

Smallest load that still regulates: R1 can pass at most (20 − 6.2)/1 kΩ = 13.8 mA. The load gets all of that only when $I_Z \to 0$, so $R_{L(min)}$ = 6.2 V / 13.8 mA ≈ **450 Ω**. This matches the knee in the plot. Below 450 Ω the zener is off, and $V_o = 20 \times R_L/(1000 + R_L)$: at 100 Ω that is 1.82 V ✓.

WRITE IN YOUR RECORD

Result: % line regulation = **2.8 %** (14–17 V, R_L = 1 kΩ); % load regulation = **0.81 %** (no load → 1 kΩ, V_{in} = 20 V). Lower % = better regulation.

Theory for the record: a zener shunt regulator places a reverse-biased zener across the load, fed through a series resistor R_S. In breakdown, the zener voltage is nearly constant, so changes in input voltage appear across R_S, and changes in load current are absorbed by a change in zener current. Regulation holds while $I_{Z(min)} < I_Z < I_{Z(max)}$. R_S is chosen so that $I_Z \geq I_{Z(min)}$ at minimum input and maximum load. Line regulation = $\Delta V_o/\Delta V_{in} \times 100$ %; load regulation = $(V_{NL} - V_{FL})/V_{FL} \times 100$ %.

TRAP: AN "UNREGULATED" LINE PLOT

If V_o keeps rising with slope ½ all the way to 20 V, the zener is in backwards (forward-biased, clamping at about 0.7 V) or is the wrong part. If V_o sticks at about 0.7 V, it is definitely reversed. The cathode (bar) must face the output node.

CHECK 1

(a) Why does V_o in the line plot still creep up slightly (6.23 → 6.36 V) instead of staying perfectly flat? (b) With V_{in} = 20 V and R_L = 1 kΩ, how much current does the zener carry? (c) What happens to I_Z when the load is removed, and why is that the worst case for zener power?

Answers

Opening question. Across R1. The extra input raises the current through R1 (it flows on into the zener), so R1's voltage drop rises by about 2.92 V.

Check 1. (a) The zener has a dynamic resistance r_z (Exp 1). More current through it means slightly more voltage: $\Delta V = r_z \Delta I_Z$. (b) $I_{R1} = (20 - 6.358)/1000 = 13.64$ mA, $I_L = 6.358/1000 = 6.36$ mA, so $I_Z = 13.64 - 6.36 =$ **7.28 mA**. (c) All of R1's current then flows through the zener, its maximum, so I_Z and $P_Z = V_Z I_Z$ peak. Design must keep this below $I_{Z(max)}$.

Exp 3 · Diode clipper circuits

Manual pp. 13–14 · hands-on · CO3

0 Instruments → 1 Zener curve → 2 Regulator → **3 Clipper** → 4 Clamper

What you already have: a silicon diode conducts once its anode is about 0.7 V above its cathode (Exp 1, forward region). Below that it is an open circuit.

Three terms first:

- **Clipper:** a circuit that cuts off the part of a waveform above (or below) a set level.
- **Clipping level:** the voltage at which the cut happens.
- **Cut-in voltage V_γ:** the forward voltage at which a diode starts to conduct (≈ 0.7 V for silicon).

MANUAL EXP 3, AIM

To implement a diode clipper circuit and observe its output waveform. Viva form: "A 10 V-peak sine feeds Fig 3.1(a). At what level is it clipped, and which half is affected?"

GUESS FIRST

Look at the circuit below and commit to a number and a half (top or bottom) before reading on.

How it works

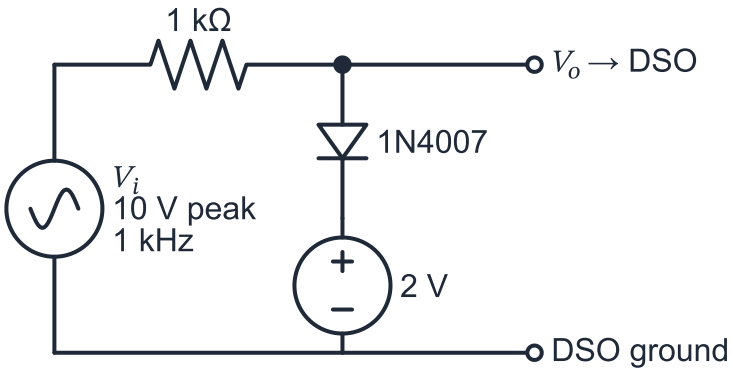


Fig 3.1(a): diode anode at V_o, cathode at the + of a 2 V battery.

Treat the diode as a switch, and ask when its anode is more than 0.7 V above its cathode.

- The cathode sits at **+2 V** (the battery).
- The diode conducts when $V_o > 2 + 0.7 =$ **2.7 V**. Then V_o is held at 2.7 V, and the rest of V_{in} drops across the 1 kΩ.
- When $V_{in} < 2.7$ V the diode is off: no current flows, so there is no drop across the 1 kΩ, and $\mathbf{V_o = V_{in}}$.

Result: the **top** of the sine is cut flat at +2.7 V, and the bottom passes untouched.

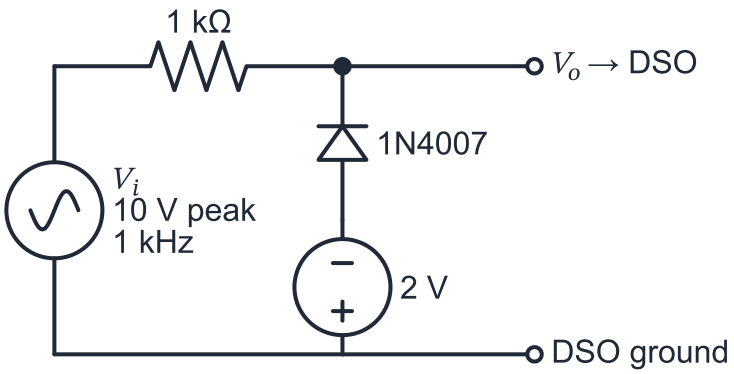
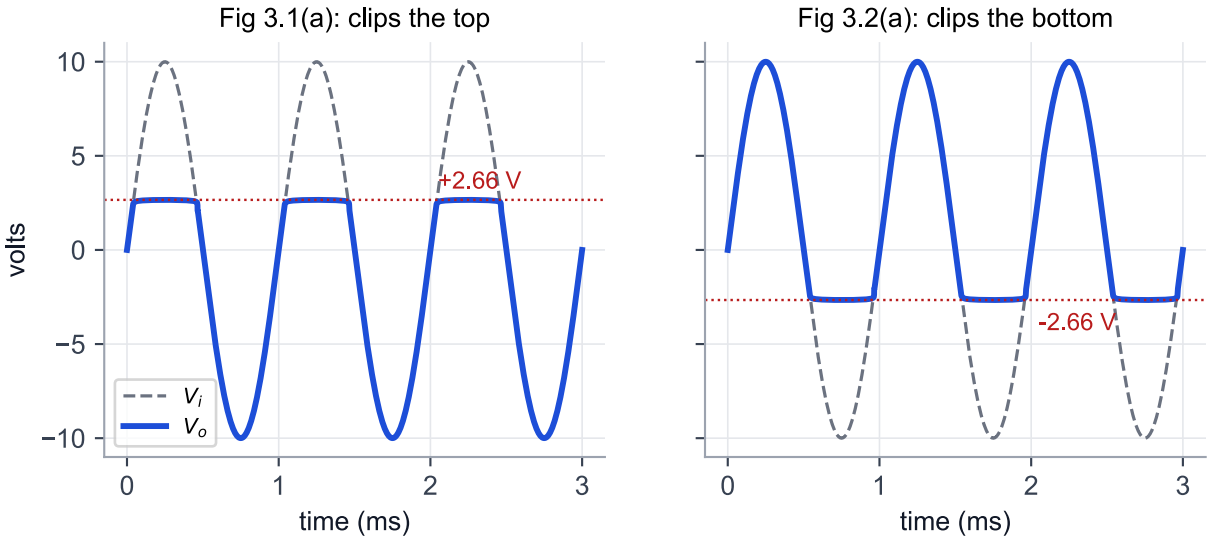


Fig 3.2(a): diode and battery both reversed.

Here the diode's anode is at −2 V. It conducts when $V_o < -2.7$ V, so the **bottom** is clipped at −2.7 V.



LTspice with a 1N4007: clipping at ±2.66 V (this diode drops 0.66 V at the clipping current).

Transfer characteristic (V_o plotted against V_i, manual Fig 3.3): a 45° line ($V_o = V_i$) that turns flat at +2.7 V for (a), or at −2.7 V for (b).

DO IT (HANDS-ON)

1. Generator: sine, **10 V peak** (20 V p-p on the DSO), 1 kHz. Check it on the DSO **before** connecting the circuit.
2. Breadboard: generator → 1 kΩ → output node. From the output node, connect the diode with its **anode at the output** (band/cathode away from the output), then its cathode to the **+** of the 2 V supply. Supply **−** goes to the generator's ground.
3. DSO CH1 on V_{in} and CH2 on V_o, both grounds at circuit ground. Coupling **DC** (AC coupling shifts the trace and hides the true level).
4. Read the clipping level: divisions × VOLTS/DIV.
5. Swap the diode **and** the supply polarity to build Fig 3.2(a). Repeat.
6. Optional: DSO **X-Y mode** (CH1 = x, CH2 = y) draws the transfer characteristic directly.

WRITE IN YOUR RECORD

Result: clipping level ≈ **+2.7 V** for Fig 3.1(a) (positive clipper) and **−2.7 V** for Fig 3.2(a) (negative clipper). Measured with the 1N4007: about ±2.66 V.

Theory for the record: a clipper removes the part of a signal beyond a chosen level without disturbing the rest. A diode in series with a DC reference V_R, placed across the output, conducts when the output exceeds $V_R + V_\gamma$, holding the output at that level. Otherwise the diode is off and $V_o = V_i$. Reversing the diode and the reference clips the opposite half. The series resistor limits the current and drops the excess voltage. Clippers are used for wave shaping and to protect inputs from overvoltage.

TRAP: THE DSO ON AC COUPLING

AC coupling removes the DC average, so the flat top no longer reads +2.7 V. Use DC coupling for any clipping-level or clamping measurement.

CHECK 1

(a) Change the battery in Fig 3.1(a) to 5 V. New clipping level? (b) Remove the battery (short it) in Fig 3.1(a). New clipping level? (c) Why must the series 1 kΩ be there at all?

Answers

Opening question. At about +2.7 V (2 V battery + 0.7 V diode). The **positive** half is clipped.

Check 1. (a) 5 + 0.7 = **+5.7 V**. (b) **+0.7 V**: a plain diode clipper. (c) While the diode conducts, the output is pinned. The difference V_{in} − 2.7 V must drop somewhere, and the resistor takes it. Without it, the generator would drive a large current straight into the diode and battery.

Exp 4 · Diode clamper circuit

Manual pp. 15–16 · LTspice · CO3 · the manual's sample output minimum (−9.38 V) matches the simulation below

0 Instruments → 1 Zener curve → 2 Regulator → 3 Clipper → **4 Clamper**

What you already have: the clipper from Exp 3, which **cuts** a wave. From C&S, a capacitor keeps its charge unless current flows, and τ = RC sets how fast it discharges.

Two terms first:

- **Clamper (DC restorer):** a circuit that **shifts** the whole wave up or down without changing its shape. It adds a DC level.
- **Peak-to-peak preserved:** the wave keeps its full V_{pp}. Only its position moves.

MANUAL EXP 4, AIM

To implement a diode clamper circuit and observe its output waveform. Viva form: "Input is ±5 V. The output swings from about +0.5 V to −9.4 V. Explain where the −4.4 V shift comes from."

GUESS FIRST

Which part could be storing a voltage that adds to the input?

How it works

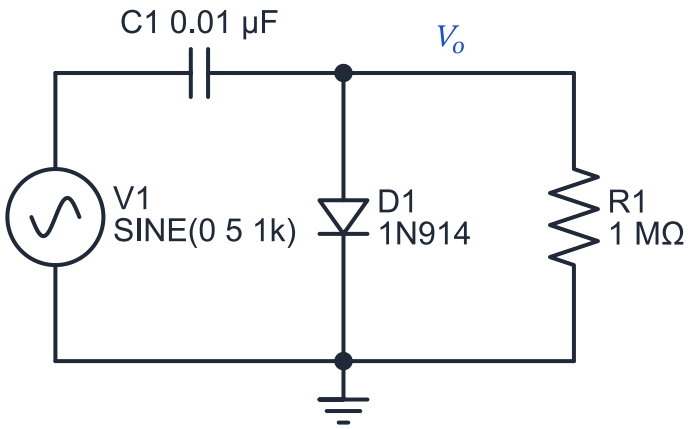
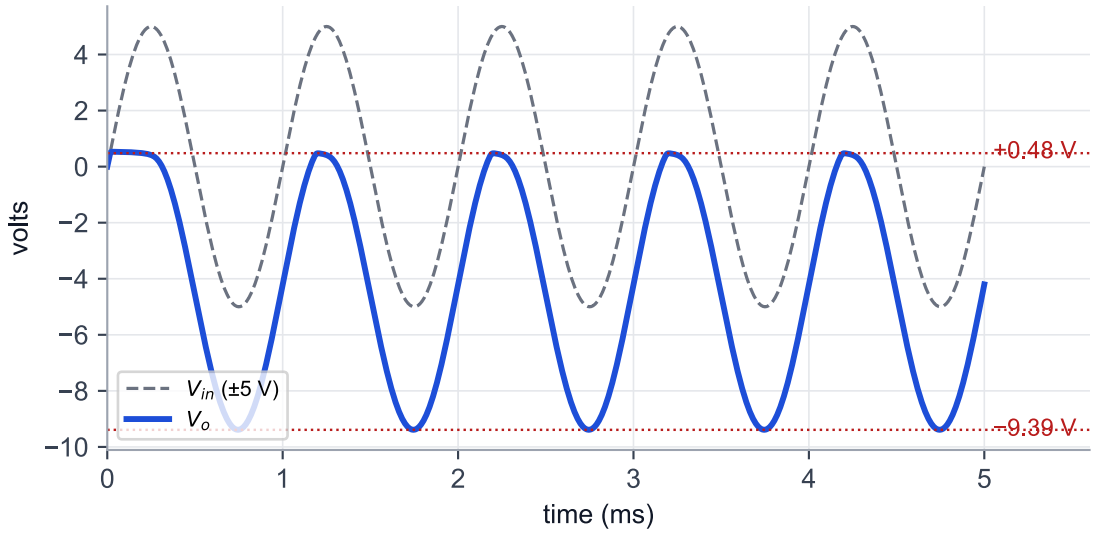


Fig 4.1: series C, diode to ground (anode at V_o), 1 MΩ load.

1. **First positive peak:** V_{in} rises, V_o tries to go positive, and the diode turns on, holding $V_o \approx +0.5$ V. The capacitor charges to about $\mathbf{V_p - 0.5 \approx 4.5\ V}$, positive on the input side.
2. **After that** the diode is off. The capacitor cannot discharge quickly, because its only path is the 1 MΩ: τ = RC = 1 MΩ × 0.01 µF = **10 ms**, which is 10× the 1 ms period. So it acts like a 4.5 V battery in series with the input:

$$V_o = V_{in} - V_C \approx V_{in} - 4.5\ V$$

So the output swings from +5 − 4.5 = **+0.5 V** to −5 − 4.5 = **−9.5 V**. Same 10 V p-p, but shifted down by the capacitor's voltage. This is a **negative clamper**: the positive peak is clamped at about 0 V, and the wave sits below it.



LTspice: V_o from +0.48 V to −9.39 V. The output is clamped in the very first cycle.

Reverse the diode (cathode at V_o): the negative peak clamps at about −0.5 V, and the wave sits above it, a **positive clamper**. **Add a battery** in series with the diode: the clamping level moves to that battery's voltage ± 0.7 V.

DO IT (LTSPICE)

1. V1: right-click, **Advanced** → **SINE**, DC offset 0, amplitude 5, frequency 1k (= `SINE(0 5 1k)`).
2. C1 = `0.01u`. Diode D1: Pick New Diode → **1N914**, with its anode on the output node and cathode to ground. R1 = `1Meg` (**not** `1M`!).
3. Directive: `.tran 0 5m 0 2u`. Run. Plot the input wire and V(vo).
4. Use cursors to read the maximum and minimum of V_o. Then V_{pp} = max − min, and the DC shift = (max + min)/2.

Quantity	LTspice value
V_o maximum (clamped level)	+0.48 V
V_o minimum	−9.39 V
V_o peak-to-peak	9.87 V (input 10 V)
average of V_o (DC shift)	−4.41 V

WRITE IN YOUR RECORD

Result: output clamped at about **+0.5 V** (positive peak); V_o swings +0.48 V to −9.39 V; peak-to-peak 9.87 V ≈ input 10 V; DC level shifted by about −4.4 V. Negative clamper.

Theory for the record: a clamper shifts a waveform's DC level without changing its shape or peak-to-peak value. A capacitor in series charges through the diode to (nearly) the input peak on the first half cycle. With the diode off, it discharges only through the load, so it must have RC ≫ T (here 10 ms ≫ 1 ms). The capacitor voltage then adds to the input, shifting the wave so that one peak sits at the diode's clamping level. Diode direction sets negative/positive clamping; a series battery sets the reference level.

TRAPS

- `1M` in LTspice is 1 **milli**ohm. With R ≈ 0, the capacitor discharges every cycle and nothing clamps.
- If RC is comparable to T, the capacitor droops between peaks and the output tilts. The rule is RC ≥ 10T (some books use 5T).

CHECK 1

(a) Clipper vs clamper: one sentence each on what happens to V_{pp}. (b) Replace 0.01 µF with 100 pF. What goes wrong, and why? (c) Diode reversed: give V_o's max and min.

Answers

Opening question. The capacitor charges to about 4.5 V on the first positive peak (through the diode). It then can't discharge (τ = 10 ms $\gg$ 1 ms), so it stays in series with the input as a −4.5 V offset.

Check 1. (a) A clipper **reduces** V_{pp} (part of the wave is cut). A clamper **keeps** V_{pp} and only shifts the DC level. (b) τ = 1 MΩ × 100 pF = 0.1 ms $\ll$ 1 ms: the capacitor discharges between peaks, so the shift is not held and the output distorts. (c) Positive clamper: min ≈ −0.5 V, max ≈ +9.5 V.

Exp 5 · BJT input/output characteristics (CE) and h-parameters

Manual pp. 17–19 · LTspice / Virtual Lab · CO1, CO2 · theory: ECE2101 unit 5 (BJTs)

5 BJT curves → 6 CE amplifier → 7 Emitter follower → 8 Feedback

What you already have: a diode is one pn junction (Exp 1). It conducts forward above ≈ 0.7 V.

Three terms first:

- **BJT (bipolar junction transistor):** three layers, **n-p-n** for an NPN. The layers are the **emitter** (E, heavily doped), the **base** (B, very thin, lightly doped) and the **collector** (C). That makes two junctions: base–emitter (BE) and base–collector (BC).
- **Common emitter (CE):** the emitter is shared by the input (B–E) and the output (C–E).
- **h-parameters:** four numbers that describe how the transistor responds to small changes. Exp 6–8 amplifiers are built on them.

MANUAL EXP 5, AIM

To plot the input and output characteristics of an NPN transistor in CE configuration and determine h-parameters from the characteristics. Viva form: "Your output curves are flat lines spaced 5.5 mA apart for 20 µA steps of base current. What is h_{fe}, and what does a flat line tell you?"

GUESS FIRST

Commit to a number for h_{fe}, and to a one-word meaning of "flat", before reading on.

The BJT in five lines

1. Forward-bias the BE junction (V_{BE} ≈ 0.6–0.7 V). The emitter injects electrons into the base.
2. The base is thin, so almost all of them cross it and are swept into the collector by the reverse-biased BC junction. A few recombine in the base: that is the base current.
3. So a **small base current controls a large collector current**: $I_C = \beta I_B$, with β ≈ 100–300 (≈ 290 for the BC547B).
4. KCL: $\mathbf{I_E = I_B + I_C}$.
5. Which region the transistor is in depends on how the two junctions are biased:

Region	BE junction	BC junction	Behaviour	Used as
Active	forward	reverse	$I_C = \beta I_B$, nearly independent of V_{CE}	amplifier (Exps 6–8, 11)
Saturation	forward	forward	V_{CE} ≈ 0.2 V, $I_C < \beta I_B$	closed switch
Cut-off	reverse	reverse	$I_C \approx 0$	open switch

The two families of curves

- **Input characteristic:** I_B against V_{BE}, at fixed V_{CE}. It is just the BE diode's forward curve.
- **Output characteristic:** I_C against V_{CE}, at fixed I_B. One curve per I_B.

h-parameters in CE form (small changes, written in lowercase):

$$v_{be} = h_{ie}\, i_b + h_{re}\, v_{ce} \qquad i_c = h_{fe}\, i_b + h_{oe}\, v_{ce}$$

Parameter	Name	Read from	Formula
h_{ie} (= h_{11})	input resistance	input curve, V_{CE} fixed	$\Delta V_{BE}/\Delta I_B$
h_{re} (= h_{12})	reverse voltage gain	input curves, I_B fixed	$\Delta V_{BE}/\Delta V_{CE}$
h_{fe} (= h_{21})	forward current gain	output curves, V_{CE} fixed	$\Delta I_C/\Delta I_B$
h_{oe} (= h_{22})	output admittance	one output curve, I_B fixed	$\Delta I_C/\Delta V_{CE}$

The manual swaps the names for h_{21} and h_{22}, and writes "$1/h_{oe} = \Delta I_C/\Delta V_{CE}$". The table above is the correct version. $\Delta I/\Delta V$ is a conductance, so h_{oe} is in siemens and $1/h_{oe}$ is the output resistance.

The circuit, and one fix

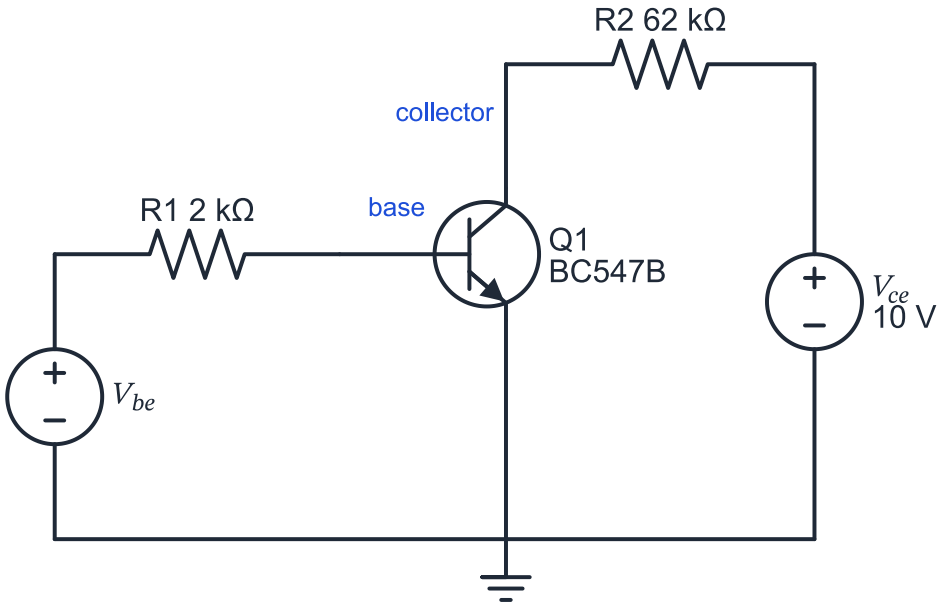


Fig 5.1 as printed: V_{be} through 2 kΩ to the base; V_{ce} through 62 kΩ to the collector.

TRAP: THE 62 KΩ MAKES THE MANUAL'S TABLE IMPOSSIBLE

With 62 kΩ in the collector, $I_C \le 10\ V / 62\ k\Omega =$ **0.16 mA**. The transistor saturates as soon as I_B passes about 0.6 µA, so "V_{CE}" is no longer the 10 V you set, and the I_B = 20/40 µA curves of the observation table can never appear. That is why the manual's own output plot only reaches about 210 µA. The fix: **replace R2 with a wire** so V_{ce} sits directly on the collector, and drive the base with a **current source** for the output curves. The device and the directives stay the same.

DO IT (LTSPICE)

Input characteristics:

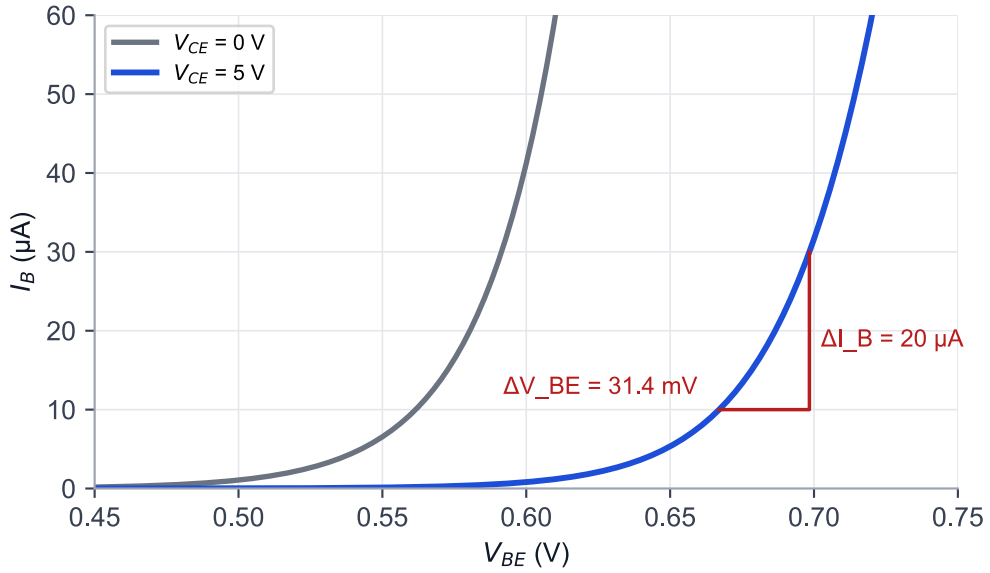
1. Draw Fig 5.1 with R2 replaced by a wire. Q1: Pick New Transistor → **BC547B**. Label the base node `base`.
2. Directive: `.dc Vbe 0 1 0.001 Vce 0 5 5` (V_{CE} = 0 V and 5 V, as the observation table asks).
3. Run, click the base lead to plot **Ib(Q1)**, then right-click the x-axis and set it to **V(base)**.

Output characteristics:

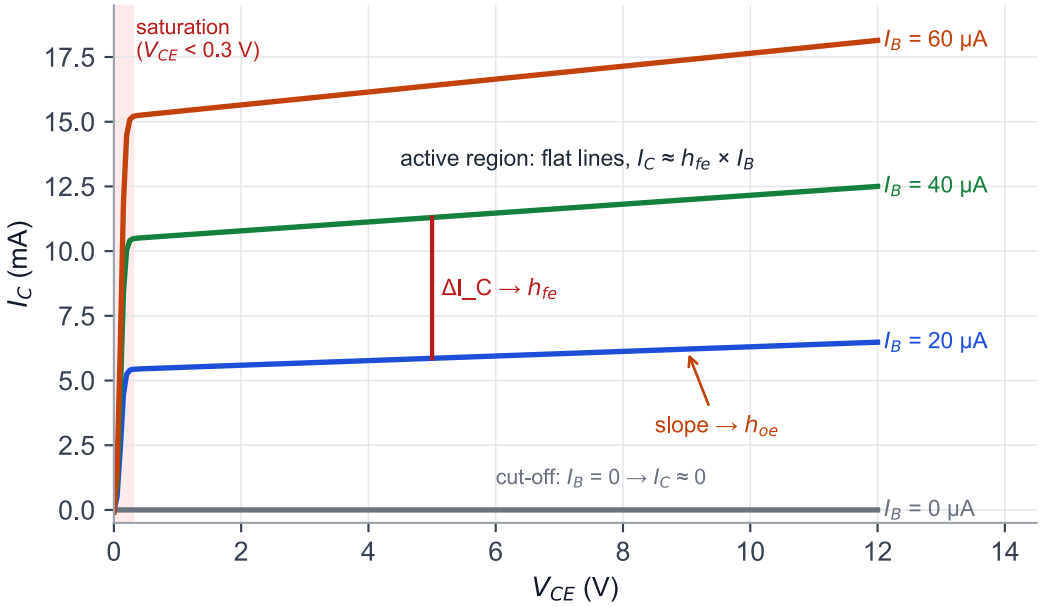
1. Replace V_{be} and R1 with a **current source** `Ib` (arrow pointing **into** the base).
2. Directive: `.dc Vce 0 12 0.05 Ib 0 60u 20u` (I_B = 0, 20, 40, 60 µA).
3. Plot **Ic(Q1)** against V_{ce}.

h_{re}: set Ib = `20u`, run `.op` at V_{ce} = 2 V and at 10 V, and read V(base) both times.

What you should see



Input characteristic: a diode curve. Raising V_{CE} shifts it right (at V_{CE} = 0 the BC junction is also forward and adds base current).



Output characteristics: saturation hugging the left edge, the active region (nearly flat), and cut-off at $I_B = 0$.

V_{CE} (V)	0.5	1	2	4	6	8	10	12
I_C (mA), I_B = 0	0	0	0	0	0	0	0	0
I_C (mA), I_B = 20 µA	5.46	5.50	5.59	**5.77**	5.95	6.13	**6.30**	6.48
I_C (mA), I_B = 40 µA	10.53	10.61	10.78	11.13	11.47	11.81	12.16	12.50
I_C (mA), I_B = 60 µA	15.27	15.40	15.65	16.15	16.65	17.14	17.64	18.14

Input, V_{CE} = 5 V: V_{BE} = 0.60 V → 0.81 µA; 0.62 → 1.73; 0.64 → 3.68; 0.66 → 7.73; 0.68 → 15.9; 0.70 → 31.6 µA. At V_{CE} = 0 V: 0.50 V → 1.07 µA; 0.55 → 6.55; 0.60 → 41.2 µA.

Calculations (from the curves above)

$$h_{ie} = \frac{\Delta V_{BE}}{\Delta I_B} = \frac{0.6984 - 0.6670}{30 - 10\ \mu A} = \frac{31.4\ mV}{20\ \mu A} = 1.57\ k\Omega \qquad (V_{CE} = 5\ V)$$

$$h_{fe} = \frac{\Delta I_C}{\Delta I_B} = \frac{11.30 - 5.86\ mA}{40 - 20\ \mu A} = \frac{5.44\ mA}{20\ \mu A} = 272 \qquad (V_{CE} = 5\ V)$$

$$h_{oe} = \frac{\Delta I_C}{\Delta V_{CE}} = \frac{6.30 - 5.77\ mA}{10 - 4\ V} = \frac{0.533\ mA}{6\ V} = 89\ \mu S \qquad (I_B = 20\ \mu A;\ 1/h_{oe} = 11.2\ k\Omega)$$

$$h_{re} = \frac{\Delta V_{BE}}{\Delta V_{CE}} = \frac{0.68671 - 0.68638}{10 - 2} = \frac{0.33\ mV}{8\ V} = 4.2 \times 10^{-5} \qquad (I_B = 20\ \mu A)$$

Sense check: h_{re} is tiny (the output barely affects the input), and h_{oe} is small (the flat lines tilt only slightly). That is why simple amplifier analysis often drops both.

WRITE IN YOUR RECORD

Result (BC547B, CE): $h_{ie} \approx$ **1.57 kΩ**, $h_{re} \approx$ **4.2×10^{-5}**, $h_{oe} \approx$ **89 µS**, $h_{fe} \approx$ **272**. State the operating point used: V_{CE} = 5 V, $I_B \approx$ 20 µA.

Theory for the record: a BJT has emitter, base and collector regions forming two junctions. For amplification the BE junction is forward-biased and the BC junction reverse-biased (active region): carriers injected from the emitter cross the thin base and reach the collector, so $I_C = \beta I_B$ and $I_E = I_B + I_C$. In CE configuration, the input characteristic (I_B vs V_{BE} at constant V_{CE}) resembles a forward diode curve. The output characteristics (I_C vs V_{CE} at constant I_B) show saturation, active and cut-off regions. The h-parameters h_{ie}, h_{re}, h_{fe}, h_{oe} are the slopes of these curves at the operating point and form the small-signal model.

CHECK 1

(a) Using the table, find h_{fe} at V_{CE} = 10 V between I_B = 40 and 60 µA. (b) Why is the output curve for I_B = 0 flat at zero? (c) Which region is the transistor in at V_{CE} = 0.1 V and I_B = 40 µA, and how can you tell?

Answers

Opening question. h_{fe} = 5.5 mA / 20 µA ≈ **275**. "Flat" means **I_C does not depend on V_{CE}**: the transistor acts as a current source controlled by I_B. That is the active region, and the slight tilt is h_{oe}.

Check 1. (a) (17.64 − 12.16) mA / 20 µA = 5.48 mA / 20 µA = **274**. (b) With no base current, the BE junction isn't injecting carriers, so the transistor is cut off and only a leakage current (nA) flows. (c) **Saturation.** V_{CE} is below about 0.2–0.3 V, the steep left-hand part of the curve, where I_C is less than βI_B and depends strongly on V_{CE}.

Exp 6 · CE amplifier: frequency response and bandwidth

Manual pp. 20–21 · hands-on · CO4, CO5 · the same stage is reused in the extra experiment

5 BJT curves → **6 CE amplifier** → 7 Emitter follower → 8 Feedback

What you already have: from Exp 5, in the active region $I_C = \beta I_B$, and the BE junction needs ≈ 0.7 V. From C&S, a capacitor's reactance $X_C = 1/(2\pi fC)$: huge at low f, tiny at high f.

Three terms first:

- **Biasing:** DC voltages that park the transistor in the active region at a chosen **Q-point** (quiescent point = the DC operating point with no signal).
- **Small-signal gain A_v:** v_o/v_{in} for a small sine riding on the Q-point. In decibels: $20 \log_{10} A_v$.
- **Bandwidth:** the frequency range over which the gain stays within 3 dB (a factor of 0.707) of its maximum.

MANUAL EXP 6, AIM

To plot the frequency response of a CE amplifier and determine its bandwidth. Viva form: "Why does the gain fall at **both** low and high frequencies, and what sets each cut-off?"

GUESS FIRST

Name one part responsible for each end before reading on.

The circuit, part by part

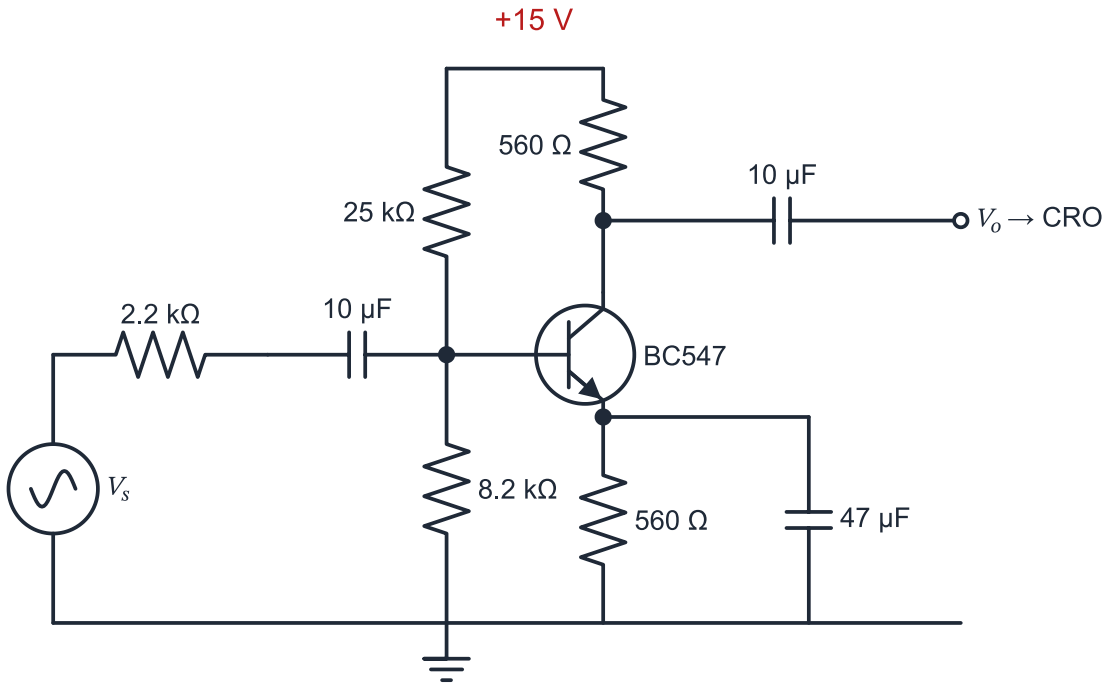


Fig 6.1, redrawn: voltage-divider bias, emitter resistor with bypass capacitor, coupling capacitors in and out.

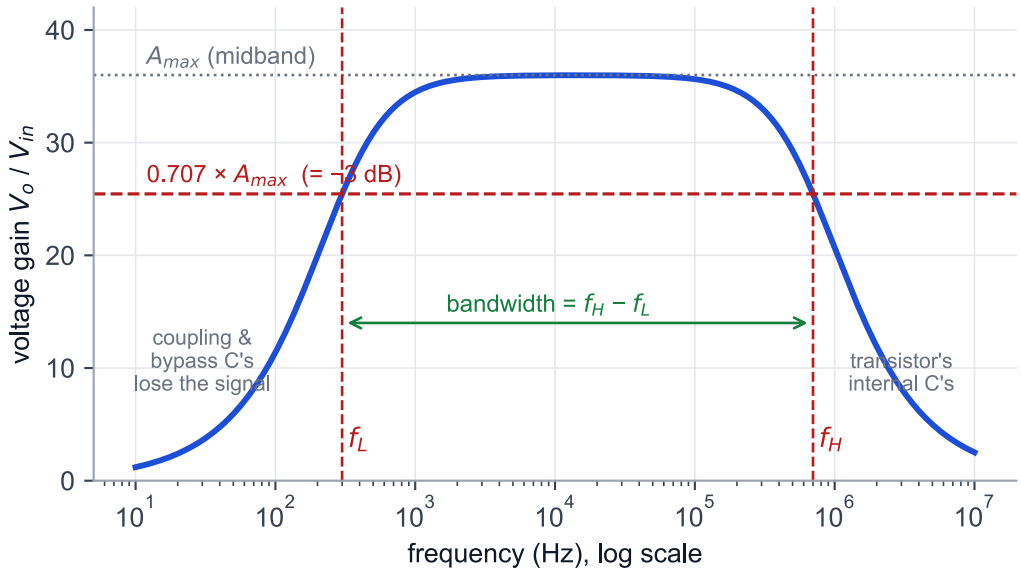
Part	Job
25 kΩ + 8.2 kΩ (R1, R2)	voltage divider: fixes $V_B \approx 15 \times 8.2/33.2 \approx 3.6$–$3.7$ V
560 Ω emitter (R_E)	sets $I_E = V_E/R_E$. It also stabilises the bias: if I_C rises with temperature, V_E rises, which reduces V_{BE}.
47 µF across R_E (C_E, **bypass**)	shorts R_E for AC, so the signal sees the emitter grounded. This gives full gain.
10 µF input (C_{in}, **coupling**)	passes the signal, blocks DC (so the generator doesn't upset the bias)
10 µF output (C_C)	passes the amplified signal, blocks the 12 V collector DC from the CRO
560 Ω collector (R_C)	turns the collector current change into a voltage change

Q-point (LTspice .op): $V_B = 3.60$ V, $V_E = 2.92$ V, $I_C = 5.2$ mA, $V_C = 12.09$ V, so $V_{CE} = 9.2$ V. The transistor is safely active.

Gain by hand:

- r_e = 26 mV / I_E = 26/5.2 = **5 Ω**. This is the BE junction's small-signal resistance.
- Base to collector: $A_v = -R_C/r_e = -560/5 = -112$.
- The 2.2 kΩ source resistor and the amplifier's input resistance (R_{in} = 25k ∥ 8.2k ∥ βr_e ≈ 1.2 kΩ) form a divider. Only 1.2/(1.2 + 2.2) = 0.35 of v_s reaches the base.
- Overall: $v_o/v_s \approx 0.35 \times 112 \approx$ **39**. LTspice gives **36**.
- The minus sign means the output is **inverted** (180° phase shift).

Why the gain falls at both ends



The general shape. Low end: capacitor reactances grow. High end: the transistor's internal junction capacitances short the signal.

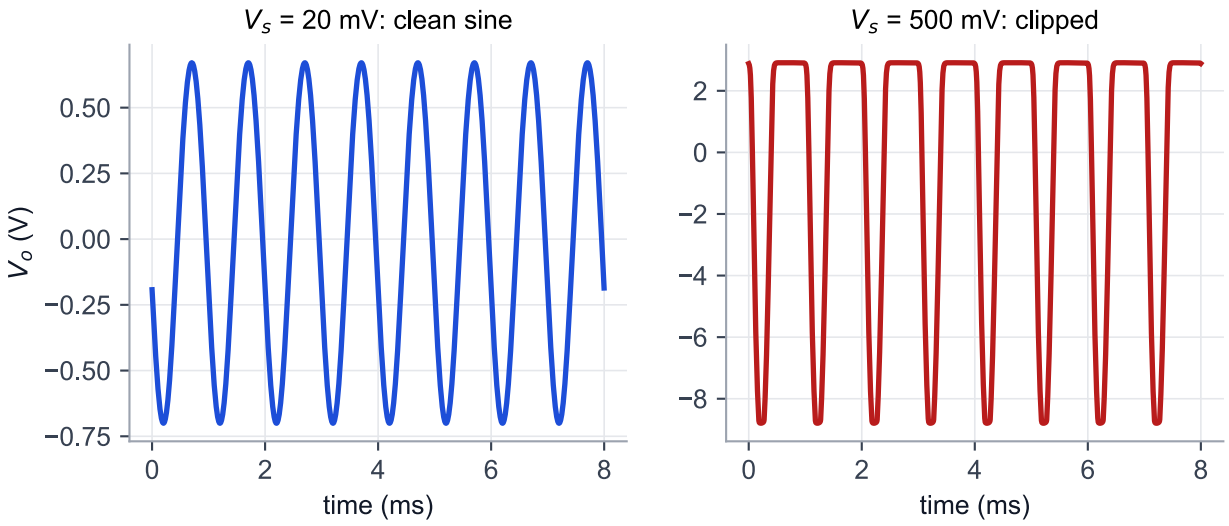
- **Low f:** $X_C = 1/(2\pi fC)$ grows. The coupling capacitors start to block the signal, and above all **C_E stops bypassing R_E**, which brings in negative feedback (see Exp 8). The lower cut-off is set mostly by C_E: $f_L \approx 1/(2\pi\, C_E\, R_e')$, where $R_e' \approx r_e$ + (source ∥ bias)/β ≈ 10.6 Ω. That gives 1/(2π × 47 µF × 10.6 Ω) ≈ **320 Hz**.
- **High f:** tiny capacitances inside the transistor (between its junctions) and in the wiring start to short the signal. The base–collector capacitance is multiplied by the gain (the *Miller effect*), so it dominates.
- **Middle (midband):** all external capacitors act as shorts and the internal ones as opens, so the gain is flat.

DO IT (HANDS-ON)

1. Build Fig 6.1 on the breadboard with supply **+15 V**. 25 kΩ is not a standard value: use 22 kΩ + 3.3 kΩ in series. Check the electrolytic capacitors' polarity: + towards the higher DC voltage (base side for C_{in}, collector side for C_C, emitter side for C_E).
2. **Check the Q-point first** with the DMM, no signal applied: $V_B \approx 3.6$ V, $V_E \approx 2.9$ V, $V_C \approx 12$ V. If $V_C \approx 15$ V the transistor is off; if $V_C \approx V_E$ it is saturated. Fix this before going on.
3. Generator: sine, 1 kHz. Set the amplitude **small: about 20 mV peak** (use the generator's −20 dB/attenuator button, or a 1 kΩ:100 Ω divider). Look at V_o on the DSO. It must be a clean sine, not flattened.
4. Keep V_s constant. Step the frequency through **100 Hz, 500, 800, 1k, 5k, 10k, 50k, 100k, 500k, 800k, 1 MHz** and read $V_{o(pp)}$ at each.
5. Calculate $A_v = V_o/V_s$ and 20 log A_v. Plot dB (y) against frequency on **semi-log** paper (log x-axis).
6. Draw a line 3 dB below the maximum. Where it cuts the curve gives f_L and f_H. BW = $f_H - f_L$.

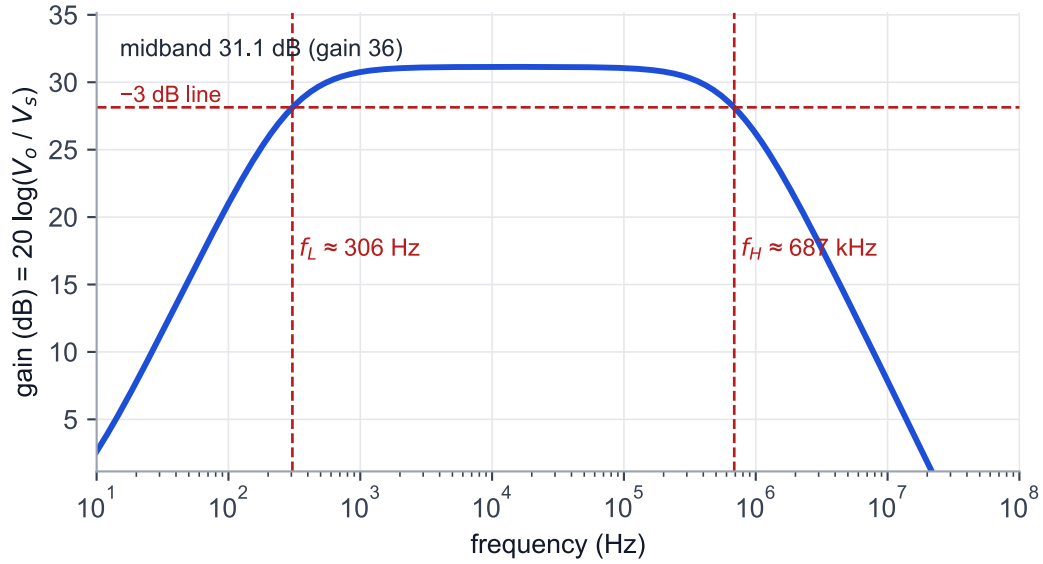
TRAP: THE MANUAL'S 500 MV INPUT

At 500 mV the output would need to swing about ±18 V, but the collector can only move between ≈ 3 V and 15 V. The output is clipped flat (below), so the "gain" you read is meaningless. Even the 100 mV in Fig 6.1 distorts (+2.5 / −3.4 V). Use about 20 mV.



LTspice: the same amplifier with 20 mV in (clean, gain ≈ 34) and 500 mV in (clipped).

What you should see



LTspice, Fig 6.1 with the CRO as a 1 MΩ load. Midband gain 36 (31.1 dB), $f_L \approx 306$ Hz, $f_H \approx 687$ kHz.

f (Hz)	100	500	800	1k	5k	10k	50k	100k	500k	800k	1M
$A_v = V_o/V_s$	11.2	30.8	33.7	34.5	36.0	36.1	36.0	35.7	29.2	23.5	20.4
gain (dB)	21.0	29.8	30.6	30.8	31.1	31.1	31.1	31.1	29.3	27.4	26.2

On the bench, f_L should land close to 300 Hz, because it is set by the capacitors you placed. f_H depends on breadboard wiring, probe and generator capacitance, so expect anything from a few hundred kHz upward. The simulation is the ideal-wiring value.

Calculation

$$BW = f_H - f_L = 687\ \text{kHz} - 0.306\ \text{kHz} \approx 687\ \text{kHz}$$

When $f_H \gg f_L$, the bandwidth is essentially f_H.

WRITE IN YOUR RECORD

Result: maximum gain ≈ **36 (31.1 dB)**; $f_L \approx$ **306 Hz**, $f_H \approx$ **687 kHz**; bandwidth ≈ **687 kHz** (simulation; use your measured values).

Theory for the record: in a CE amplifier the input is applied at the base and the output taken at the collector, with the emitter common (grounded for AC by the bypass capacitor). Voltage-divider bias with an emitter resistor sets a stable Q-point in the active region. A small base signal produces a β-times-larger collector current change, which develops a large, phase-inverted voltage across R_C. Gain ≈ R_C/r_e. The frequency response is flat in midband, falls at low frequencies because of the coupling and bypass capacitors, and falls at high frequencies because of junction and stray capacitances. Bandwidth = $f_H - f_L$, measured at the −3 dB (0.707 A_{max}) points, also called the half-power points.

CHECK 1

(a) Why is the −3 dB point called the "half-power" point? (b) You remove C_E. Does the gain go up or down, and roughly to what? (c) V_C reads 15 V with no signal. What is wrong?

Answers

Opening question. Low end: the **coupling and bypass capacitors**' reactance becomes large; C_E matters most ($f_L \approx 1/(2\pi C_E R_e')$). High end: the **transistor's internal junction capacitances** (Miller-multiplied C_{bc}) plus stray capacitance.

Check 1. (a) Power $\propto V^2$. At 0.707 of the voltage, the power is $0.707^2 = 0.5$ of maximum, and $20 \log 0.707 = -3$ dB. (b) **Down.** R_E returns to the AC path, so $A_v \approx -R_C/(r_e + R_E) = -560/565 \approx -1$. The overall gain is about 1 or less (this is Exp 8's effect). (c) No collector current: the transistor is cut off. Look for an open base connection, a missing emitter return, a wrong pin order (check the BC547's pinout: C-B-E with the flat face towards you) or a dead transistor.

Exp 7 · Emitter follower (common collector): verify its gain

Manual pp. 22–23 · hands-on (with LTspice steps in the manual's procedure) · CO4, CO5

5 BJT curves → 6 CE amplifier → **7 Emitter follower** → 8 Feedback

What you already have: from Exp 6, $r_e = 26\ mV/I_E$, and voltage-divider bias. From Exp 5, $V_E = V_B - 0.7\ V$ in the active region.

Two terms first:

- **Common collector (CC):** the collector sits at the supply, which is AC ground. Input at the base, output at the **emitter**.
- **Buffer:** a stage with gain ≈ 1, high input resistance and low output resistance. It copies a voltage without loading the source.

MANUAL EXP 7, AIM

Verify the gain of an emitter follower circuit. Viva form: "The gain is less than 1. Why would anyone build an amplifier that doesn't amplify voltage?"

GUESS FIRST

Hint: think about **current** and about **resistance seen from each side**.

How it works

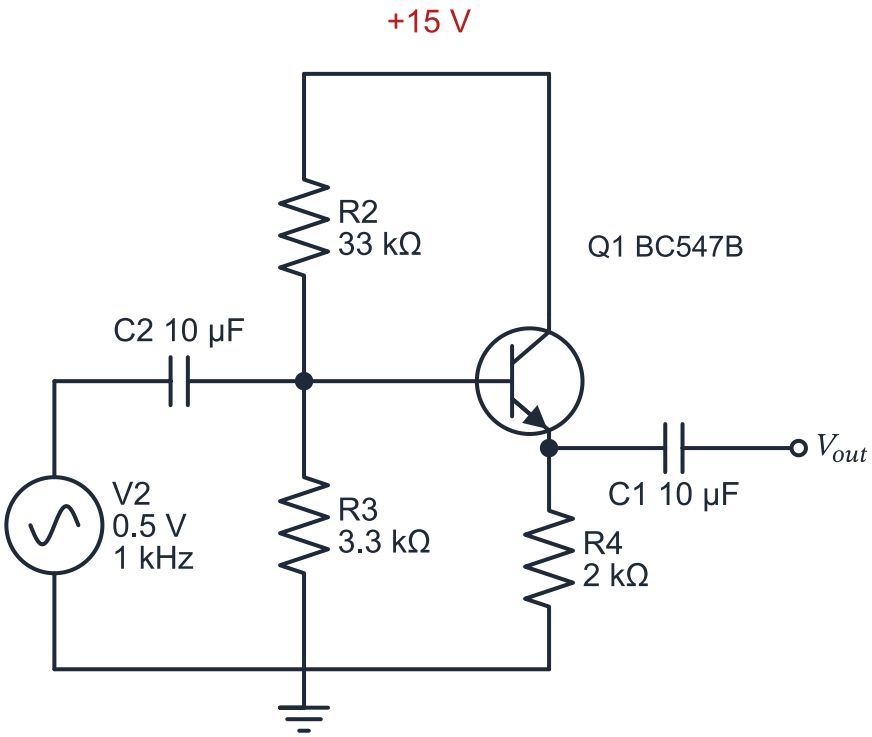


Fig 7.1, redrawn: 33 kΩ / 3.3 kΩ divider, 2 kΩ emitter resistor, output taken from the emitter through 10 µF.

The emitter is always one diode-drop below the base: $V_E = V_B - 0.7$. So when the base moves up by 0.5 V, the emitter moves up by (almost) 0.5 V. The emitter **follows** the base. That is the whole idea.

Q-point (LTspice .op): V_B = 1.36 V, V_E = 0.75 V, I_E = 0.75/2k = **0.376 mA**.

$$r_e = \frac{26\ \text{mV}}{0.376\ \text{mA}} = 69\ \Omega \qquad A_v = \frac{R_E}{R_E + r_e} = \frac{2000}{2069} = 0.967$$

Property	Value here	Why
voltage gain	≈ 0.97, **no inversion**	the emitter follows the base; the output is in phase
input resistance	≈ 3 kΩ (bias resistors), ≈ 600 kΩ into the base alone	the base sees R_E multiplied by β: $\beta(r_e + R_E)$ = 290 × 2.07 kΩ
output resistance	≈ 70 Ω (≈ r_e)	looking back into the emitter
current gain	large (≈ β)	the base current is tiny; the emitter current is β + 1 times bigger

So the answer to the viva question: it is a **current amplifier and impedance converter**. A weak, high-resistance source can drive a heavy, low-resistance load through it without its voltage sagging. That is why the extra design problem uses it as the output stage.

TRAP: THE MANUAL'S THEORY PARAGRAPH

It says the input impedance is "of the order of hundred ohms" and the output "of the order of ten ohms". The output part is right in spirit (tens of ohms). The input impedance is **high**: kilohms to hundreds of kilohms. High input, low output is the whole point.

DO IT

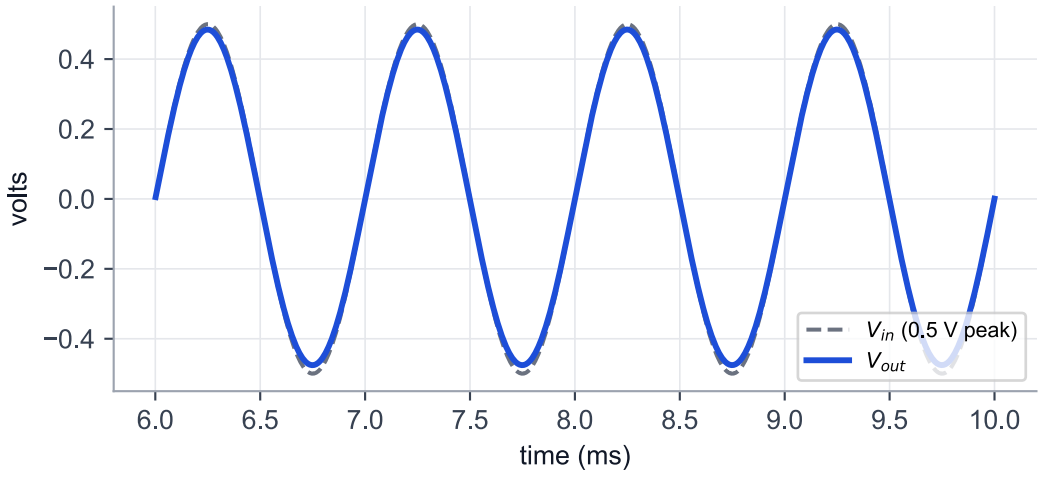
Hands-on:

1. Build Fig 7.1. Use the **15 V** shown in the figure. The parts list says 10 V; see the trap below.
2. Check the Q-point with no signal: $V_B \approx 1.36$ V, $V_E \approx 0.75$ V.
3. Generator: sine, 0.5 V peak, 1 kHz, into C2. DSO CH1 on the input, CH2 on V_{out}. The two traces should overlap, in phase.
4. Step the frequency through 200 Hz, 500, 1k, 10k, 100k, 1 MHz and record V_o. Gain = V_o/V_i.

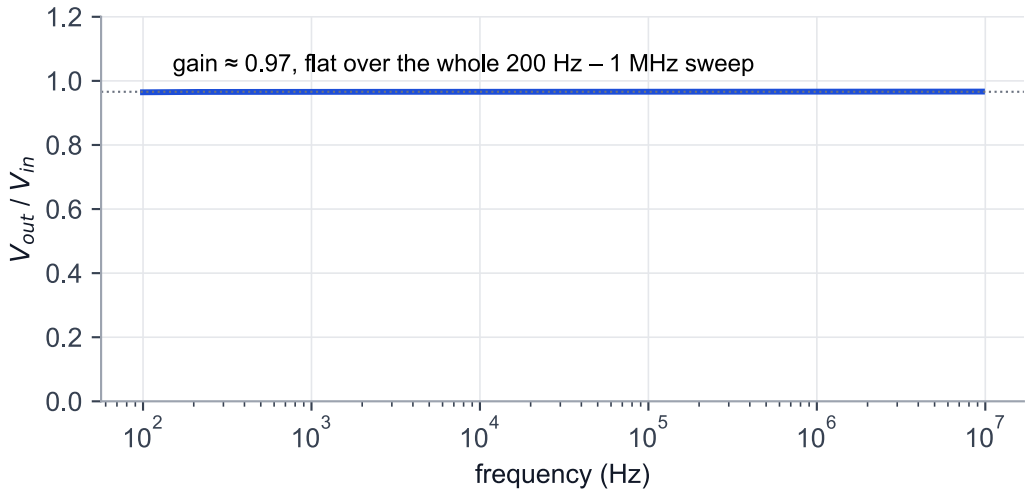
LTspice (manual's procedure):

1. V2 with `SINE(0 0.5 1k)` and AC amplitude `0.5`. Q1 = BC547B.
2. Run `.tran 10m` and compare V(out) with the input.
3. Then run `.ac dec 10 200 1Meg`. Comment out the line you are not using (right-click the directive and choose **Comment**), or the `;` prefix in the manual's figure does the same thing.

What you should see



LTspice: output (solid) sits on the input (dashed), in phase, 0.96 V p-p for 1.0 V p-p in.



Gain ≈ 0.97 across the whole 200 Hz – 1 MHz sweep. Neither −3 dB point is inside the sweep: f_L ≈ 5 Hz, f_H far above 1 MHz.

f (Hz)	200	500	1k	10k	100k	1M
A_v	0.965	0.966	0.966	0.966	0.966	0.966
dB	−0.31	−0.30	−0.30	−0.30	−0.30	−0.30

TRAP: A 10 V SUPPLY CLIPS THE OUTPUT

With 10 V, V_E drops to about 0.35 V. A 0.5 V-peak input then pulls the emitter below 0 V and the transistor cuts off: LTspice shows the negative half flattened at −0.32 V. With 10 V, either use 0.2 V peak (clean) or the figure's 15 V.

WRITE IN YOUR RECORD

Result: maximum gain of the emitter follower ≈ **0.97** (−0.3 dB), output in phase with input; flat from 200 Hz to 1 MHz, so the bandwidth exceeds the measured range (simulation: f_L ≈ 5 Hz).

Theory for the record: in the common-collector (emitter follower) circuit, the input is applied to the base and the output taken from the emitter, while the collector is at AC ground. Because $V_E = V_B - V_{BE}$, the emitter follows the base: voltage gain is slightly below 1 ($R_E/(R_E + r_e)$) with no phase inversion. The input resistance is high (≈ βR_E) and the output resistance low (≈ r_e), so the stage is used as a buffer for impedance matching from a high-impedance source to a low-impedance load. It has a high current gain and power gain, and a wide bandwidth.

CHECK 1

(a) Why is A_v always a little less than 1, never exactly 1? (b) Change R_E to 1 kΩ (same bias). Estimate the new I_E, r_e and A_v. (c) Which stage, CE or CC, would you put directly in front of an 8 Ω speaker, and why?

Answers

Opening question. It amplifies **current** (≈ β) and therefore power. It also converts a high source resistance into a low output resistance (a buffer), so a weak source can drive a heavy load.

Check 1. (a) The output is taken across R_E, which is in series with r_e: $A_v = R_E/(R_E + r_e)$. r_e is never zero. (b) I_E = 0.75/1k = 0.75 mA, r_e = 26/0.75 = 35 Ω, A_v = 1000/1035 = **0.966**: still about the same. (c) **CC (emitter follower).** Its output resistance is tiny, so the 8 Ω load does not collapse the voltage. A CE stage's output resistance (≈ R_C) would drop most of the signal internally.

Exp 8 · Effect of negative feedback (removing the emitter bypass capacitor)

Manual pp. 24–25 · LTspice · CO4, CO5 · the manual's two plots (≈ 42 dB and ≈ 4 dB) match the simulation below

5 BJT curves → 6 CE amplifier → 7 Emitter follower → **8 Feedback**

What you already have: from Exp 6, C_E bypasses R_E, and gain ≈ R_C/r_e. From Exp 7, a resistor in the emitter makes the output "follow" and pulls the gain down.

Two terms first:

- **Feedback:** feeding part of the output back to the input.
- **Negative** feedback opposes the input. It lowers the gain but steadies everything else.
- **Positive** feedback adds to the input. It leads to oscillation (Exp 11).
- **Current-series feedback:** the feedback voltage is produced by the output **current** ($I_E \approx I_C$ through R_E) and subtracted in **series** with the input ($V_{BE} = v_{in} - I_E R_E$).

MANUAL EXP 8, AIM

To observe the effect of negative feedback (emitter bypass capacitor) on the frequency response of an amplifier. Viva form: "Removing one capacitor cuts the gain from 122 to 1.7. Why would anyone want that?"

GUESS FIRST

Name one thing that gets **better** when the gain drops.

The idea

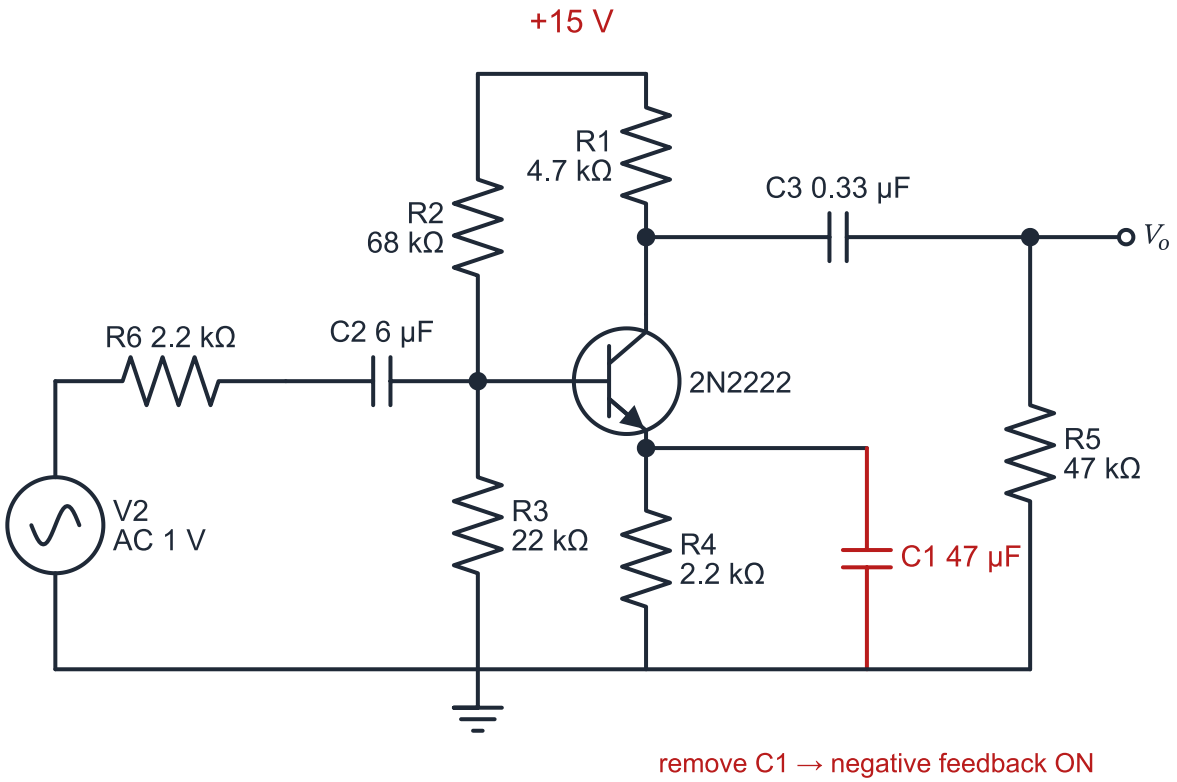


Fig 8.1, redrawn: a CE stage (2N2222) with load. C1 (red) is the bypass capacitor. Removing it is the whole experiment.

- **C1 in:** R4 is shorted for AC, so there is no feedback. $A_v \approx (R_C \parallel R_L)/r_e$, which is large.
- **C1 out:** the signal current through R4 (2.2 kΩ) makes a voltage that rises with the input, so the BE voltage the transistor actually sees shrinks. That is negative feedback. $A_v \approx (R_C \parallel R_L)/(r_e + R_E)$ = 4.27k/2.2k ≈ 1.9 at the base.

What negative feedback buys, at the cost of gain:

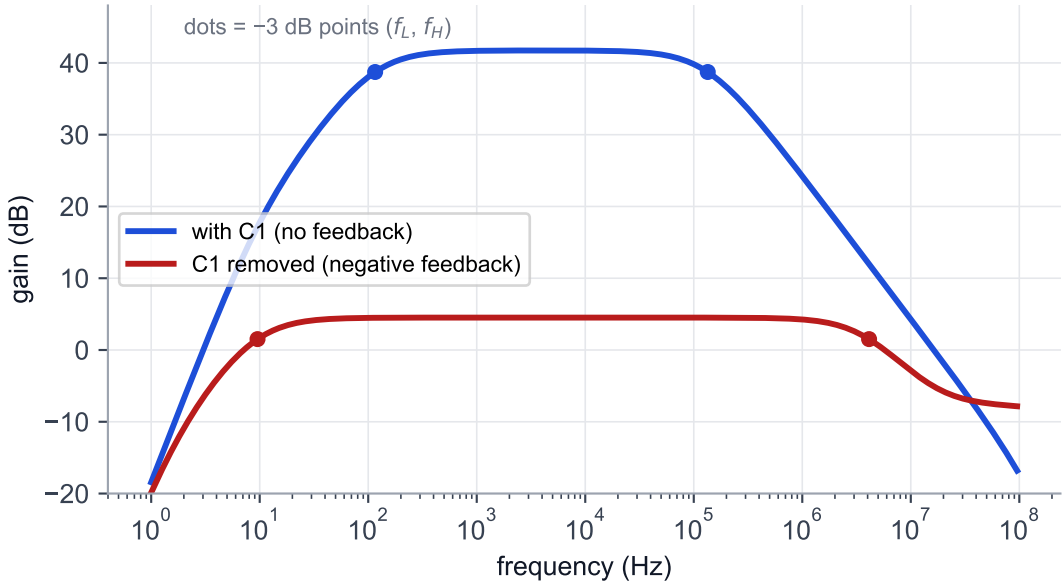
Property	Effect
gain	falls, but becomes **set by resistors** (R_C/R_E), not by β or temperature, so it is **stable**
bandwidth	**increases** (gain is traded for bandwidth)
distortion and noise	decrease
input resistance	increases (series feedback)

Do it

DO IT (LTSPICE)

1. Draw Fig 8.1. Q1 = **2N2222**. V2: AC amplitude `1`. Supply V1 = 15 V. Label the output `vo` (across R5, 47 kΩ).
2. Directive: `.ac dec 10 1 1Meg` (manual: 1 Hz – 1 MHz). Run. Plot V(vo): the solid line is the dB gain.
3. Read the peak dB with a cursor. Move the cursor to the frequencies where the gain is 3 dB lower: those are f_L and f_H.
4. **Delete C1** (or set it to `1p`). Run again and repeat.
5. Fill the table at 100 Hz, 500, 1k, 5k, 10k, 50k, 100k, 500k, 1M for both cases.

What you should see



LTspice. Blue: with C1, 41.7 dB peak, f_L 116 Hz, f_H 135 kHz. Red: without C1, 4.5 dB, f_L 9.5 Hz, f_H 4.1 MHz.

f (Hz)	100	500	1k	5k	10k	50k	100k	500k	1M
A_v with C1	79.8	119.1	121.4	122.1	121.9	114.5	98.0	31.7	16.3
A_{vf} without C1	1.67	1.68	1.68	1.68	1.68	1.68	1.68	1.67	1.63

	Without feedback (C1 in)	With feedback (C1 out)
maximum gain	122 (41.7 dB)	1.68 (4.5 dB)
f_L	116 Hz	9.5 Hz
f_H	135 kHz	4.1 MHz
bandwidth	≈ **135 kHz**	≈ **4.1 MHz**

Hand check (Q-point: V_B = 3.56 V, V_E = 2.90 V, so I_E = 1.32 mA and r_e = 19.7 Ω):

- R_{in} with feedback = 68k ∥ 22k ∥ β × 2.2k ≈ 16 kΩ. The source divider is 16/(16 + 2.2) = 0.88.
- A_{vf} ≈ 0.88 × 4.27k/2.22k = **1.70** (LTspice: 1.68) ✓.

WRITE IN YOUR RECORD

Result: bandwidth ≈ **135 kHz** without feedback (gain 41.7 dB). With negative feedback (C_E removed), the gain falls to 4.5 dB and the bandwidth rises to ≈ **4.1 MHz**: the higher cut-off is far above 1 MHz, as the manual expects.

Theory for the record: removing the emitter bypass capacitor leaves R_E unbypassed. The AC emitter current develops a voltage across R_E that subtracts from the input in series: current-series negative feedback. The voltage gain falls from ≈ R_C/r_e to ≈ $R_C/(r_e + R_E)$, but becomes stable (independent of β and temperature). The bandwidth increases, distortion and noise fall, and the input resistance rises. Negative feedback trades gain for bandwidth and stability.

CHECK 1

(a) Gain × bandwidth without feedback, and with feedback. Is the product the same? (b) Why is the 2N2222's own β almost irrelevant to the gain once C1 is out? (c) Which part of the response barely changed between the two cases at 100 kHz–1 MHz, and why?

Answers

Opening question. Negative feedback makes the gain **stable and predictable** (set by resistors, not β), **widens the bandwidth** (135 kHz → 4.1 MHz), and reduces distortion and noise. You add gain back with more stages; stability is hard to add later.

Check 1. (a) 122 × 135 kHz ≈ 16.5 MHz; 1.68 × 4.1 MHz ≈ 6.9 MHz. **Not the same**: "gain × bandwidth = constant" holds for a single dominant pole, not exactly for this circuit, but the direction (gain down, bandwidth up) holds. (b) $A_{vf} \approx (R_C \parallel R_L)/(r_e + R_E)$, and $R_E \gg r_e$, so β drops out. (c) The without-C1 curve is flat across it, while the with-C1 curve is already falling. With feedback, f_H moved out to MHz.

Exp 9 · JFET drain and transfer characteristics, and device parameters

Manual pp. 26–28 · LTspice · CO1, CO2 · theory: ECE2101 unit 3 · the simulation reproduces the manual's printed table exactly · the professor flagged these four parameters as an 8-mark question (classmate's report, 2026-10-05)

9 JFET curves → 10 CS amplifier → 11 Oscillator → 12 MOSFET inverter

What you already have: from Exp 1, a reverse-biased pn junction carries no current and has a depletion region with no free carriers. From Exp 5, families of output curves and slopes taken as parameters.

Three terms first:

- **FET (field-effect transistor):** current flows through a **channel**, and a **voltage** on the gate controls the channel's width. It is *voltage-controlled*. A BJT, by contrast, is controlled by base current.
- **JFET terminals: drain** (D, current in), **source** (S, current out), **gate** (G, controls).
- **Unipolar:** only one carrier type carries the current (electrons, in an n-channel JFET). A BJT uses both electrons and holes, which is why it is called *bipolar*.

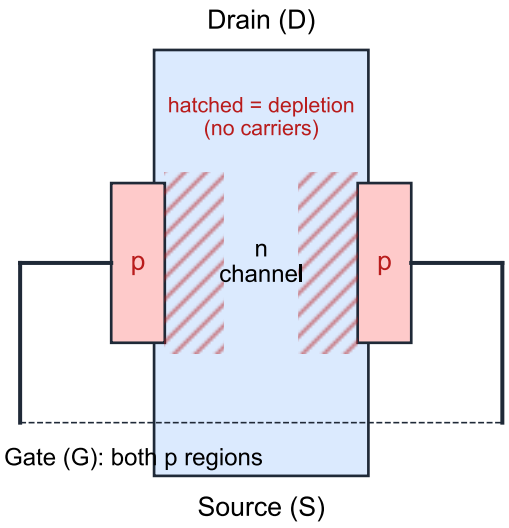
MANUAL EXP 9, AIM · PROFESSOR'S SLIDE

To plot the drain and transfer characteristics of a JFET in CS configuration and determine the device parameters. Viva / 8-mark form: "Define g_m, r_d, g_d and μ, find each from the curves, and show $\mu = g_m r_d$."

GUESS FIRST

Which of the four do you think is read from the *transfer* curve, and which from the *drain* curves?

How a JFET works



n-channel JFET: an n bar between two p gate regions. The gate–channel junctions are kept reverse-biased ($V_{GS} \le 0$).

1. **V_{GS} = 0:** the channel is fully open, and current flows from D to S.
2. **V_{GS} more negative:** the reverse-biased gate junctions' depletion regions widen into the channel, so the channel narrows and I_D falls.
3. **$V_{GS} = V_P$** (pinch-off voltage, ≈ −3 V here): the depletion regions meet and $I_D \approx 0$. The JFET is off.
4. **Raising V_{DS}** at fixed V_{GS}: at first I_D rises like a resistor (**ohmic region**). Then the channel pinches off near the drain and I_D levels out (**saturation**, or pinch-off region). The amplifier operates here.
5. The gate junction is reverse-biased, so **gate current ≈ 0**: the input resistance is enormous (≫ MΩ). This is the JFET's big advantage.

Shockley's equation (the transfer curve, in saturation):

$$I_D = I_{DSS}\left(1 - \frac{V_{GS}}{V_P}\right)^2$$

- I_{DSS} = drain current with the gate **S**horted to the **S**ource (V_{GS} = 0), in saturation.

The four parameters (the professor's slide)

Parameter	Definition	Held constant	Read from
transconductance g_m	$\Delta I_D/\Delta V_{GS}$	V_{DS}	two drain curves at one V_{DS} (or the transfer curve's slope)
drain resistance r_d	$\Delta V_{DS}/\Delta I_D$	V_{GS}	one drain curve, in saturation
drain conductance g_d	$\Delta I_D/\Delta V_{DS} = 1/r_d$	V_{GS}	same
amplification factor μ	$-\Delta V_{DS}/\Delta V_{GS}$	I_D	**$\mu = g_m \times r_d$**

Why $\mu = g_m r_d$: μ asks how much V_{DS} change balances a V_{GS} change at constant I_D. Write it as a chain of ratios: $(\Delta V_{DS}/\Delta I_D) \times (\Delta I_D/\Delta V_{GS}) = r_d \times g_m$. The minus sign records that a more negative V_{GS} must be balanced by a larger V_{DS}.

Do it

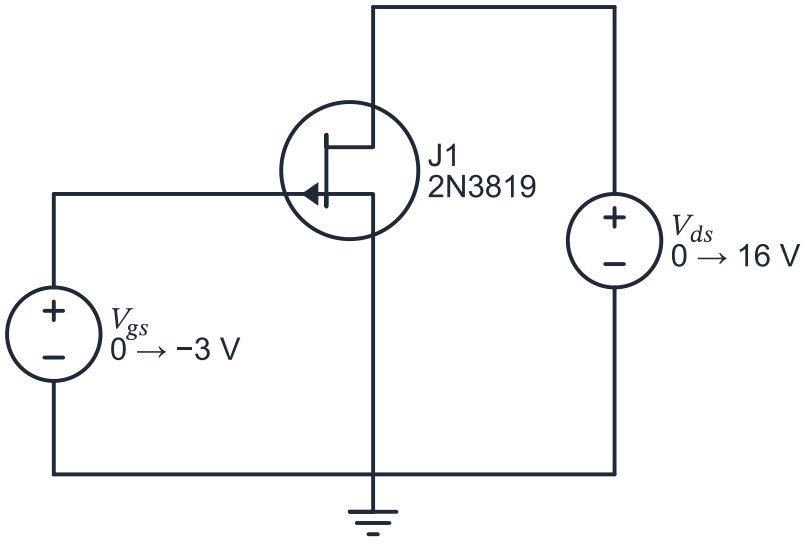
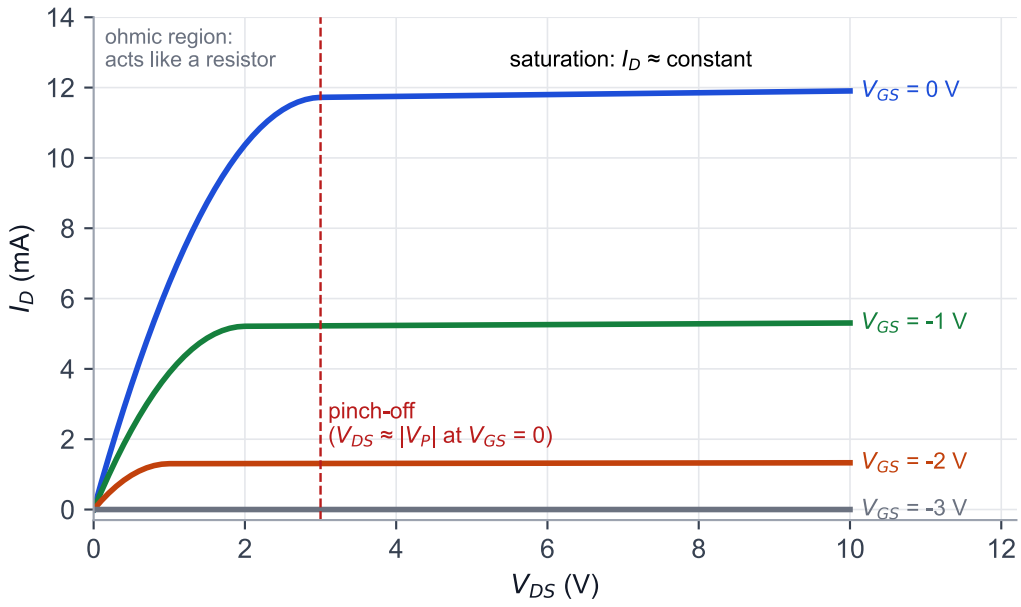


Fig 9.2, redrawn: V_{gs} on the gate, V_{ds} on the drain, source grounded.

DO IT (LTSPICE)

1. Draw the circuit with J1: Pick New JFET → **2N3819** (an n-channel JFET, `njf`). Name the sources `Vgs` and `Vds`.
2. **Drain characteristics:** `.dc Vds 0 16 0.1 Vgs 0 -3 -1`. This gives four curves, for V_{GS} = 0, −1, −2, −3 V. Plot **Id(J1)**.
3. **Transfer characteristic:** set Vds = 8 V, then `.dc Vgs -5 0 0.01`. Plot Id(J1) against V_{gs}.
4. Read I_{DSS} (cursor at V_{GS} = 0) and V_P (where I_D reaches ≈ 0).

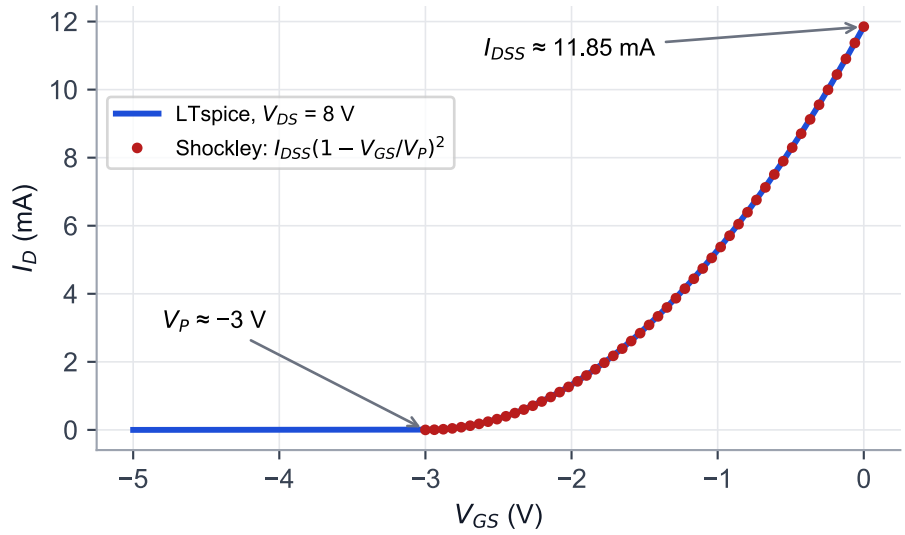
What you should see



Drain characteristics: ohmic rise, pinch-off, then flat saturation. Each curve flattens at $V_{DS} \approx V_{GS} - V_P$.

V_{DS} (V)	0	1	2	3	4	5	6	7	8
I_D (mA), $V_{GS} = 0$	0	6.45	10.37	11.72	**11.75**	11.77	**11.80**	**11.83**	11.85
I_D (mA), $V_{GS} = -1$	0	3.89	5.21	5.22	**5.24**	5.25	5.26	5.27	5.28
I_D (mA), $V_{GS} = -2$	0	1.30	1.31	1.31	1.31	1.32	1.32	1.32	1.32
I_D (mA), $V_{GS} = -3$	0	0	0	0	0	0	0	0	0

These are the manual's printed numbers (it lists them in amps: 0.01185 A = 11.85 mA).



Transfer characteristic at $V_{DS} = 8$ V, with Shockley's equation (dots) laid on top: a near-perfect match.

Calculations

- **I_{DSS} = 11.85 mA** ($V_{GS} = 0$, $V_{DS} = 8$ V).
- **$V_P \approx -3$ V** (I_D reaches 0 at $V_{GS} = -3$ V).

$$g_m = \frac{\Delta I_D}{\Delta V_{GS}} = \frac{11.75 - 5.24 \text{ mA}}{0 - (-1) \text{ V}} = 6.51 \text{ mA/V} \; (= 6.51 \text{ mS}) \qquad \text{at } V_{DS} = 4 \text{ V}$$

$$r_d = \frac{\Delta V_{DS}}{\Delta I_D} = \frac{8 - 3 \text{ V}}{11.852 - 11.722 \text{ mA}} = \frac{5 \text{ V}}{0.130 \text{ mA}} = 38.5 \text{ k}\Omega \qquad \text{at } V_{GS} = 0$$

$$g_d = \frac{1}{r_d} = 26 \; \mu\text{S} \qquad \mu = g_m r_d = 6.51 \text{ mS} \times 38.5 \text{ k}\Omega \approx 250$$

Cross-check with Shockley:

- The slope at $V_{GS} = 0$ is $g_{m0} = 2I_{DSS}/|V_P| = 2 \times 11.85/3 = 7.9$ mS.
- The 0 → −1 V chord equals the slope at the midpoint, $V_{GS} = -0.5$ V: $g_m = g_{m0}(1 - 0.5/3) = 6.6$ mS ✓ (6.51 measured).

TRAP: THE MANUAL'S R_D LINE

It writes (7 − 6)/(0.1183 − 0.1180) = 3.3 kΩ. The currents are 0.01183 A and 0.01180 A, so the difference is 0.03 mA, and r_d = 1 V / 0.03 mA = **33 kΩ**. Also, a 1 V span with 4-digit data is coarse: use a wide V_{DS} span (3 → 8 V) inside saturation, which gives 38.5 kΩ.

WRITE IN YOUR RECORD

Result: I_{DSS} = **11.85 mA**, V_P = **−3 V**, g_m = **6.51 mS** (V_{DS} = 4 V), r_d = **38.5 kΩ** (V_{GS} = 0); also g_d = 26 µS and $\mu = g_m r_d \approx 250$.

Theory for the record: an n-channel JFET is an n-type channel between two p-type gate regions. With the gate–source junction reverse-biased, the depletion regions narrow the channel as V_{GS} becomes more negative, so the gate voltage controls the drain current. Gate current is negligible, giving very high input resistance. As V_{DS} increases, I_D first rises linearly (ohmic region), then the channel pinches off and I_D saturates. At $V_{GS} = V_P$ the channel is cut off. In saturation, $I_D = I_{DSS}(1 - V_{GS}/V_P)^2$ (Shockley). Parameters: $g_m = \Delta I_D/\Delta V_{GS}$ at constant V_{DS}; $r_d = \Delta V_{DS}/\Delta I_D$ at constant V_{GS}; $\mu = g_m r_d$.

CHECK 1

(a) Use Shockley to predict I_D at $V_{GS} = -2$ V, and compare with the table. (b) Why is a JFET's gate never forward-biased in normal use? (c) BJT vs JFET: give two differences an examiner would accept.

Answers

Opening question. g_m from the **transfer** curve (or two drain curves at one V_{DS}); r_d and g_d from **one drain curve** in saturation; μ from $g_m \times r_d$.

Check 1. (a) $11.85 \times (1 - 2/3)^2 = 11.85/9 =$ **1.32 mA**; the table gives 1.31–1.32 ✓. (b) Forward bias would make the gate junction conduct. The gate would draw current, the high input resistance would be lost, and the device could be damaged. (c) Any two: a JFET is voltage-controlled, a BJT current-controlled; a JFET is unipolar, a BJT bipolar; a JFET has very high input resistance, a BJT moderate ($h_{ie} \approx$ kΩ); a JFET is less noisy and more thermally stable; a BJT has higher gain (g_m).

Exp 10 · JFET common-source amplifier: frequency response

Manual pp. 29–30 · LTspice · CO4, CO5 · the JFET is the 2N5434 (the manual's "2N54349" is a typo)

9 JFET curves → **10 CS amplifier** → 11 Oscillator → 12 MOSFET inverter

What you already have: the BJT's CE amplifier (Exp 6), meaning bias, coupling and bypass capacitors, gain, and −3 dB points. The JFET's g_m (Exp 9).

Two terms first:

- **Common source (CS):** the FET's version of CE. Input at the gate, output at the drain, source common.
- **Self-bias:** the source resistor R_s lifts the source above the gate, which makes V_{GS} negative without a negative supply.

MANUAL EXP 10, AIM

To plot the frequency response of a common-source amplifier. Viva form: "Estimate the gain from g_m and the resistors, and say why the output is 180° out of phase."

GUESS FIRST

CE gain was R_C/r_e. What do you think replaces $1/r_e$ for a FET?

The circuit and its bias

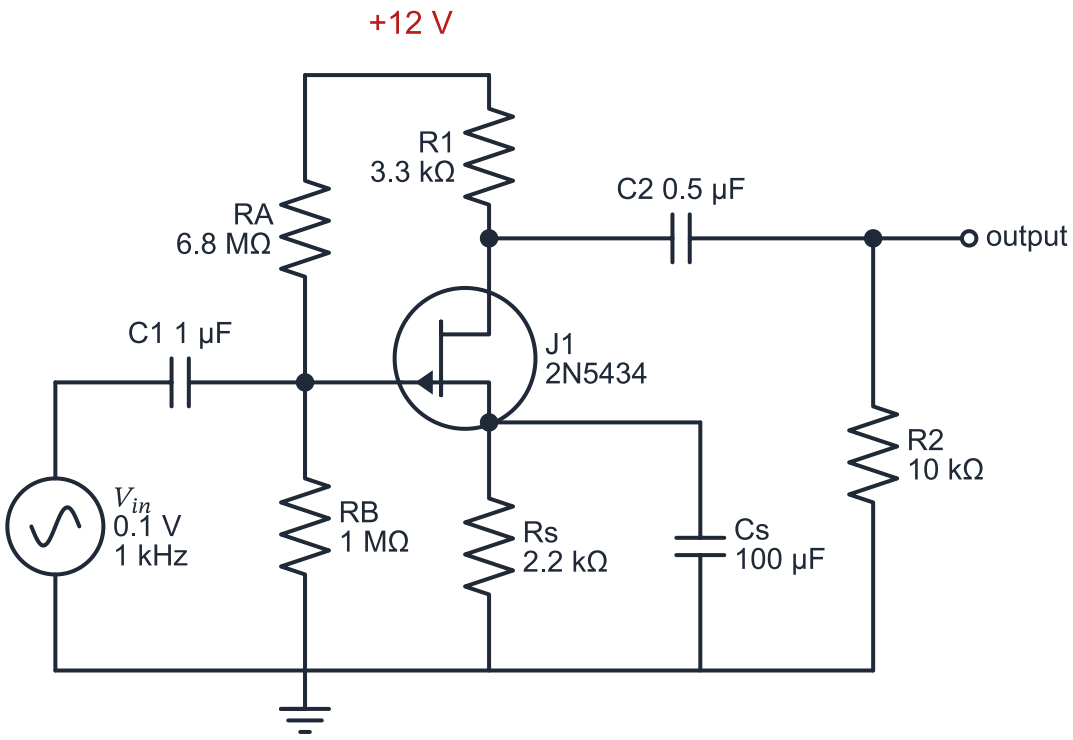


Fig 10.1, redrawn: gate divider RA/RB, drain resistor R1, source resistor Rs bypassed by Cs, 10 kΩ load through C2.

Part	Job
RA 6.8 MΩ, RB 1 MΩ	gate divider: $V_G = 12 \times 1/7.8 \approx 1.54$ V. Megohms are fine because the gate draws no current.
Rs 2.2 kΩ	self-bias: $V_{GS} = V_G - I_D R_s$ is negative
Cs 100 µF	bypasses Rs for AC, giving full gain (as C_E did in Exp 6)
R1 3.3 kΩ	drain load: converts the I_D change into a voltage change
C1, C2	coupling: block DC, pass the signal

Q-point (LTspice .op): $V_G = 1.54$ V, $V_S = 3.17$ V, so **$V_{GS} = -1.63$ V**. $I_D = 3.17/2.2\text{k} =$ **1.44 mA**, and $V_D = 7.25$ V. The drain sits well above the source, so the JFET is in saturation ✓.

Gain by hand: the small-signal model is a current source $g_m v_{gs}$ into the drain.

$$A_v = -g_m (R_1 \parallel R_2) = -g_m \times (3.3\ \text{k}\Omega \parallel 10\ \text{k}\Omega) = -g_m \times 2.48\ \text{k}\Omega$$

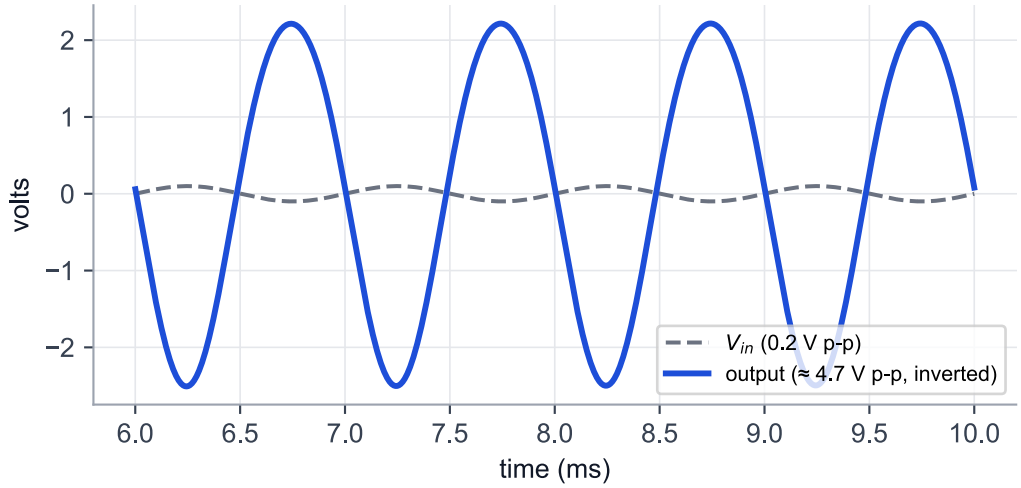
For the 2N5434 at this Q-point, $g_m \approx 9.7$ mS (from its model: $I_{DSS} \approx 65$ mA, $V_P \approx -1.9$ V, using $g_m = (2I_{DSS}/|V_P|)(1 - V_{GS}/V_P)$). So $A_v \approx -24$. The **minus sign** is the 180° phase inversion: as v_{gs} goes up, I_D goes up, and the drain voltage $V_{DD} - I_D R_1$ goes **down**.

Do it

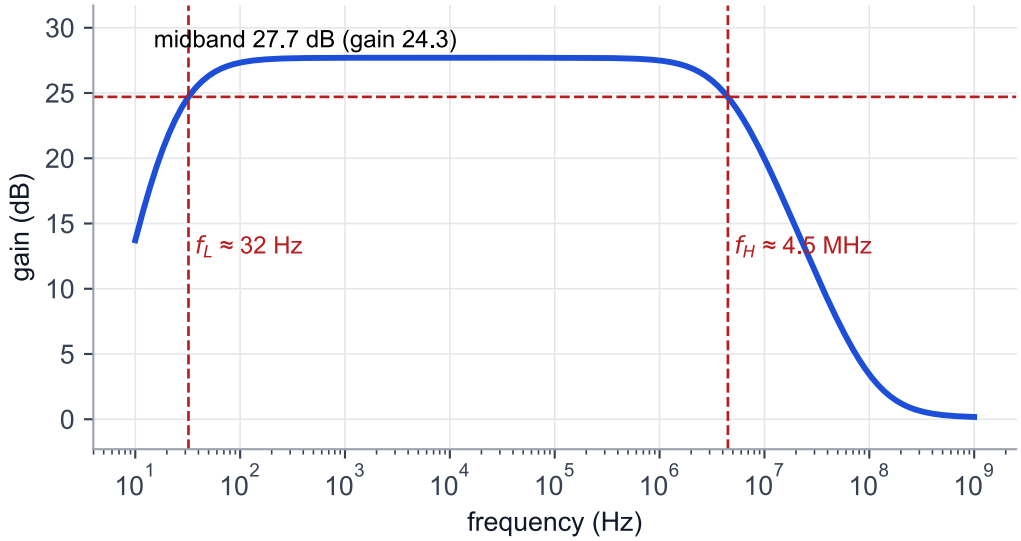
DO IT (LTSPICE)

1. Draw Fig 10.1 with J1 = **2N5434**, VDD = 12 V. Name the output node `out` (across R2).
2. Vin: `SINE(0 0.1 1k)` with AC amplitude `1`.
3. **Transient:** `.tran 10m`. Plot the input and V(out), and read both peak-to-peak values with cursors.
4. **Frequency response:** `.ac dec 10 100 100Meg`. Read the midband dB and find the two −3 dB frequencies.
5. Fill the table at 100 Hz, 1k, 5k, 10k, 50k, 100k, 500k, 1M, 5M, 10M, 100M.

What you should see



Transient: 0.2 V p-p in, ≈ 4.7 V p-p out, inverted.



AC: midband gain 24.3 (27.7 dB), f_L ≈ 32 Hz, f_H ≈ 4.5 MHz.

f (Hz)	100	1k	5k	10k	50k	100k	500k	1M	5M	10M	100M
gain	23.3	24.3	24.3	24.3	24.3	24.3	24.1	23.7	16.2	10.0	1.5
dB	27.3	27.7	27.7	27.7	27.7	27.7	27.6	27.5	24.2	20.0	3.4

Calculation

- Transient gain = $V_{O(pp)}/V_{in(pp)}$ = 4.73/0.2 = **23.6**.
- AC midband gain = **24.3**.
- BW = $f_H - f_L$ = 4.48 MHz − 32 Hz ≈ **4.5 MHz**.

The low cut-off comes from Cs (with $R_s \parallel 1/g_m \approx 100\ \Omega$) and from C2 (with R1 + R2). The high cut-off comes from the JFET's gate–drain and gate–source capacitances (35 pF each in its model), Miller-multiplied.

WRITE IN YOUR RECORD

Result: maximum gain ≈ **24.3 (27.7 dB)**; transient gain 4.73 V/0.2 V = 23.6; output 180° out of phase; f_L ≈ 32 Hz, f_H ≈ 4.5 MHz, bandwidth ≈ **4.5 MHz**.

Theory for the record: in a common-source amplifier the signal is applied to the gate and taken from the drain, with the source common (bypassed to AC ground). The gate divider and source resistor set a negative V_{GS} that biases the JFET in saturation. A gate-voltage change produces a drain-current change $g_m v_{gs}$, which develops an output voltage across the drain load: $A_v = -g_m(R_D \parallel R_L)$. The output is inverted (180°). The input resistance is very high (set by the gate divider). The gain is lower than a BJT CE stage, but the input resistance and noise performance are better. Coupling/bypass capacitors set f_L; device capacitances set f_H.

CHECK 1

(a) Remove Cs. What happens to the gain, and why? (b) Why can the gate resistors be megohms here, when the BJT's bias resistors were kilohms? (c) Estimate the input resistance seen by the source.

Answers

Opening question. g_m replaces $1/r_e$: $A_v = -g_m(R_D \parallel R_L) \approx -24$. The output is inverted because a rising gate voltage raises I_D, which increases the drop across R1 and lowers the drain voltage.

Check 1. (a) The gain drops: R_s now gives negative feedback (as in Exp 8). $A_v \approx -g_m R_D'/(1 + g_m R_s) = -24/(1 + 9.7\text{m} \times 2.2\text{k}) \approx$ **−1.1**. (b) The JFET gate takes no current, so the divider is not loaded. A BJT's base current would make megohm resistors drop volts and wreck the bias. (c) $R_{in} \approx$ RA ∥ RB = 6.8 MΩ ∥ 1 MΩ ≈ **0.87 MΩ**.

Exp 11 · RC phase-shift oscillator

Manual pp. 31–33 · LTspice · CO4 · the manual's own observed frequency (13.76 kHz) and the simulation below (13.96 kHz) agree within 1.5 %

9 JFET curves → 10 CS amplifier → 11 Oscillator → 12 MOSFET inverter

What you already have:

- From Exp 6, a CE stage inverts its signal (180°) and has gain.
- From Exp 8, negative feedback opposes the input.
- From C&S, in an RC circuit the voltage across R leads the input by an angle that depends on frequency.

Two terms first:

- **Oscillator:** a circuit that produces a steady AC output with **no** AC input. It is an amplifier whose output is fed back to its own input **in phase** (positive feedback).
- **Barkhausen criterion:** for steady oscillation, the loop gain must be $|A\beta| = 1$ (slightly more than 1 to start up), and the total phase shift around the loop must be **360°** (or 0°).

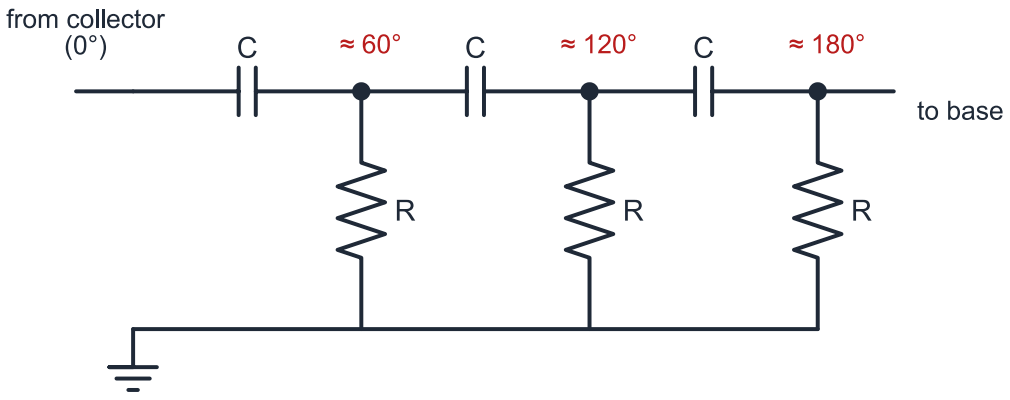
MANUAL EXP 11, AIM

To observe the frequency of oscillation of a phase-shift oscillator. Viva form: "Why three RC sections, and what is the frequency for R = 10 kΩ, C = 0.5 nF?"

GUESS FIRST

The transistor gives 180°. How much must the RC network give, and how many degrees can one RC section give at most?

How it works



The feedback network: three CR sections, each adding phase lead. At one frequency the total is exactly 180°.

- The CE transistor inverts: **180°**. The RC network must add the other **180°** at the oscillation frequency, so the loop totals 360°.
- One RC section can give at most 90°, and only at f → 0, where the signal is gone. So **three** sections are used, about 60° each (in the manual's simple picture), to make 180° at a usable frequency.
- At exactly that frequency, the ladder's output is 1/29 of its input. So the **amplifier gain must be ≥ 29** for the loop gain to reach 1.

$$f = \frac{1}{2\pi RC\sqrt{(2N)}} = \frac{1}{2\pi RC\sqrt{(6)}} \qquad (N = 3 \text{ sections})$$

For R = 10 kΩ, C = 0.5 nF:

$$f = \frac{1}{2\pi \times 10^{4} \times 0.5 \times 10^{-9} \times 2.449} = 13.0 \text{ kHz}$$

For C = 1 nF: **6.50 kHz**.

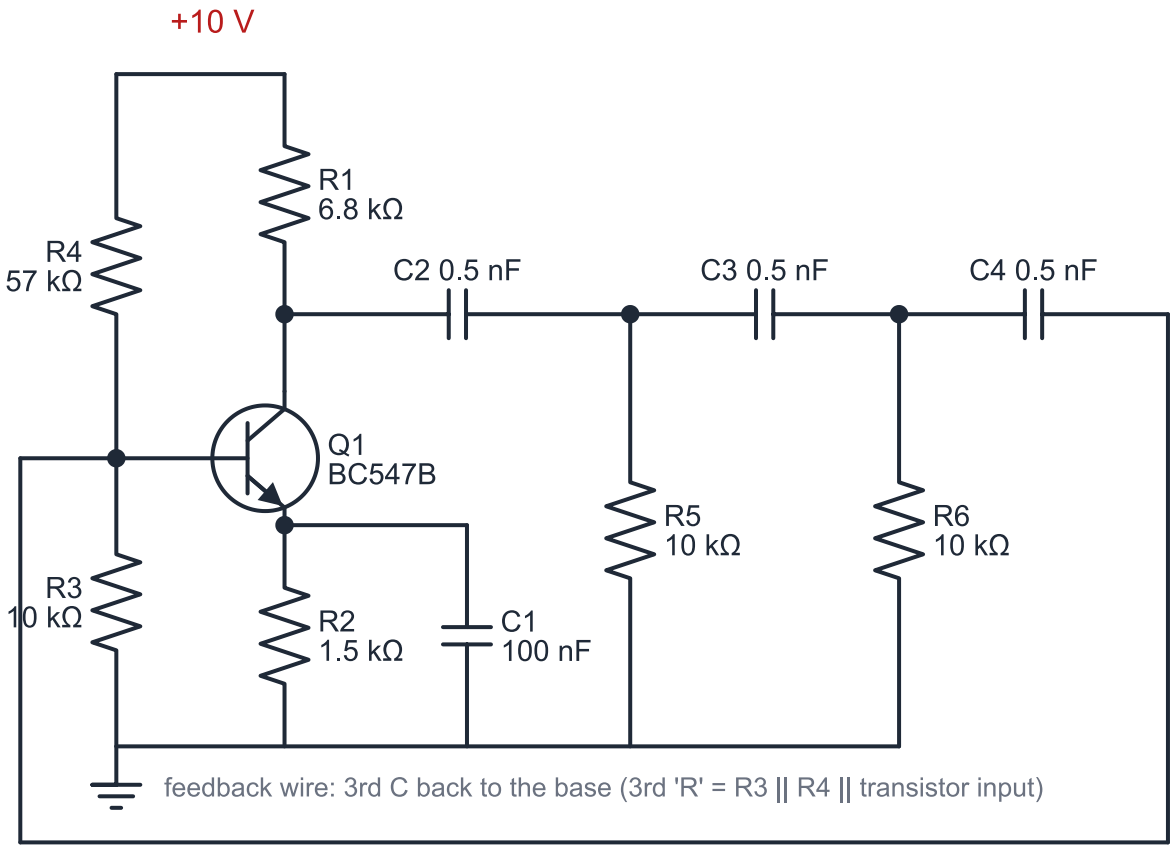
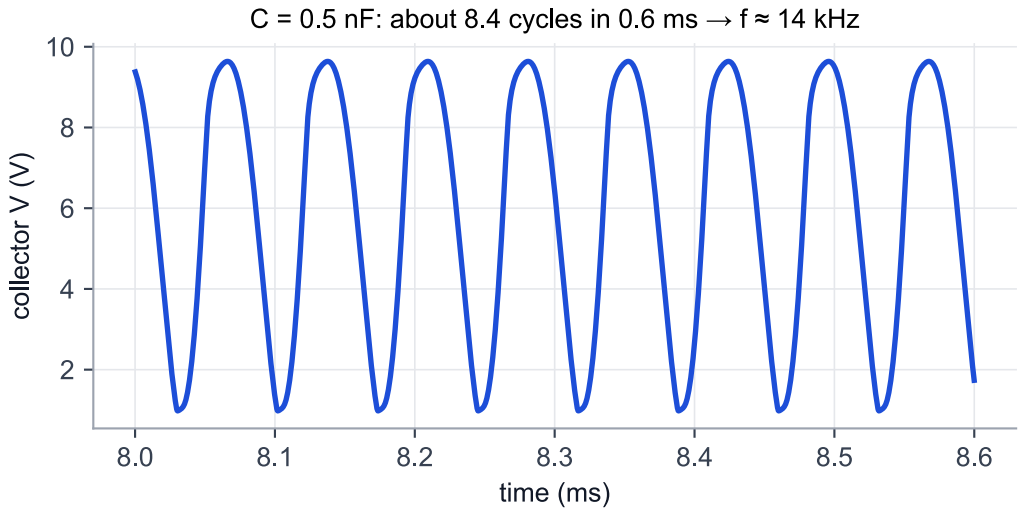


Fig 11.1, redrawn: a BC547B CE stage (10 V) with a three-section CR ladder from collector back to base.

DO IT (LTSPICE)

1. Draw Fig 11.1: Q1 = BC547B; R4 57k, R3 10k (bias); R1 6.8k (collector); R2 1.5k with C1 100n (emitter); ladder C2/C3/C4 = 0.5n, R5/R6 = 10k; V1 = 10 V. The third C goes back to the **base**.
2. Directive: `.tran 0 5m 2.5m` (simulate 5 ms, and plot only from 2.5 ms, after start-up). The circuit starts oscillating on its own. If the trace stays flat, add `startup` at the end of the directive to kick it.
3. Plot the collector voltage. Place cursor 1 on one peak and cursor 2 on a peak N cycles later. The readout gives the time difference and **Freq** (1/Δt). Divide the time difference by N for one period.
4. Change all three capacitors to `1n` and repeat.

What you should see



LTspice collector voltage (C = 0.5 nF): a steady oscillation of about 8.7 V p-p.

S.No.	R	C	Observed f (LTspice)	Theoretical f	% error
1	10 kΩ	0.5 nF	13.96 kHz	13.00 kHz	7.4 %
2	10 kΩ	1 nF	8.40 kHz	6.50 kHz	29 %

$$\% \text{ error} = \frac{|f_{theory} - f_{observed}|}{f_{theory}} \times 100 = \frac{|13.00 - 13.96|}{13.00} \times 100 = 7.4\,\%$$

Why the error exists (and grows): the formula assumes three **identical** sections and an ideal amplifier that doesn't load them. In the real circuit:

- the first C is driven through the 6.8 kΩ collector resistor, not from a zero-resistance source;
- the "third R" is not 10 kΩ but the base side: R3 ∥ R4 ∥ the transistor's input resistance, about 1–2 kΩ.

So the real network differs from the ideal one, and so does its 180° frequency. A % error of several percent is normal. LTspice shows the mismatch is worse at 1 nF.

WRITE IN YOUR RECORD

Result: the phase-shift oscillator produced **13.96 kHz** (C = 0.5 nF; theory 13.0 kHz, error 7.4 %) and **8.40 kHz** (C = 1 nF; theory 6.50 kHz). Increasing C (or R) lowers the frequency, as f ∝ 1/RC predicts.

Theory for the record: an oscillator is an amplifier with positive feedback that sustains an output without any input. The Barkhausen criterion requires a loop gain of at least 1 and a total loop phase shift of 360°. In the RC phase-shift oscillator, a CE transistor stage provides 180°, and a three-section RC ladder provides another 180° at $f = 1/(2\pi RC\sqrt{6})$. The ladder attenuates by 1/29 at that frequency, so the amplifier gain must be at least 29. Using more sections (e.g. four, 45° each) improves frequency stability. Varying R or C tunes the frequency. Used for audio-frequency sine generation.

CHECK 1

(a) Why can't two RC sections work? (b) The amplifier's gain is 20. What happens? (c) Calculate f for R = 4.7 kΩ, C = 1 nF.

Answers

Opening question. The network must add 180°. One section gives less than 90° (90° only at zero frequency), so at least three are needed: ≈ 60° each in the simple picture. $f = 1/(2\pi \times 10\text{k} \times 0.5\text{n} \times \sqrt{6}) =$ **13.0 kHz**.

Check 1. (a) Two sections reach 180° only at $f \to 0$, where the signal is fully attenuated, so the loop can never meet the phase condition at a usable frequency. (b) Loop gain = 20/29 < 1: any oscillation dies out and the output stays flat. (c) $f = 1/(2\pi \times 4700 \times 10^{-9} \times 2.449) =$ **13.8 kHz**.

Exp 12 · MOSFET inverter (resistive load)

Manual pp. 34–35 · LTspice · CO4 · theory: ECE2101 unit 4 (MOSFETs) · links straight to the Digital Electronics lab (logic gates)

9 JFET curves → 10 CS amplifier → 11 Oscillator → **12 MOSFET inverter**

What you already have:

- From Exp 9, a FET's gate voltage controls its channel.
- From Exp 5, a transistor can be a switch: cut-off = open, saturation = closed.
- From digital electronics: NOT gate, logic 1 and logic 0.

Three terms first:

- **Enhancement nMOS:** a MOSFET whose gate is insulated by oxide (so it draws **no** gate current). It has **no** channel at $V_{GS} = 0$. A channel forms only when $V_{GS} > \mathbf{V_{TH}}$ (threshold voltage).
- **Inverter:** a NOT gate. Input high gives output low, and input low gives output high.
- **VTC (voltage transfer characteristic):** V_{out} plotted against V_{in}.

MANUAL EXP 12, AIM

Analyse an inverter circuit using a MOSFET. Viva form: "Explain the two states of the circuit, and mark V_{OH}, V_{OL}, V_{IL}, V_{IH} and V_M on the VTC."

GUESS FIRST

When the input is 0 V, is the MOSFET on or off? What is the output then?

How it works

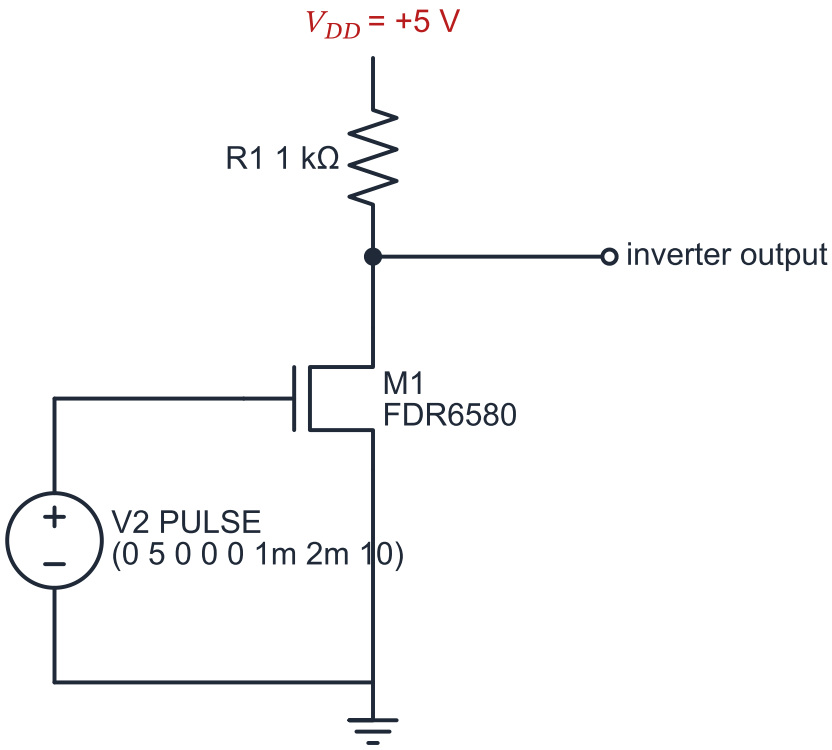


Fig 12.1, redrawn: 1 kΩ from +5 V to the drain (output); input pulse on the gate; source to ground.

Input	MOSFET	Current	Output
0 V (logic 0), below V_{TH}	**off** (cut-off): no channel	I_D = 0, so no drop across the 1 kΩ	$V_{out} = V_{DD}$ = **5 V (logic 1)**
5 V (logic 1), above V_{TH}	**on**, fully (deep triode): the channel is a very low resistance (≈ 9 mΩ for this part)	I_D ≈ 5 V/1 kΩ = 5 mA	V_{out} ≈ 5 mA × 9 mΩ ≈ **0 V (logic 0)**

VTC terms:

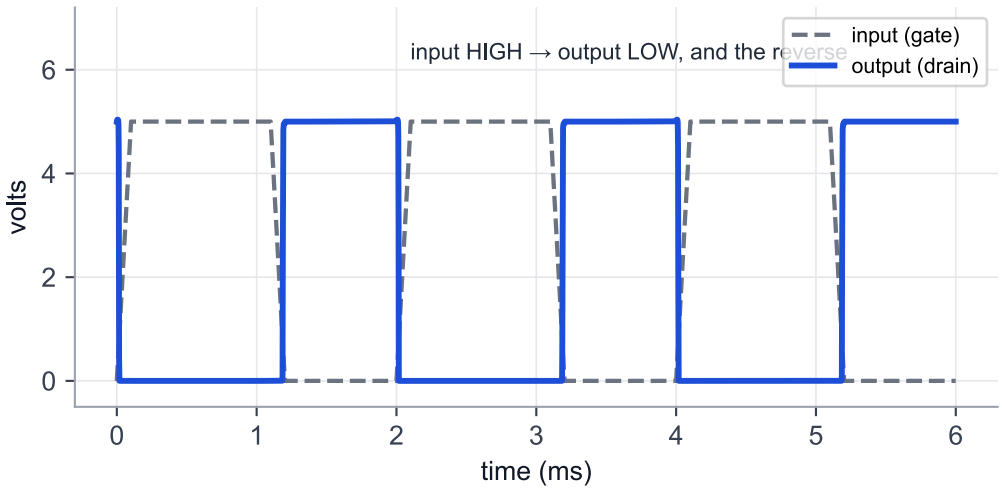
Term	Meaning
V_{OH} / V_{OL}	output high / output low levels
V_{IL}	the highest input still read as 0. Where the VTC slope first reaches −1.
V_{IH}	the lowest input read as 1. Where the slope returns to −1.
V_M	switching threshold, where $V_{out} = V_{in}$
noise margins	$NM_L = V_{IL} - V_{OL}$ and $NM_H = V_{OH} - V_{IH}$: how much noise an input can carry before the gate misreads it

Do it

DO IT (LTSPICE)

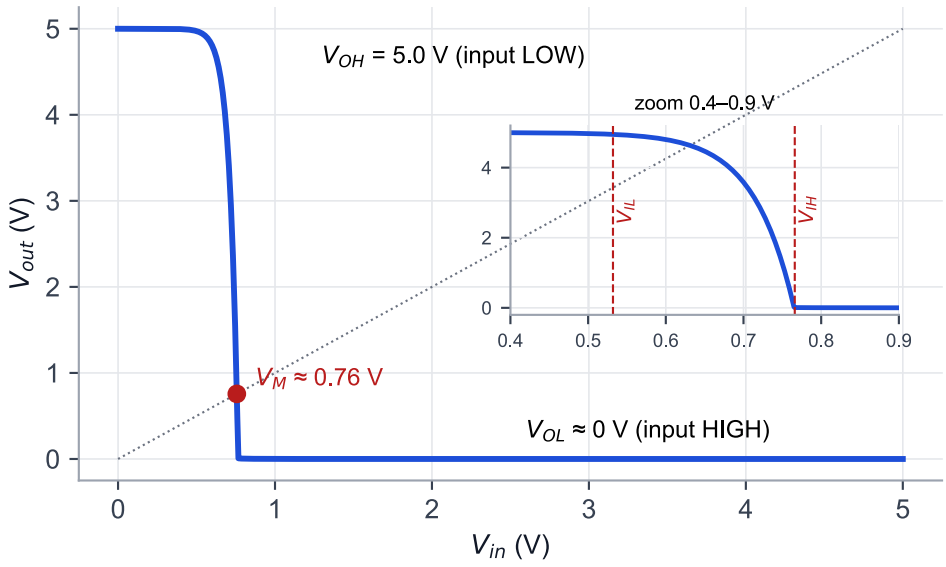
1. Draw Fig 12.1: M1: Pick New MOSFET → **FDR6580**; R1 = 1k; V1 = 5 V.
2. V2: **PULSE**: V1 = 0, V2 = 5, Tdelay = 0, Trise = 0, Tfall = 0, Ton = 1m, Tperiod = 2m, Ncycles = 10, which gives `PULSE(0 5 0 0 0 1m 2m 10)`.
3. Directive: `.tran 0 20m 0 1u`. The manual's bare `.tran 20m` works too, but its coarse steps show slanted edges and a false spike up to about 6.8 V.
4. Plot the input and output. Set the y-axis limits manually to −0.5 … 5.5 V. Read V_{min} and V_{max} of each.
5. **VTC (extra marks):** replace the pulse with a DC source, run `.dc V2 0 5 0.001`, and plot V(out) against V2.

What you should see



Transient: the output is the input upside-down: 5 V when the input is 0, about 0 V when the input is 5 V.

Voltage	V_{min}	V_{max}
V_{in}	0 V	5 V
V_{out}	≈ 0 V (0.04 mV)	5.0 V



VTC from LTspice: a very sharp switch near 0.76 V (zoom inset).

V_{OH}	V_{OL}	V_{IL}	V_{IH}	V_M	NM_L	NM_H
5.00 V	≈ 0 V	0.53 V	0.77 V	0.76 V	0.53 V	4.23 V

Why it switches so low and so sharply: the FDR6580 is a **power** MOSFET. It conducts amps from a tiny gate overdrive, so just past V_{TH} it already pulls the 5 mA that the 1 kΩ allows. The switch therefore happens right at V_{TH}, far below $V_{DD}/2$.

The manual's Fig 12.2 (with $V_{IL} \approx 0.9$ V and $V_{IH} \approx 2.2$ V) is a generic textbook VTC, not this part's.

TRAP: V_{TH} VS V_M

The manual says "V_{th} is the inverter threshold voltage, which is $V_{dd}/2$". These are two different quantities:

- **V_{TH}** is the MOSFET's own turn-on voltage.
- **V_M** is the inverter's switching point (where $V_{out} = V_{in}$).

$V_M = V_{DD}/2$ is the **design goal** of a symmetric CMOS inverter. In this resistor-load circuit, $V_M \approx 0.76$ V.

WRITE IN YOUR RECORD

Result: the output is inverted with respect to the input, with maximum voltage **5 V** as logic 1 and minimum voltage **≈ 0 V** as logic 0. (VTC: $V_{IL} = 0.53$ V, $V_{IH} = 0.77$ V, $V_M = 0.76$ V.)

Theory for the record: the inverter is the basic digital gate; NAND, NOR and larger logic are built from the same idea. In the resistive-load nMOS inverter, an enhancement nMOS pulls the output down and a resistor pulls it up to V_{DD}. For $V_{in} < V_{TH}$ the MOSFET is cut off, no current flows, and $V_{out} = V_{DD}$ (logic 1). For a high input the MOSFET conducts in the linear region and pulls the output close to 0 (logic 0). The VTC shows V_{OH}, V_{OL} and the transition region, with V_{IL} and V_{IH} at the unity-slope points and V_M where $V_{out} = V_{in}$. Noise margins are $NM_L = V_{IL} - V_{OL}$ and $NM_H = V_{OH} - V_{IH}$. A drawback: current (and power) flows continuously when the output is low, which CMOS removes.

CHECK 1

(a) How much power does this inverter waste when its output is LOW? When HIGH? (b) Why does the MOSFET's gate draw no DC current? (c) What replaces the resistor in a CMOS inverter, and what does that fix?

Answers

Opening question. Input 0 V: the MOSFET is **off**, so no current flows and the output = **5 V**. Input 5 V: the MOSFET is **on** and the output ≈ 0 V. V_{OH} = 5 V, V_{OL} ≈ 0, V_{IL} = 0.53 V, V_{IH} = 0.77 V, V_M = 0.76 V.

Check 1. (a) LOW: 5 mA flows from the 5 V supply, so P = 5 V × 5 mA = **25 mW**, nearly all in the resistor. HIGH: no current, so ≈ 0 W. (b) The gate is insulated from the channel by a thin oxide layer: a capacitor, which passes no DC. (c) A **pMOS** transistor (pull-up). Only one transistor is on in each state, so no DC current flows from V_{DD} to ground. The static power becomes ≈ 0, and V_{OL} is exactly 0.

Extra · Two-stage RC-coupled amplifier, and the design problem

Manual pp. 3 and 36–38 · LTspice · the design answer is verified spec-by-spec in LTspice (src/sim/design.py)

6 CE amplifier → 7 Emitter follower → 8 Feedback → **Extra: cascade + design**

What you already have:

- From Exp 6, one CE stage: gain ≈ R_C/r_e, bandwidth f_L to f_H.
- From Exp 7, the emitter follower: low output resistance.
- From Exp 8, unbypassed R_E lowers gain and raises input resistance.

Two terms first:

- **Cascade:** stages in series, each one's output feeding the next one's input.
- **RC coupling:** the link between stages is a capacitor (it blocks one stage's DC from the next) plus the next stage's bias resistors.

Part 1 · Two-stage RC-coupled amplifier

MANUAL, ADDITIONAL EXPERIMENT

To measure the gain and plot the frequency response of a two-stage RC-coupled amplifier and determine its 3 dB bandwidth. Viva form: "Two stages: does the gain add or multiply, and what happens to the bandwidth?"

GUESS FIRST

Answer both halves before reading on.

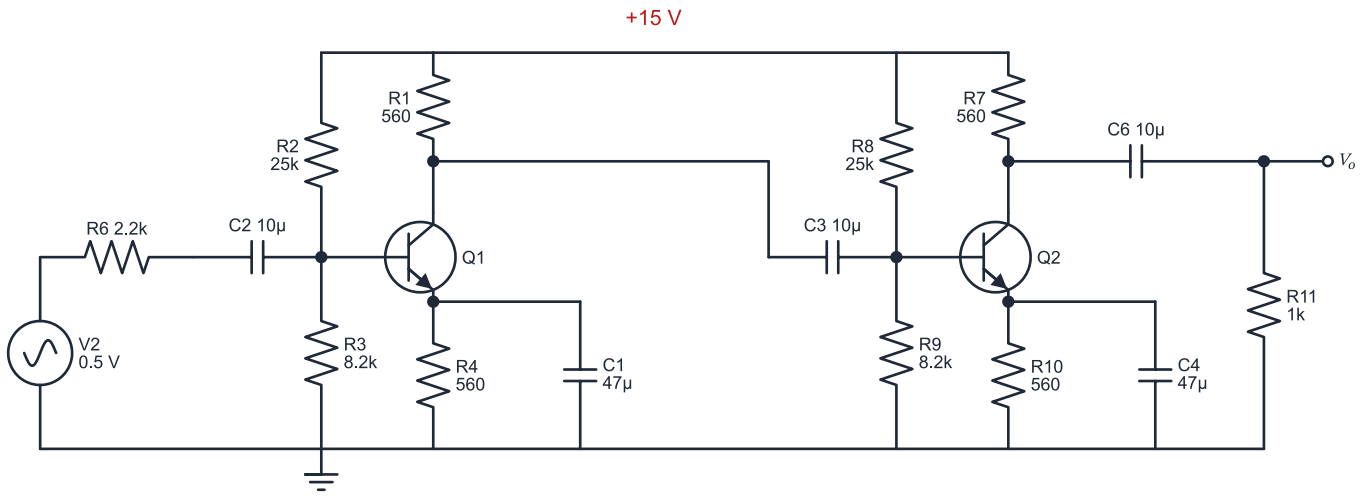


Fig E.2, redrawn: two copies of the Exp 6 stage. C3 couples Q1's collector to Q2's base; the load is R11 = 1 kΩ.

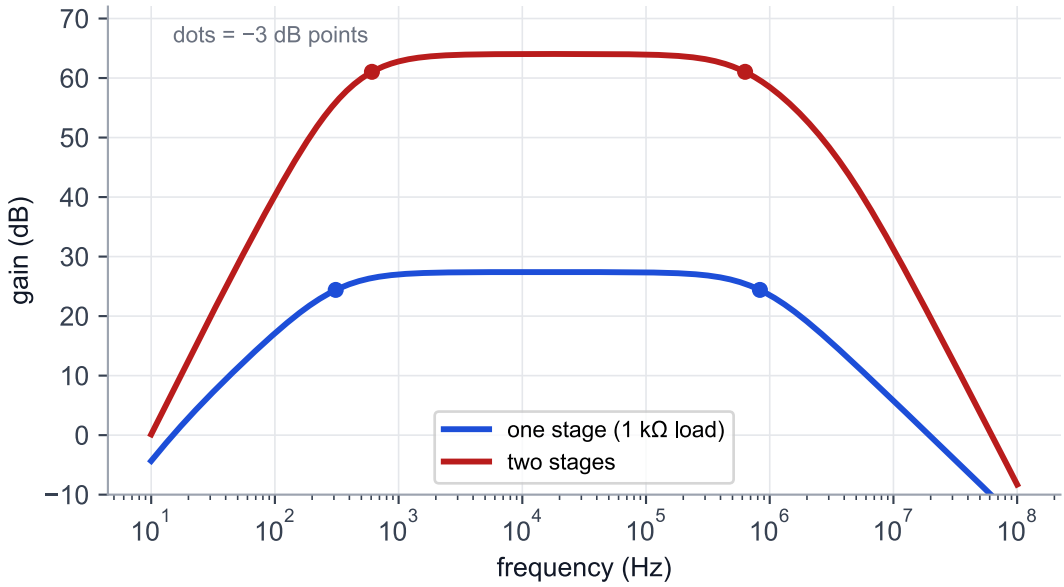
- **Gains multiply:** $A_{total} = A_1 \times A_2$. Decibels therefore **add**: $dB_{total} = dB_1 + dB_2$.
- **Each A must be the gain *in place*.** Stage 1 is loaded by stage 2's input resistance. Stage 2 is driven by stage 1's output resistance (560 Ω) instead of the 2.2 kΩ generator resistance, so it gains more than stage 1 does.
- **Bandwidth shrinks.** At f_L and f_H, *each* stage is already down by some amount, and the losses multiply. So the cascade's −3 dB points move inward: f_L rises and f_H falls.

DO IT (LTSPICE)

1. Draw Fig E.1 (single stage, R5 = 1 kΩ load). Q1 = BC547B, V1 = 15 V, V2 with AC amplitude `0.5` (the manual says 5 V; either works for `.ac`, which is linear).
2. Directive: `.ac dec 10 100 1Meg` (or 500 Hz – 3 MHz as in the procedure). Run, then read the midband dB, f_L and f_H.
3. Copy the stage. Connect C3 from Q1's collector to Q2's base, and put the 1 kΩ load at the end. Run again.
4. Plot both V(b2) (the output of stage 1) and V(out), so you can fill both columns of the table.

TRAP: NO TRANSIENT AT 0.5 V

With a gain of about 1600, a 0.5 V input asks for an 800 V output. Any `.tran` run at that level is pure clipping. Use millivolts for transient, or stick to `.ac`.



LTspice. One stage: 27.4 dB, f_L 310 Hz, f_H 833 kHz. Two stages: 64.0 dB, f_L 609 Hz, f_H 632 kHz.

f (Hz)	100	500	1k	5k	10k	100k	500k	1M	2M	3M
one stage A_v	7.2	19.9	22.4	23.4	23.4	23.3	20.1	15.0	9.0	6.3
two stages A_v	103	990	1379	1584	1591	1575	1252	839	435	264

	Single stage	Two stage
midband gain	23.4 (27.4 dB)	1591 (64.0 dB)
f_L	310 Hz	609 Hz
f_H	833 kHz	632 kHz
bandwidth	≈ 833 kHz	≈ 632 kHz

Gains multiply, in place: inside the cascade at 10 kHz, stage 1 gives 25.8 and stage 2 gives 61.8, so 25.8 × 61.8 = **1594** ≈ 1591 ✓. In dB: 28.2 + 35.8 = 64.0 dB ✓. The stage-alone figure (23.4) is not the in-place gain, so $23.4^2 \neq 1591$.

WRITE IN YOUR RECORD

Result: two-stage gain **64.0 dB** (1591) against **27.4 dB** for one stage; bandwidth falls from ≈ 833 kHz to ≈ **632 kHz** (f_L = 609 Hz, f_H = 632 kHz). Higher gain, narrower bandwidth.

Theory for the record: when amplifier stages are cascaded, the overall voltage gain is the product of the individual (loaded) stage gains, so gains in dB add. In RC coupling, the coupling capacitor passes the AC signal from one stage's collector to the next stage's base while blocking DC, so each stage keeps its own bias. It is cheap and gives a flat midband response. The overall bandwidth is narrower than a single stage's, because each stage's roll-off multiplies: the cascade's lower cut-off rises and its upper cut-off falls.

Part 2 · The design problem (manual p. 3)

MANUAL P. 3 · DESIGN EXPERIMENT

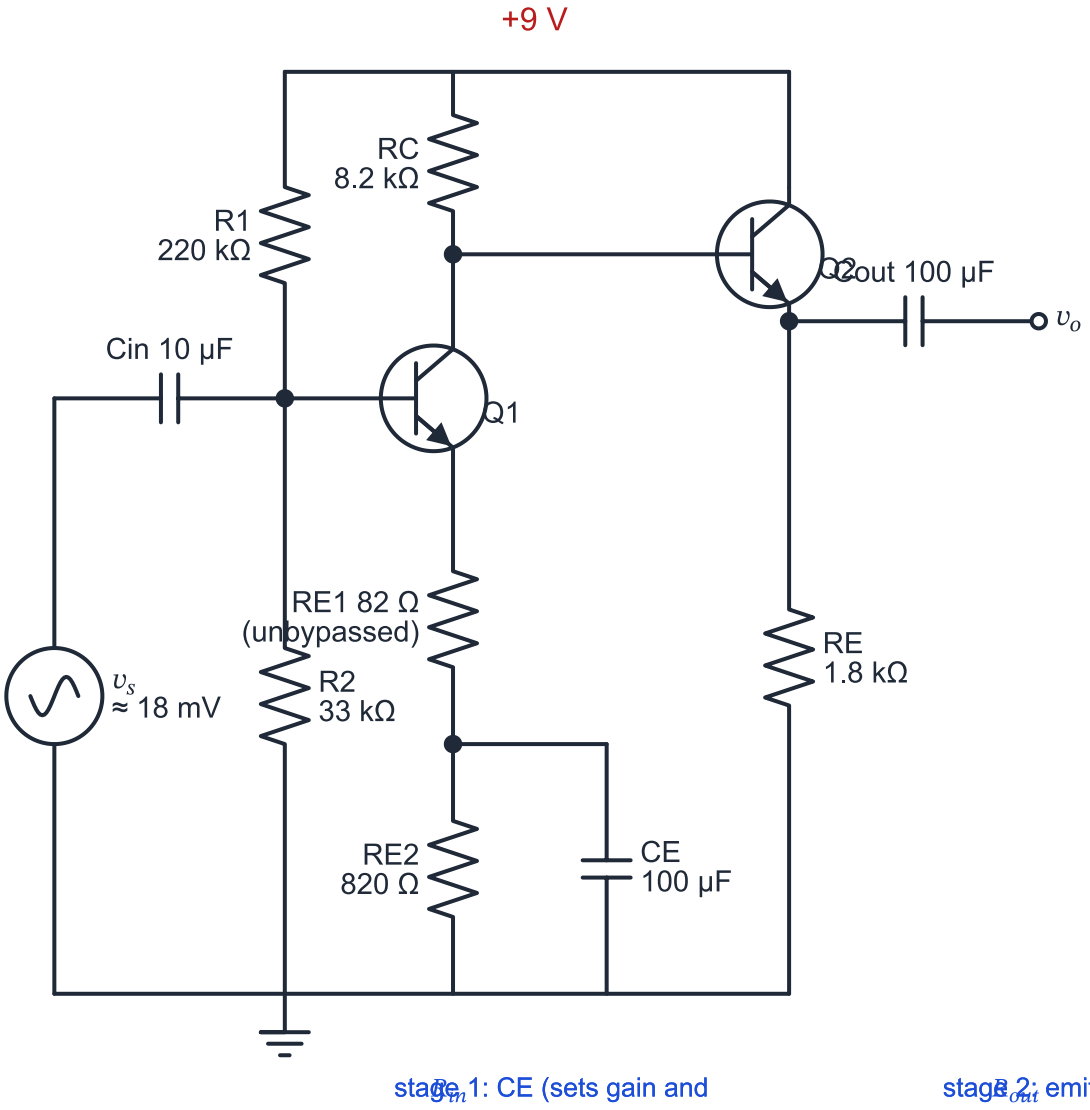
Design a small-signal voltage amplifier with: (1) voltage gain ≥ 50; (2) input resistance ≥ 15 kΩ; (3) output resistance < 100 Ω; (4) maximum unclipped output of 2 V p-p; (5) 9 V supply and low current drain.

GUESS FIRST

Which single stage from Exps 6–8 meets *both* "gain ≥ 50" and "R_{out} < 100 Ω"? If none, what combination does?

The reasoning, one spec at a time:

1. **Gain ≥ 50** calls for a CE stage. But a fully bypassed CE stage has a low R_{in} (≈ βr_e, about 1 kΩ in Exp 6).
2. **R_{in} ≥ 15 kΩ:** leave a small part of R_E **unbypassed** (R_{E1}). The base then sees $\beta(r_e + R_{E1})$, which is much larger, at the cost of some gain (Exp 8, partly). Use large bias resistors.
3. **R_{out} < 100 Ω:** a CE output is ≈ R_C (kilohms), so it fails. Add an **emitter follower** as stage 2 (Exp 7): R_{out} ≈ $r_{e2} + R_C/\beta$, which is tens of ohms.
4. **Low current drain:** run stage 1 at about 0.6 mA and stage 2 at about 2 mA.



Verified design. Stage 1 is a CE with R_{E1} = 82 Ω unbypassed; stage 2 is an emitter follower direct-coupled from Q1's collector. Both BC547B.

Hand design:

- **Stage 1 bias:** V_{B1} = 9 × 33/(220 + 33) = 1.17 V, V_{E1} ≈ 0.5 V, I_{E1} = 0.5/(82 + 820) ≈ 0.55 mA, r_{e1} = 26/0.55 ≈ 47 Ω.
- **Stage 1 gain:** ≈ $R_C/(r_{e1} + R_{E1})$ = 8.2k/(47 + 82) ≈ **64**.
- **R_{in}:** = 220k ∥ 33k ∥ β(r_{e1} + R_{E1}) = 28.7k ∥ (290 × 129 Ω = 37k) ≈ **16 kΩ** ✓.
- **Stage 2:** V_{C1} ≈ 9 − 0.55m × 8.2k ≈ 4.4 V, so V_{E2} ≈ 3.7 V, I_{E2} = 3.7/1.8k ≈ 2 mA, r_{e2} = 13 Ω. R_{out} ≈ 13 + 8.2k/290 ≈ **41 Ω** ✓.
- **Swing:** V_{E2} = 3.7 V sits mid-supply, with room for ±1 V and more ✓.

LTspice check of every spec (`src/sim/design.py`):

Spec	Required	Simulated	
voltage gain (1 kHz, 10 kΩ load)	≥ 50	**59.7**	✓
input resistance	≥ 15 kΩ	**16.2 kΩ**	✓
output resistance	< 100 Ω	**39 Ω**	✓
unclipped output	2 V p-p	**2.95 V p-p** clean (±1.43/−1.52 V at 25 mV in)	✓
supply / current	9 V, low	9 V, **2.68 mA** total	✓
bandwidth (bonus)		14 Hz to 4.4 MHz	

CHECK 1

(a) Two stages have midband gains of 20 dB and 26 dB in place. Give the overall gain as a ratio. (b) Why not reach "gain ≥ 50" with a fully bypassed emitter in the design? (c) In the design, which single resistor would you change to raise the gain, and what does it cost?

Answers

Part 1 question. Gains **multiply** (dB add). Bandwidth **shrinks**: f_L rises and f_H falls.

Part 2 question. No single stage meets both specs: CE has gain but high R_{out}, and CC has low R_{out} but gain < 1. So use **CE followed by CC**.

Check 1. (a) 46 dB = $10^{46/20} \approx$ **200**. (b) The input resistance would drop to about $\beta r_e \approx 13$ kΩ in parallel with the bias network (and lower at higher current), failing the 15 kΩ spec. The unbypassed R_{E1} is what lifts R_{in}. (c) Lower R_{E1} (or raise R_C). Lower R_{E1} raises the gain but lowers R_{in} (back towards the 15 kΩ limit) and adds distortion. Raising R_C raises the gain but lowers V_{C1}, which eats into the output swing.

16 · Revision set: one question per idea, for the night before

Answer from memory on paper, then check the answers page. Any miss points back to its chapter, shown in brackets.

Draw from memory

1. The zener regulator (Exp 2) with values, the clamper (Exp 4), and the CE amplifier (Exp 6) with every part labelled. Then say each part's job in one line.

Diodes

2. A DSO shows 6 divisions peak-to-peak at 0.5 V/div on a sine. Give V_{rms}. *(Exp 0)*
3. Zener: define dynamic resistance, and say why "dynamic". *(Exp 1)*
4. A 6.2 V zener: which breakdown mechanism, and why is its temperature drift small? *(Exp 1)*
5. In the zener regulator, V_{in} goes up by 3 V. Which part takes almost all of it? *(Exp 2)*
6. Write the formulas for % line regulation and % load regulation. *(Exp 2)*
7. A positive clipper uses a 3 V battery. Clipping level? *(Exp 3)*
8. Clipper vs clamper: what happens to V_{pp} in each? *(Exps 3–4)*
9. What condition on RC makes a clamper work at 1 kHz? *(Exp 4)*

BJT

10. Bias of the two junctions in active, saturation and cut-off. *(Exp 5)*
11. Name the four h-parameters, and say from which curve, with what held constant, each is read. *(Exp 5)*
12. In Exp 6, which capacitor mostly sets f_L, and why does the gain fall at high f? *(Exp 6)*
13. Define bandwidth and the −3 dB point. *(Exp 6)*
14. Why is the CE output 180° out of phase? *(Exp 6)*
15. Emitter follower: gain, phase, input and output resistance (high/low). *(Exp 7)*
16. Removing C_E: name the type of feedback, and two things that improve. *(Exp 8)*

FET, oscillator, MOSFET

17. Define g_m, r_d, μ and show $\mu = g_m r_d$. *(Exp 9)*
18. Write Shockley's equation, and define I_{DSS} and V_P. *(Exp 9)*
19. CS amplifier gain formula, and why the gate divider can use megohms. *(Exp 10)*
20. Barkhausen criterion, and the frequency formula of the RC phase-shift oscillator. *(Exp 11)*
21. Why must the amplifier's gain be at least 29 in the phase-shift oscillator? *(Exp 11)*
22. Resistive-load inverter: the MOSFET's state and the output for each input. Define V_M. *(Exp 12)*
23. Two cascaded stages: what happens to the gain and to the bandwidth? *(Extra)*
24. In LTspice, how do you enter 1 MΩ, and which analysis gives a frequency response? *(01)*

Answers

1. See each chapter's redrawn figure and part table.
2. V_{pp} = 3 V, so V_p = 1.5 V and V_{rms} = 1.5/√2 = **1.06 V**.
3. $r_z = \Delta V_Z/\Delta I_Z$ in breakdown: the slope of the curve, not V/I at one point.
4. A mix of tunnelling (Zener) and avalanche. Their temperature coefficients are opposite (− and +), so they nearly cancel.
5. The series resistor R1. The extra current goes into the zener, and V_o barely moves.
6. Line: $\Delta V_o/\Delta V_{in} \times 100$. Load: $(V_{NL} - V_{FL})/V_{FL} \times 100$.
7. 3 + 0.7 = **3.7 V**.
8. A clipper reduces V_{pp}. A clamper keeps V_{pp} and shifts the DC level.
9. RC ≫ T (= 1 ms). Here RC = 10 ms.
10. Active: BE forward, BC reverse. Saturation: both forward. Cut-off: both reverse.
11. $h_{ie} = \Delta V_{BE}/\Delta I_B$ (I_B fixed). $h_{re} = \Delta V_{BE}/\Delta V_{CE}$ (I_B fixed). $h_{fe} = \Delta I_C/\Delta I_B$ (V_{CE} fixed). $h_{oe} = \Delta I_C/\Delta V_{CE}$ (I_B fixed).
12. The emitter bypass C_E. At high f, the transistor's internal junction capacitances (Miller effect) and stray capacitance shunt the signal.
13. BW = $f_H - f_L$, between the frequencies where the gain is 0.707 × max = −3 dB (half power).
14. A rising base voltage raises I_C, which increases the drop across R_C, so the collector voltage falls.
15. Gain ≈ 1 (slightly less), in phase; input resistance high, output resistance low; used as a buffer.
16. Current-series negative feedback. Gain becomes stable (independent of β), bandwidth widens, distortion and noise fall, input resistance rises.
17. $g_m = \Delta I_D/\Delta V_{GS}$ (V_{DS} const), $r_d = \Delta V_{DS}/\Delta I_D$ (V_{GS} const), $\mu = -\Delta V_{DS}/\Delta V_{GS}$ (I_D const) = $(\Delta V_{DS}/\Delta I_D) \times (\Delta I_D/\Delta V_{GS}) = r_d g_m$.
18. $I_D = I_{DSS}(1 - V_{GS}/V_P)^2$. I_{DSS} = drain current at V_{GS} = 0 in saturation. V_P = the V_{GS} at which $I_D \to 0$.
19. $A_v = -g_m(R_D \parallel R_L)$. The gate draws no current, so megohm resistors drop no voltage.
20. $|A\beta| \geq 1$ and loop phase 360°. $f = 1/(2\pi RC\sqrt{6})$.
21. At the oscillation frequency, the three-section RC ladder attenuates the signal to 1/29.
22. Input low: MOSFET off, output = V_{DD}. Input high: MOSFET on, output ≈ 0. V_M = the input voltage at which $V_{out} = V_{in}$.
23. Gains multiply (dB add). Bandwidth shrinks.
24. `1Meg` (`1M` is milli). `.ac`.

00 · Start here: the trainer kit, wiring discipline, and the whole lab

ECE2131 Digital Electronics Lab · source: the lab manual (2025), 55 pp. · every design in this pack is machine-checked over all inputs (src/verify.py, 49 checks pass) · marks: 100 = internal 60 + external 40; exam 3 hours

The lab as one path

1 Gates & universality → 2 Adders/subtractors → 3 NAND/NOR design → 4 Code converters →
5 Comparator & parity → 6–8 MUX, DEMUX, decoder → 9–10 Flip-flops →
11–12 Counters & shift registers → 13 FSM → 14 TTL

- **Experiments 1–8 are combinational:** the output depends only on the present inputs.
- **Experiments 9–13 are sequential:** they add memory (flip-flops), so the output also depends on the past.
- **Exp 14** looks inside one gate.

You already know the theory (ECE2102). What this pack adds is the part the exam tests at the bench: **which IC, which pin, which wire, what the LEDs should show, and why.**

The IC trainer kit

Part of the kit	Use
+5 V and GND terminals	power every IC: **+5 V only. More destroys TTL.**
logic input switches	give 0/1 to inputs (up = 1)
LED indicators	show outputs (lit = 1)
clock generator (≈ 1 Hz)	slow continuous clock for counters
pulser / mono-pulse (debounced)	one clean clock edge per press. Use this for flip-flop tables.
breadboard	5-hole vertical strips are one node; the long rails are the power buses

Wiring discipline (the manual's "common causes of problems", made into a routine)

EVERY CIRCUIT, EVERY TIME

1. Power **off**.
2. Plug each IC across the breadboard's centre gap, with the **notch to the left** (or up) and all ICs facing the same way. Pin 1 is next to the notch or dot. Pins count **anticlockwise** seen from the top.
3. Wire **VCC and GND of every IC first**: - 14-pin gates: VCC = 14, GND = 7. - 16-pin MSI chips: VCC = 16, GND = 8. - **Exceptions: 7483 (VCC 5, GND 12) and 7473 (VCC 4, GND 11).**
4. Wire short connections before long ones, and tick each one off on the circuit diagram as you go.
5. Tie every unused input. Unused AND/NAND inputs go to an input you are using or to +5 V; unused OR/NOR inputs go to GND. A floating TTL input reads as HIGH, which silently forces a NOR output to 0.
6. Have a partner check the wiring. Then power **on**, and walk the truth table row by row.
7. Never drive one gate input from two outputs. Never rewire with the power on.

WHEN NOTHING WORKS

In this order:

1. Is the IC powered? Check with the meter on the IC's own VCC/GND pins.
2. Is the IC in the right way round?
3. Is the IC dead? Swap it.
4. Is the LED wired to the output pin or to an input pin?

Most "wrong" truth tables are a missing ground or an IC in backwards.

How each chapter works

Each experiment gives you, in order:

1. the aim as the question;
2. a guess-first line;
3. the design (expressions, already checked);
4. the **pin-to-pin wiring table**;
5. what the LEDs should show;
6. a ready "write in your record" block;
7. traps and a check, with answers at the end of the chapter.

The pin diagram of every IC appears where it is first used.

Errors in the manual

Where	The manual says	Correct
Exp 1, NOT gate	truth table A = 0 → Y = 0, then 0 → 1	A = 0 → Y = 1; A = 1 → Y = 0
IC table	74138 listed as "negative edge triggered dual J-K F/F"	74138 is a 3:8 decoder (the 7473 is the JK)
Exp 2, Fig 2.5	"Pin Diagram of IC 7485 (4-bit binary full adder)"	the adder is the **7483** (the 7485 is the comparator)
Exp 2, subtractor	"borrow output at C_{out}"	for A − B, C_{out} = 1 means **no** borrow (A ≥ B); C_{out} = 0 means the answer is negative, in 2's complement
Exp 4, Table 4.2	"BCD to grey code"	BCD to **Excess-3**
Exp 7	aim says "multiplexer"; parts 74151, 74153	demultiplexer; the 1:8 needs the **74138**
Exp 13	the list says detect **101**; the experiment designs **1010**	this pack gives both
Exp 13 (ii)	$F^+ = XY + FY''$	$F^+ = XY + FY'$
Exp 11, ring counter	"clear all flip-flops, then clock"	a ring counter cleared to 0000 stays at 0000 forever. One flip-flop must be **preset** to 1.
Exp 14	no circuit given	see Exp 14 in this pack (LTspice-verified)

Exp 1 · Logic gates, and NAND/NOR as universal gates

Manual pp. 1–4 · ICs 7400, 7402, 7404, 7408, 7432, 7486 · trainer kit

1 Gates & universality → 2 Adders/subtractors → 3 NAND/NOR design → 4 Code converters

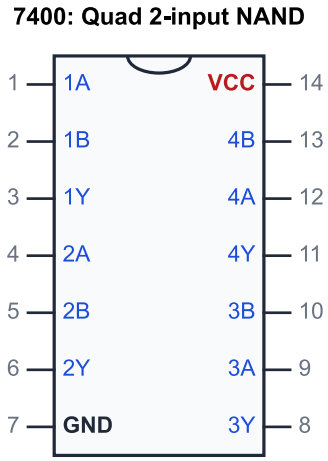
MANUAL EXP 1, AIM

Verify the truth tables of AND, OR, NOT, NAND, NOR and XOR; realise NOT, AND and OR using only NAND gates, then only NOR gates. Viva form: "Why are NAND and NOR called universal? Show OR from NANDs only."

GUESS FIRST

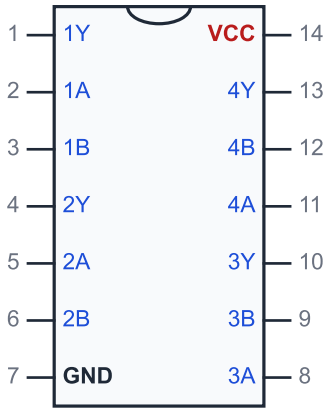
How many 2-input NANDs make an OR? Sketch it before reading on.

The six ICs



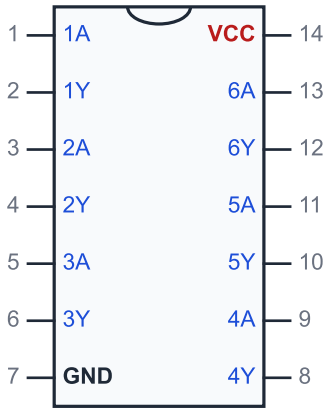
7408 (AND), 7432 (OR) and 7486 (XOR) use exactly this layout. Only the gate inside changes.

7402: Quad 2-input NOR



The 7402 is different: each output comes FIRST (pins 1, 4, 10, 13).

7404: Hex inverter



Six inverters.

Testing a gate IC (e.g. the 7408): connect A → pin 1, B → pin 2, LED ← pin 3, VCC 14, GND 7. Walk 00, 01, 10, 11. Repeat for gates 2–4 if you have time: a chip can have one dead gate.

A B	AND	OR	NAND	NOR	XOR	NOT A
0 0	0	0	1	1	0	1
0 1	0	1	1	0	1	1
1 0	0	1	1	0	1	0
1 1	1	1	0	0	0	0

Universality

A gate is **universal** if NOT, AND and OR (and so any function) can be built from it alone.

Make	From NAND (7400)	From NOR (7402)
NOT A	NAND(A, A)	NOR(A, A)
AND	NAND, then a NAND-inverter: ((AB)')'	invert both inputs, then NOR: (A' + B')' = AB (De Morgan)
OR	invert both inputs, then NAND: (A'B')' = A + B (De Morgan)	NOR, then a NOR-inverter: ((A + B)')'
gates needed	NOT 1, AND 2, OR 3	NOT 1, OR 2, AND 3

Wiring (the manual's pin numbers)

Circuit	Wires
NOT (NAND)	A → 7400 pins 1 and 2; Y = pin 3
AND (NAND)	A → 1, B → 2; pin 3 → pins 4 and 5; Y = pin 6
OR (NAND)	A → 1 and 2 (pin 3 = A'); B → 4 and 5 (pin 6 = B'); pin 3 → 9, pin 6 → 10; Y = pin 8
NOT (NOR)	A → 7402 pins 2 and 3; Y = pin 1
OR (NOR)	A → 2, B → 3; pin 1 → pins 5 and 6; Y = pin 4
AND (NOR)	A → 2 and 3 (pin 1 = A'); B → 5 and 6 (pin 4 = B'); pin 1 → 8, pin 4 → 9; Y = pin 10

WRITE IN YOUR RECORD

Result: the truth tables of AND, OR, NOT, NAND, NOR and XOR were verified. NOT, AND and OR were realised using only NAND gates (1, 2, 3 gates) and using only NOR gates (1, 3, 2 gates), and their truth tables verified.

Theory for the record: logic gates implement Boolean operations: AND (Y = AB), OR (Y = A + B), NOT (Y = A'), NAND (Y = (AB)'), NOR (Y = (A + B)'), XOR (Y = AB' + A'B). TTL ICs of the 74 series contain several gates in a 14-pin DIP, with VCC on pin 14 and GND on pin 7. NAND and NOR are universal gates: NOT, AND and OR, and therefore any Boolean function, can be built using only one of them. By De Morgan's theorems, (A'B')' = A + B and (A' + B')' = AB.

CHECK 1

(a) Make XOR from 2-input NANDs only: how many gates? (b) Why does a floating input on a 7402 NOR give a stuck-at-0 output? (c) Which pin is the output of gate 3 on the 7400, and on the 7402?

Answers

Opening question. Each of them alone can make NOT, AND and OR, and so any function. OR = NAND(NAND(A,A), NAND(B,B)): **3 NANDs**.

Check 1. (a) **4 NANDs**: n1 = (AB)', then A $\oplus$ B = NAND(NAND(A, n1), NAND(B, n1)). (b) A floating TTL input behaves as logic 1, and a NOR with any input at 1 outputs 0. (c) 7400 gate 3: inputs 9, 10 → output **8**. 7402 gate 3: inputs 8, 9 → output **10**.

Exp 2 · Half/full adder and subtractor; 7483 adder, 2's-complement subtractor, BCD adder

Manual pp. 5–9 · ICs 7486, 7408, 7432, 7404, 7483 · all expressions checked over every input

MANUAL EXP 2, AIM

Design and test a half adder/half subtractor and a full adder/full subtractor; study the 7483. Viva form: "How does a single 7483 subtract, and what does C_{out} tell you after a subtraction?"

GUESS FIRST

Subtraction A − B = A + (2's complement of B). How would you get B's 2's complement using XOR gates and the carry-in?

Designs

Circuit	Outputs
half adder	$S = A \oplus B$, $C = AB$
half subtractor (A − B)	$D = A \oplus B$, $B_o = A'B$
full adder	$S = A \oplus B \oplus C_{in}$, $C_{out} = AB + BC_{in} + AC_{in}$ $(= AB + C_{in}(A \oplus B))$
full subtractor	$D = A \oplus B \oplus B_i$, $B_o = A'B + (A \oplus B)'B_i$

A B C_{in}	000	001	010	011	100	101	110	111
full adder S C_{out}	0 0	1 0	1 0	0 1	1 0	0 1	0 1	1 1
full subtractor D B_o (third input = B_i)	0 0	1 1	1 1	0 1	1 0	0 0	0 0	1 1

Wiring

Circuit	Wires (7486/7408/7432: gate 1 = pins 1, 2 → 3; gate 2 = 4, 5 → 6; gate 3 = 9, 10 → 8; gate 4 = 12, 13 → 11)
half adder	A → 7486-1 and 7408-1; B → 7486-2 and 7408-2. **S = 7486-3, C = 7408-3**
half subtractor	D = 7486 (A → 1, B → 2 → 3). A → 7404-1 (pin 2 = A'); 7404-2 → 7408-1, B → 7408-2; **B_o = 7408-3**
full adder: sum	7486 gate 1: A, B → 3; gate 2: pin 3 → 4, C_{in} → 5; **S = 7486-6**
full adder: carry (Fig 2.3)	7408: A, B → 3 (AB); B → 4, C_{in} → 5 → 6 (BC_{in}); A → 9, C_{in} → 10 → 8 (AC_{in}). 7432: 7408-3 → 1, 7408-6 → 2 → 3; 7432-3 → 4, 7408-8 → 5; **C_{out} = 7432-6**
full subtractor: borrow	7404: A → 1 → 2 (A'); 7486-3 (A ⊕ B) → 3 → 4. 7408: 7404-2 → 1, B → 2 → 3 (A'B); 7404-4 → 4, B_i → 5 → 6. 7432: 3 → 1, 6 → 2; **B_o = 7432-3**

The 7483: 4-bit adder (note the odd power pins)



VCC is pin 5 and GND is pin 12, not 16/8. Wiring power to 16 and 8 is the classic mistake with this chip.

Signal	A3 A2 A1 A0	B3 B2 B1 B0	C_{in}	S3 S2 S1 S0	C_{out}
pin	1, 3, 8, 10	16, 4, 7, 11	13	15, 2, 6, 9	14

4-bit adder / 2's-complement subtractor (Fig 2.6)

- Feed each B bit through an XOR whose other input is the control **M** (7486), and also send **M to C_{in}** (pin 13).
- **M = 0:** B passes unchanged, C_{in} = 0, so the 7483 computes **A + B**.
- **M = 1:** each B bit is inverted (1's complement) and C_{in} = 1 adds the 1, giving 2's complement. So the 7483 computes **A − B**.

Wire	
b0 → 7486-1, M → 2 → pin 3 → 7483-11	b1 → 7486-4, M → 5 → pin 6 → 7483-7
b2 → 7486-9, M → 10 → pin 8 → 7483-4	b3 → 7486-12, M → 13 → pin 11 → 7483-16
M → 7483-13 (C_{in})	A3..A0 → pins 1, 3, 8, 10

Try	Inputs	S3..S0	C_{out}	Meaning
7 + 5	M = 0, A = 0111, B = 0101	1100	0	12
7 − 5	M = 1, A = 0111, B = 0101	**0010**	**1**	+2. C_{out} = 1 means no borrow, result positive.
5 − 7	M = 1, A = 0101, B = 0111	**1110**	**0**	C_{out} = 0 means negative: 1110 is −2 in 2's complement

BCD adder (Fig 2.7)

- Add the two BCD digits in 7483 #1.
- If the sum is more than 9, or produced a carry, add 6 (0110) in 7483 #2.

The correction signal is:

$$K = C_4 + S_3S_2 + S_3S_1$$

Make S3S2 and S3S1 with two 7408 gates, then OR all three with two 7432 gates. Feed **0 K K 0** into B3 B2 B1 B0 of 7483 #2, with the first sum on its A inputs and C_{in} = 0. **K is the decimal carry.**

Example: 7 + 5. 7483 #1 gives 1100 (12 > 9), so K = 1. Then 1100 + 0110 = (1)0010. Output: carry **1**, digit **0010**, which reads as 12 ✓.

WRITE IN YOUR RECORD

Result: the truth tables of the half/full adder and half/full subtractor were verified. The 7483 added two 4-bit numbers; with the XOR/M arrangement it subtracted in 2's complement (7 − 5 = 0010, C_{out} = 1; 5 − 7 = 1110, C_{out} = 0). The BCD adder gave 7 + 5 = 1 0010.

Theory for the record: a half adder adds two bits (S = A ⊕ B, C = AB). A full adder adds three, including the carry-in (S = A ⊕ B ⊕ C_{in}, C_{out} = AB + BC_{in} + AC_{in}), and can be built from two half adders and an OR. Subtractors produce a difference and a borrow (half: D = A ⊕ B, B_o = A'B). The 7483 is a 4-bit binary adder with fast carry. Subtraction uses A − B = A + B' + 1, made by XOR-ing B with a control M and feeding M to C_{in}. For BCD addition, 6 is added when the binary sum exceeds 9 or produces a carry.

CHECK 1

(a) In the subtractor, why does C_{out} = 1 mean "no borrow"? (b) BCD 8 + 9: give 7483 #1's output, K, and the final output. (c) A student powers the 7483 on pins 16 and 8. What happens?

Answers

Opening question. XOR each B bit with M (M = 1 inverts it) and feed M into C_{in} (+1). Together that adds the 2's complement of B, so the adder subtracts. After a subtraction, C_{out} = 1 means A ≥ B (positive result); C_{out} = 0 means the result is negative, in 2's-complement form.

Check 1. (a) A + (2^4 − B) = 2^4 + (A − B). When A ≥ B this overflows 4 bits, so carry = 1. When A < B it doesn't. (b) 8 + 9 = 17: #1 gives C_4 = 1, S = 0001, so K = 1. 0001 + 0110 = **0111**, carry 1, i.e. **1 0111** = 17 ✓. (c) Pin 16 is B3 and pin 8 is A1, so the chip gets no power. If the real VCC/GND pins (5, 12) are left floating while inputs are driven, it misbehaves and can be damaged. The outputs are garbage.

Exp 3 · f(W,X,Y,Z) = Σm(1,3,5,7,8,12,14) with basic gates, NAND only, NOR only

Manual p. 10 · ICs 7404, 7408, 7411, 7432 (basic); 7400 + 7410 (NAND); 7402 + 7427 (NOR) · all four forms checked over all 16 inputs

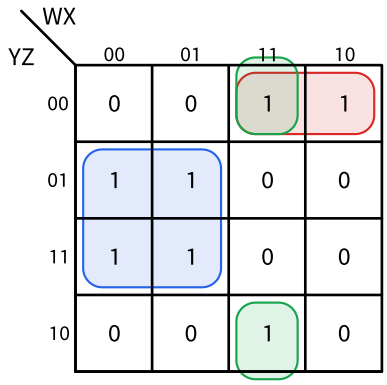
MANUAL EXP 3, AIM

Realise f(W, X, Y, Z) = Σm(1, 3, 5, 7, 8, 12, 14) using (i) basic gates, (ii) only NAND gates, (iii) only NOR gates.

GUESS FIRST

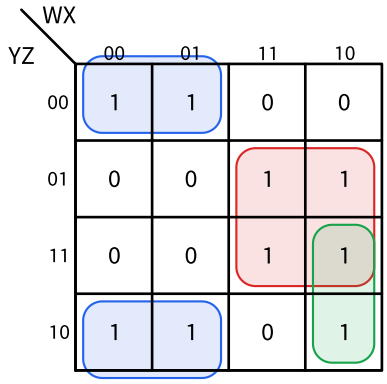
Which form do you minimise for NAND-only, and which for NOR-only: SOP or POS?

Minimise



K-map of f (the 1s). Blue: W'Z (m1, 3, 5, 7); red: WY'Z' (m8, 12); green: WXZ' (m12, 14).

f = W'Z + WY'Z' + WXZ' (SOP, for basic gates and for NAND–NAND)

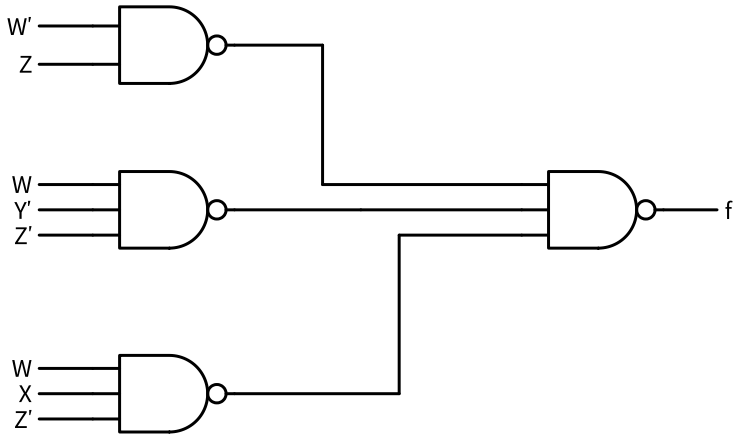


K-map of f' (the 0s). Blue: W'Z'; red: WZ; green: WX'Y.

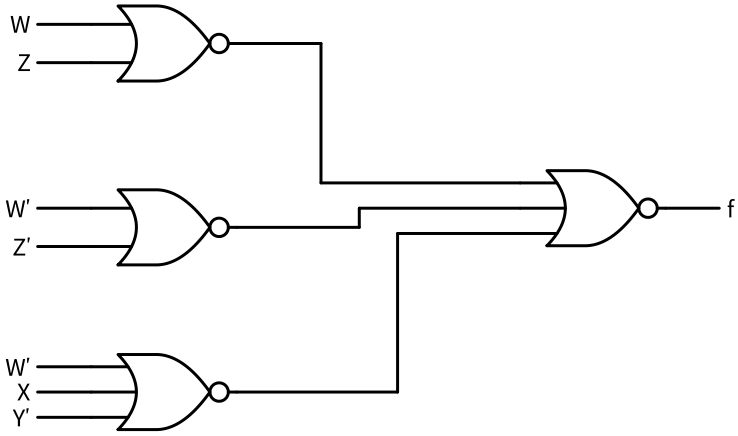
f' = W'Z' + WZ + WX'Y ⇒ f = (W + Z)(W' + Z')(W' + X + Y') (POS, for NOR–NOR)

Rule:

- SOP becomes **NAND–NAND**: replace every gate with a NAND, and keep the inverters.
- POS becomes **NOR–NOR**: replace every gate with a NOR.



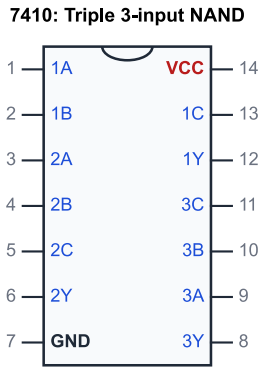
NAND–NAND realisation of the SOP. The inverters W', Y', Z' are NANDs with their inputs tied.



NOR–NOR realisation of the POS.

Wiring

Form	Wires
basic gates	7404: W → 1 → 2 (W'); Y → 3 → 4 (Y'); Z → 5 → 6 (Z'). 7408: 7404-2 → 1, Z → 2 → **3** (W'Z). 7411: W → 1, 7404-4 → 2, 7404-6 → 13 → **12** (WY'Z'); W → 3, X → 4, 7404-6 → 5 → **6** (WXZ'). 7432: 7408-3 → 1, 7411-12 → 2 → 3; 7432-3 → 4, 7411-6 → 5; **f = 7432-6**
NAND only	7400: W → 1, 2 → 3 (W'); Y → 4, 5 → 6 (Y'); Z → 9, 10 → 8 (Z'); 7400-3 → 12, Z → 13 → **11**. 7410: W → 1, 7400-6 → 2, 7400-8 → 13 → **12**; W → 3, X → 4, 7400-8 → 5 → **6**; 7400-11 → 9, 7410-12 → 10, 7410-6 → 11; **f = 7410-8**
NOR only	7402: W → 2, 3 → 1 (W'); Y → 5, 6 → 4 (Y'); Z → 8, 9 → 10 (Z'); W → 11, Z → 12 → **13** (W + Z)'. 7427: 7402-1 → 1, 7402-10 → 2, **GND** → **13** → **12** (W' + Z')'; 7402-1 → 3, X → 4, 7402-4 → 5 → **6**; 7402-13 → 9, 7427-12 → 10, 7427-6 → 11; **f = 7427-8**



7410 (3-input NAND); the 7411 (AND) and 7427 (NOR) share this layout.

Truth table to verify

WXYZ	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
f	0	1	0	1	0	1	0	1	1	0	0	0	1	0	1	0

TRAP: THE SPARE NOR INPUT

The 7427 has only 3-input NORs, and the term (W' + Z')' needs 2 inputs. Tie the third input to **GND**: 0 does not change an OR. Left floating, it reads as 1 and the gate's output sticks at 0.

WRITE IN YOUR RECORD

Result: f = Σm(1, 3, 5, 7, 8, 12, 14) was minimised to f = W'Z + WY'Z' + WXZ' (SOP) and f = (W + Z)(W' + Z')(W' + X + Y') (POS). It was realised with basic gates, with NAND gates only and with NOR gates only, and all three circuits matched the truth table.

Theory for the record: a Boolean function is minimised with a K-map by grouping adjacent 1s (giving SOP) or 0s (giving POS) in powers of two. A two-level SOP (AND–OR) converts directly to NAND–NAND, and a POS (OR–AND) to NOR–NOR, by De Morgan's theorem. Inverters are made from NAND/NOR gates with their inputs tied. Universal-gate realisations need only one IC type.

CHECK 1

(a) Prove WY'Z' + WXZ' = WZ'(Y' + X) by K-map or algebra, and say why the pack keeps it as two terms. (b) In the NAND circuit, what is the output of the 7400 gate on pins 12–13 when W = 0, Z = 1? (c) Count the ICs in each version.

Answers

Opening question. NAND-only: **SOP** (AND–OR → NAND–NAND). NOR-only: **POS** (OR–AND → NOR–NOR).

Check 1. (a) Factor out WZ'. Keeping two terms gives a pure two-level circuit, which converts gate-for-gate to NAND–NAND. The factored form is three levels. (b) Inputs are W' = 1 and Z = 1, so NAND = **0**. The final NAND then outputs f = 1 ✓ (m1, m3, … all have W = 0, Z = 1). (c) Basic: 4 ICs (7404, 7408, 7411, 7432). NAND: 2 (7400, 7410). NOR: 2 (7402, 7427).

Exp 4 · Code converters: binary → Gray, BCD → Excess-3

Manual pp. 11–12 · ICs 7486, 7432, 7408, 7404 · both converters checked over every valid input

MANUAL EXP 4, AIM

Design and implement (A) a 4-bit binary to Gray code converter, (B) a BCD to Excess-3 converter. Viva form: "Why is Gray code used in position encoders, and why is Excess-3 self-complementing?"

GUESS FIRST

How many bits change between consecutive Gray codes? What is 9's complement of 2 in Excess-3?

A · Binary → Gray

$$G_3 = B_3, \qquad G_2 = B_3 \oplus B_2, \qquad G_1 = B_2 \oplus B_1, \qquad G_0 = B_1 \oplus B_0$$

Wiring (manual's pins):

- B3 → output G3 directly.
- 7486 gate 1: B3 → 1, B2 → 2 → **3 = G2**.
- 7486 gate 2: B2 → 4, B1 → 5 → **6 = G1**.
- 7486 gate 3: B1 → 9, B0 → 10 → **8 = G0**.

B	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
G	0000	0001	0011	0010	0110	0111	0101	0100	1100	1101	1111	1110	1010	1011	1001	1000

Gray → binary (exercise): B3 = G3, B2 = B3 ⊕ G2, B1 = B2 ⊕ G1, B0 = B1 ⊕ G0. It is a chain: each XOR uses the *previous output*, not the previous input.

B · BCD → Excess-3 (XS-3 = BCD + 3)

BCD ABCD	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001
XS-3 WXYZ	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100

The K-maps use 1010–1111 as don't-cares (they are not BCD). The manual's expressions, and a factored form that saves gates:

$$W = A + BC + BD = A + B(C + D) \qquad X = B'C + B'D + BC'D' = B \oplus (C + D)$$

$$Y = CD + C'D' = (C \oplus D)' \qquad Z = D'$$

Wiring (factored form):

Gate	Wires
7432 gate 1	C → 1, D → 2 → 3 (C + D)
7408 gate 1	B → 1, 7432-3 → 2 → 3 (B(C + D))
7432 gate 2	A → 4, 7408-3 → 5 → **6 = W**
7486 gate 1	B → 1, 7432-3 → 2 → **3 = X**
7486 gate 2	C → 4, D → 5 → 6; then 7404: 7486-6 → 1 → **2 = Y**
7404	D → 3 → **4 = Z**

TRAP: INPUTS ABOVE 1001

Set 1010–1111 and the outputs look like nonsense. Those inputs aren't BCD, and the design treated them as don't-cares. Test only 0000–1001.

WRITE IN YOUR RECORD

Result: the binary-to-Gray converter (G3 = B3, Gi = Bi+1 ⊕ Bi) and the BCD-to-Excess-3 converter (W = A + BC + BD, X = B'C + B'D + BC'D', Y = CD + C'D', Z = D') were implemented and their truth tables verified.

Theory for the record: a code converter changes one binary code into another. In Gray code, successive values differ in one bit only. That avoids false intermediate readings in shaft encoders, and it gives the order of K-map labels. Binary → Gray: $G_n = B_n$, $G_i = B_{i+1} \oplus B_i$. Excess-3 is BCD + 3. It is self-complementing: inverting all bits gives the 9's complement, which simplifies decimal subtraction. Each output bit is found from a K-map, with the six unused BCD codes as don't-cares.

CHECK 1

(a) Convert Gray 1011 to binary. (b) Show XS-3 is self-complementing using 2 and 7. (c) Why can X = B'C + B'D + BC'D' be written B ⊕ (C + D)?

Answers

Opening question. One bit changes between consecutive Gray codes, so a sensor caught mid-transition can be off by at most one step. 2 in XS-3 is 0101; inverting gives 1010 = XS-3 of 7 = 9 − 2. So the 9's complement is just the bitwise inverse.

Check 1. (a) B3 = 1, B2 = 1 ⊕ 0 = 1, B1 = 1 ⊕ 1 = 0, B0 = 0 ⊕ 1 = 1, giving **1101**. (b) XS-3(2) = 0101 and XS-3(7) = 1010: bitwise inverses, and 2 + 7 = 9. (c) B'C + B'D = B'(C + D), and BC'D' = B(C + D)'. B'P + BP' with P = C + D is exactly B ⊕ P.

Exp 5 · Magnitude comparator (1-bit and 7485) and odd-parity generator/checker (gates and 74180)

Manual pp. 13–18 · ICs 7404, 7408, 7486, 7485, 74180

3 NAND/NOR design → 4 Code converters → **5 Comparator & parity** → 6–8 MUX, DEMUX, decoder

MANUAL EXP 5, AIM

Design and study 1-bit and 4-bit (7485) magnitude comparators, and a 3-bit odd-parity generator/checker, then the 74180. Viva form: "A lone 7485 lights both A>B and A<B when the inputs are equal. What did you forget?"

GUESS FIRST

The 7485 has three *cascade inputs*. What must they be set to when only one chip is used?

1-bit comparator

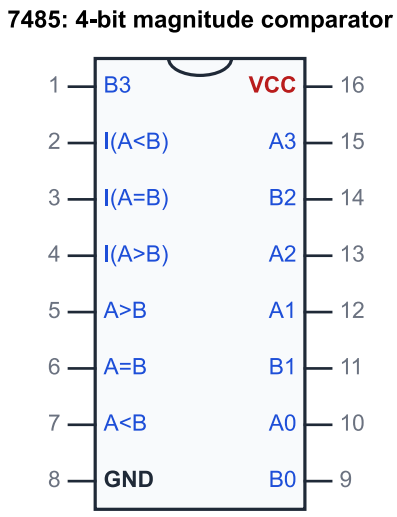
$$(A > B) = AB' \qquad (A < B) = A'B \qquad (A = B) = (A \oplus B)'$$

Wiring:

- 7404: A → 1 → 2 (A'); B → 3 → 4 (B').
- 7408: A → 1, 7404-4 → 2 → **3 = (A > B)**; 7404-2 → 4, B → 5 → **6 = (A < B)**.
- 7486: A → 1, B → 2 → 3; 7404: 7486-3 → 5 → **6 = (A = B)**.

Only one output is HIGH at a time.

4-bit comparator, 7485



7485. The cascade inputs I(A>B), I(A=B), I(A

Signal	A3 A2 A1 A0	B3 B2 B1 B0	I(A>B)	I(A=B)	I(A<B)	A>B, A=B, A<B out
pin	15, 13, 12, 10	1, 14, 11, 9	4 → **GND**	3 → **+5 V**	2 → **GND**	5, 6, 7

The chip compares from the MSB down. The first unequal bit decides the result. Only when all four bit pairs are equal does it pass the **cascade inputs** through. So a single chip needs I(A=B) = 1 (and the others 0) for "equal" to come out right.

8-bit (exercise):

- The **low** nibble's chip has cascade inputs 0, 1, 0.
- Its three outputs (pins 5, 6, 7) go into the **high** nibble chip's cascade inputs (pins 4, 3, 2).
- The final answer comes from the high chip.

Odd parity generator and checker

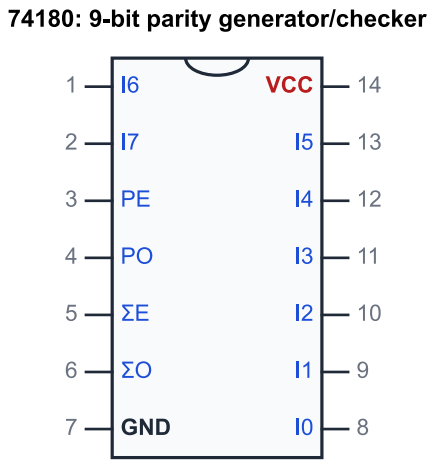
Odd parity makes the total number of 1s (data + parity) **odd**.

- **Generator:** P = (X ⊕ Y ⊕ Z)'. Two XORs (7486) and an inverter (7404). The manual's Fig 5.4 feeds Y, Z into gate 1 and X with gate 1's output into gate 2.
- **Checker:** C = (X ⊕ Y ⊕ Z ⊕ P)'. Three XORs and an inverter. **C = 1 flags an error** (an even count arrived).

Manual Fig 5.5 puts inverters after the first two XORs. Those two inversions cancel inside the last XOR, so the result is the same.

XYZ	000	001	010	011	100	101	110	111
P (odd)	1	0	0	1	0	1	1	0

74180: 9-bit parity generator/checker



74180. Data I0–I7; enables PE (pin 3) and PO (pin 4);
outputs ΣE (pin 5) and ΣO (pin 6).

Count of 1s in I0–I7	PE	PO	ΣE	ΣO
even	1	0	1	0
odd	1	0	0	1
even	0	1	0	1
odd	0	1	1	0
any	1	1	0	0
any	0	0	1	1

The manual's observation table, filled (checked):

I7...I0	PE PO	ΣE ΣO
1111 0000	1 0	**1 0**
1111 0001	1 0	**0 1**
1010 1100	0 1	**0 1**
1010 1101	0 1	**1 0**
0000 0000	1 0	**1 0**
1111 1111	0 1	**0 1**

WRITE IN YOUR RECORD

Result: the 1-bit comparator and the 7485 4-bit comparator gave exactly one HIGH output per input pair. The 3-bit odd-parity generator/checker was verified (checker output 1 = error). The 74180 outputs matched the observation table.

Theory for the record: a magnitude comparator decides A > B, A = B or A < B. For one bit: AB', A'B and (A ⊕ B)'. The 7485 compares two 4-bit words from the MSB down. Its cascade inputs let chips be chained for longer words, and must be set to (0, 1, 0) for a single chip. A parity bit makes the number of 1s even or odd so that single-bit errors can be detected. Odd-parity generator: P = (X ⊕ Y ⊕ Z)'. The checker XORs all received bits. The 74180 generates/checks parity on 8 bits, with PE/PO selecting even/odd.

CHECK 1

(a) A = 1010, B = 1010 on a lone 7485 with I(A=B) left floating. What do you expect, and why is "floating" risky here? (b) Odd parity, data 101: give P, and the checker output if the received word is 1011. (c) How do you make a 9-bit odd-parity checker from one 74180 and an inverter?

Answers

Opening question. The cascade inputs: I(A=B) (pin 3) must be **HIGH**, and I(A>B), I(A<B) (pins 4, 2) **LOW**.

Check 1. (a) A floating TTL input usually acts HIGH, so it may happen to work. That relies on undefined behaviour, so tie pin 3 to +5 V. The datasheet's function table shows what wrong cascade settings do when A = B: all three at 0 gives A>B = A**B) = I(A<B) = 1 with I(A=B) = 0 gives all three outputs 0. (b) P = (1 ⊕ 0 ⊕ 1)' = 1. The word 1011 has three 1s (odd) = valid, so the checker output is 0 (no error). (c) Feed 8 bits to I0–I7. Feed the 9th bit to PO, and its inverse (via the inverter) to PE. Then ΣO/ΣE give the odd-parity check of all 9 bits.**

Exps 6–8 · Multiplexers, demultiplexers, decoders

Manual pp. 19–24 and 2–3 (second numbering) · ICs 7410, 7420, 7411, 7404, 7408, 74151, 74153, 74157, 74138 · MUX data inputs checked over all 16 minterms

5 Comparator & parity → **6 MUX** → 7 DEMUX → 8 Decoder → 9–10 Flip-flops

MANUAL EXP 6 (II)

Implement f = Σm(0, 3, 5, 7, 8, 10, 14) using (i) an 8:1 MUX, (ii) a 4:1 MUX and additional gates. Viva form: "With 4 variables and an 8:1 MUX, what goes on the data inputs?"

GUESS FIRST

Three variables go to the select lines. What can each data input be, in terms of the fourth variable?

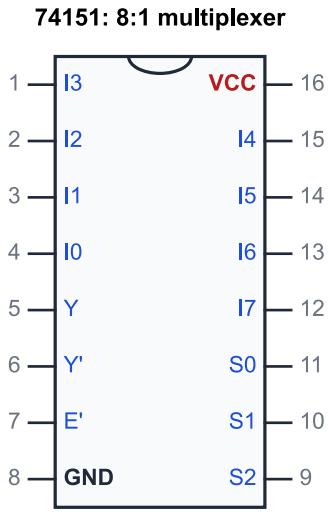
Exp 6 · Multiplexer

4:1 MUX: Y = S1'S0'D0 + S1'S0D1 + S1S0'D2 + S1S0D3.

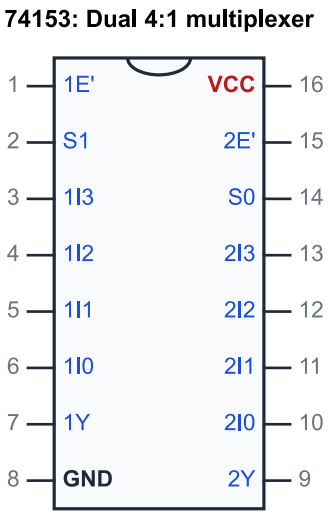
NAND-only version (Fig 6.2):

- Two 3-input NANDs with tied inputs make S1' and S0'.
- Four 3-input NANDs (D_i, select terms) feed one 4-input NAND (7420).
- That is two 7410s and one 7420.

Gate	Wires
7410 #1	S1 → 1, 2, 13 → **12 = S1'**; S0 → 3, 4, 5 → **6 = S0'**; D0 → 9, S1' → 10, S0' → 11 → 8
7410 #2	D1 → 1, S1' → 2, S0 → 13 → 12; D2 → 3, S1 → 4, S0' → 5 → 6; D3 → 9, S1 → 10, S0 → 11 → 8
7420	#1-8 → 1, #2-12 → 2, #2-6 → 4, #2-8 → 5 → **6 = Y** (pin 3 is NC)



74151 (8:1). E' (pin 7) must be LOW to enable. Y (pin 5), Y' (pin 6).



74153 (dual 4:1). S0 = pin 14, S1 = pin 2 are shared; each half has its own E'.

f = Σm(0,3,5,7,8,10,14) on the 74151 (select = A B C, data in terms of D):

Select ABC	000	001	010	011	100	101	110	111
minterms	0, 1	2, 3	4, 5	6, 7	8, 9	10, 11	12, 13	14, 15
f values	1, 0	0, 1	0, 1	0, 1	1, 0	1, 0	0, 0	1, 0
data input	**I0 = D'**	**I1 = D**	**I2 = D**	**I3 = D**	**I4 = D'**	**I5 = D'**	**I6 = 0**	**I7 = D'**
74151 pin	4	3	2	1	15	14	13	12

Wiring:

- A → pin 9 (S2), B → 10 (S1), C → 11 (S0).
- D → pins 3, 2, 1. D' (from 7404: D → 1 → 2) → pins 4, 15, 14, 12.
- Pin 13 → GND. E' (7) → GND.
- **f = pin 5.**

On the 74153 (select = A B, data in terms of C, D): I0 = (C ⊕ D)', I1 = D, I2 = D', I3 = CD'.

- S1 (pin 2) ← A, S0 (14) ← B.
- 1I0 (6) ← 7486(C, D) through a 7404 inverter.
- 1I1 (5) ← D; 1I2 (4) ← D'; 1I3 (3) ← 7408(C, D').
- 1E' (1) → GND. **f = 1Y (pin 7).**



74157 (quad 2:1). One select S (pin 1), common enable E' (pin 15). Test: S = 0 → each Y = A input; S = 1 → each Y = B input.

Exp 7 · Demultiplexer

1:4 demux with basic gates (Fig 7.1): Y0 = D S1'S0', Y1 = D S1'S0, Y2 = D S1S0', Y3 = D S1S0.

- 7404: S1 → 1 → 2 (S1'); S0 → 3 → 4 (S0').
- 7411 #1: D → 1, S1' → 2, S0' → 13 → **12 = Y0**; D → 3, S1' → 4, S0 → 5 → **6 = Y1**; D → 9, S1 → 10, S0' → 11 → **8 = Y2**.
- 7411 #2: D → 1, S1 → 2, S0 → 13 → **12 = Y3**.



74138. Address A0–A2 (pins 1–3), enables E1', E2' (active LOW) and E3 (active HIGH), outputs O0'–O7' (active LOW).

1:8 demux on the 74138 (the manual's parts list wrongly says 74151/74153):

- Select → A0 (pin 1, LSB), A1 (2), A2 (3).
- **Data D → E1' (pin 4)**; E2' (5) → GND; E3 (6) → +5 V.
- When D = 0 the chip is enabled, so the selected output goes LOW = D. When D = 1 it is disabled, so all outputs are HIGH, and the selected one again equals D.
- So **the selected output follows D, and every other output stays 1.** Outputs O0'…O7' are pins 15, 14, 13, 12, 11, 10, 9, 7.

Exp 8 · Decoder

2:4 decoder with basic gates (Fig 8.1): D0 = A'B', D1 = A'B, D2 = AB', D3 = AB.

- 7404: A → 1 → 2, B → 3 → 4.
- 7408: 7404-2, 7404-4 → 1, 2 → **3 = D0**; 7404-2, B → 4, 5 → **6 = D1**; A, 7404-4 → 9, 10 → **8 = D2**; A, B → 12, 13 → **11 = D3**.

2:4 decoder with enable, NAND only (Fig 8.2): outputs are **active-LOW**: $D_i' = (E \cdot \text{minterm}_i)'$.

- 7410 #1: A → 1, 2, 13 → 12 (A'); B → 3, 4, 5 → 6 (B'); E → 9, A' → 10, B' → 11 → **8 = D0'**.
- 7410 #2: E, A', B → 1, 2, 13 → **12 = D1'**; E, A, B' → 3, 4, 5 → **6 = D2'**; E, A, B → 9, 10, 11 → **8 = D3'**.

E A B	0 x x	1 0 0	1 0 1	1 1 0	1 1 1
D0' D1' D2' D3'	1 1 1 1	**0** 1 1 1	1 **0** 1 1	1 1 **0** 1	1 1 1 **0**

TRAP: ACTIVE-LOW OUTPUTS

On the trainer, the selected output's LED goes **out** while the others stay lit, for both the NAND decoder and the 74138. That's correct, not a fault.

WRITE IN YOUR RECORD

Result: the 4:1 MUX (NAND gates) and the 74151/74153/74157 function tables were verified. f = Σm(0, 3, 5, 7, 8, 10, 14) was realised on the 74151 (I0 = D', I1 = I2 = I3 = D, I4 = I5 = I7 = D', I6 = 0) and on the 74153 with gates. The 1:4 DEMUX (gates), the 1:8 DEMUX (74138, data on E1'), and the 2:4 decoders (basic gates; NAND with enable, active-low) were verified.

Theory for the record: a multiplexer selects one of 2^n data inputs using n select lines: $Y = \Sigma\, m_i D_i$. Any function of n + 1 variables can be built on a 2^n:1 MUX by putting n variables on the selects and 0, 1, x or x' on each data input. A demultiplexer routes one input to one of 2^n outputs. A decoder activates exactly one of 2^n outputs for an n-bit input. A decoder with an enable becomes a demultiplexer when the data is applied to the enable.

CHECK 1

(a) Full adder on one 8:1 MUX (select A, B, C_{in}): list the data inputs for S and for C_{out}. (b) The 74151's E' is left floating. What do you see? (c) Why does a decoder plus an OR gate realise any function?

Answers

Opening question. Each data input is **0, 1, D or D'**. Read each pair of minterms that share the same select value (see the table).

Check 1. (a) S: I1, I2, I4, I7 = 1, others 0. C_{out}: I3, I5, I6, I7 = 1, others 0 (two 74151s, or one 74153 per output with S = C_{in}, C_{in}', C_{in}', C_{in} and C_{out} = 0, C_{in}, C_{in}, 1). (b) Floating acts HIGH, which disables the chip: Y stays 0 and Y' stays 1 whatever the inputs. (c) The decoder produces every minterm. OR-ing the minterms in Σm gives the function. (With active-low outputs, use a NAND instead of the OR.)

Exps 9–10 · Flip-flops from NAND gates; 7474 and 7473; flip-flop conversion

Manual pp. 4–8 (second numbering) · ICs 7400, 7410, 7474, 7473 · use the trainer's debounced pulser as the clock

MANUAL EXPS 9–10

Study SR, JK and T flip-flops built from NAND gates, the 7474 (positive-edge D, preset and clear) and the 7473 (negative-edge JK, clear), and conversions SR → D, D → SR. Viva form: "Your NAND JK flip-flop with J = K = 1 ends in a random state. Why, and how do the ICs avoid it?"

GUESS FIRST

The gate JK is *level*-triggered: it reacts for as long as CLK = 1. What happens if Q flips while CLK is still 1?

Exp 9 · From gates

Clocked SR (Fig 9.1), one 7400:

- G1 = NAND(S, CLK): 1, 2 → 3.
- G2 = NAND(R, CLK): 4, 5 → 6.
- G3 = NAND(G1, Q'): 3 → 9, 11 → 10 → **8 = Q**.
- G4 = NAND(G2, Q): 6 → 12, 8 → 13 → **11 = Q'**.

CLK	S R	Q_{n+1}
0	x x	Q_n (hold)
1	0 0	Q_n
1	0 1	0
1	1 0	1
1	1 1	**invalid** (Q = Q' = 1, then unpredictable when CLK falls)

JK (Fig 9.2): feed Q' back into the J gate and Q into the K gate. That makes S = R = 1 impossible, and J = K = 1 toggles instead.

- 7410: J → 1, CLK → 2, Q' → 13 → 12; K → 3, CLK → 4, Q → 5 → 6.
- 7400 latch: 7410-12 → 1, Q' → 2 → **3 = Q**; 7410-6 → 4, Q → 5 → **6 = Q'**.

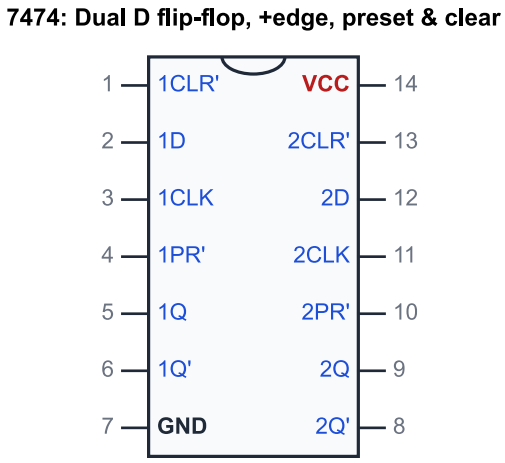
CLK pulse	J K	Q_{n+1}
↑ rising	0 0	Q_n
↑ rising	0 1	0
↑ rising	1 0	1
↑ rising	1 1	Q_n' (toggle)

RACE-AROUND

This JK is level-triggered. With J = K = 1, it toggles, the new Q feeds back, and it toggles again, as many times as fit inside one CLK-high time. The final state on the trainer is random. Edge-triggered ICs (7473, 7474) and master-slave designs act only at a clock **edge**, so each pulse gives exactly one toggle.

T flip-flop (Fig 9.3): tie J = K = T. T = 0 holds; T = 1 toggles on every clock.

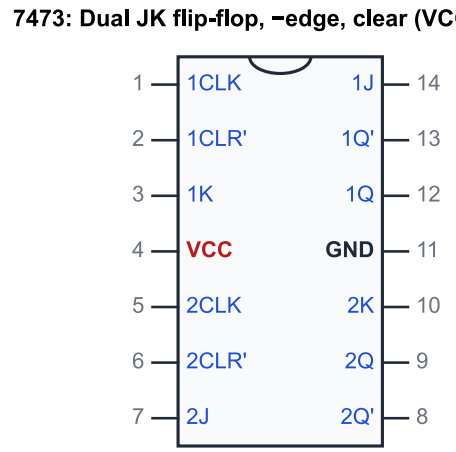
Exp 10 · Flip-flop ICs



7474: two positive-edge D flip-flops with active-LOW preset and clear.

7474 (FF 1): CLR' 1, D 2, CLK 3, PR' 4, Q 5, Q' 6.

PR'	CLR'	CLK	D	Q Q'
0	1	x	x	1 0 (preset, ignores clock)
1	0	x	x	0 1 (clear)
0	0	x	x	1 1 (not allowed)
1	1	↑ rising	1	1 0
1	1	↑ rising	0	0 1
1	1	0	x	Q_n Q_n'



7473: two negative-edge JK flip-flops with active-LOW clear. VCC = pin 4, GND = pin 11.

7473 (FF 1): CLK 1, CLR' 2, K 3, J 14, Q 12, Q' 13. **Table 10.2, filled:**

CLR'	CLK	J K	Q_{n+1}	Q_{n+1}'
0	x	x x	0	1
1	↓ falling	0 0	Q_n	Q_n'
1	↓ falling	1 0	1	0
1	↓ falling	0 1	0	1
1	↓ falling	1 1	Q_n'	Q_n (toggle)
1	1 (no edge)	x x	Q_n	Q_n'

Conversions:

Want	Using	Logic
D from SR (Fig 10.1)	S = D, R = D'	7404 inverts D for R
SR from D (Fig 10.2)	D = S + R'Q	7404 (R'), 7408 (R'·Q), 7432 (+S) into the 7474's D; Q fed back
JK from T (exercise)	T = JQ' + KQ	
T from D (exercise)	D = T ⊕ Q	one 7486
SR from JK (exercise)	J = S, K = R	S = R = 1 must still be avoided
SR from NOR (exercise)	cross-coupled NORs	active-HIGH inputs; S = R = 1 forbidden

WRITE IN YOUR RECORD

Result: the truth tables of the NAND-gate SR, JK and T flip-flops and of the 7474 (positive-edge D, PR'/CLR') and 7473 (negative-edge JK, CLR') were verified. SR → D (S = D, R = D') and D → SR (D = S + R'Q) conversions were verified.

Theory for the record: a flip-flop is a 1-bit memory element. The clocked SR flip-flop has an invalid state for S = R = 1. The JK removes it by feedback, giving toggle for J = K = 1. A level-triggered JK suffers race-around, which edge triggering or master-slave construction prevents. The D flip-flop copies D at the clock edge; the T flip-flop toggles when T = 1. Preset and clear are asynchronous, active-LOW inputs that override the clock. Conversion uses the excitation table of the available flip-flop to find its inputs in terms of the desired inputs and Q.

CHECK 1

(a) On the 7473, CLR' = 1, J = K = 1, Q = 0. Give Q after three clock pulses. (b) Why do you see no change on the 7474 while you hold the pulser down? (c) Derive D for a T flip-flop from the D excitation ($D = Q_{n+1}$).

Answers

Opening question. Race-around: with J = K = 1 and a long CLK-high, the gate JK toggles repeatedly through its own feedback, so the final state is random. ICs trigger on a clock **edge** (7474 ↑, 7473 ↓), so they change once per pulse.

Check 1. (a) It toggles each falling edge: 0 → 1 → 0 → **1**. (b) The 7474 acts only on the **rising** edge, which happens when you press. Holding the button is a steady level, so nothing more happens. (A 7473 acts on the release, the falling edge.) (c) $Q_{n+1} = Q_n$ when T = 0 and Q_n' when T = 1, so **D = T ⊕ Q**.

Exps 11–12 · Synchronous counter (7473), ring and Johnson counters, shift registers (7474)

Manual pp. 9–14 (second numbering) · ICs 7473 ×2, 7408, 7474 ×2, 7400 · every sequence below was simulated clock by clock

9–10 Flip-flops → **11 Counters** → 12 Shift registers → 13 FSM

MANUAL EXP 11, AIM

Design and test a 4-bit synchronous up counter using the 7473, and test ring and Johnson counters. Viva form: "Derive J and K for each flip-flop of the synchronous counter, and say why the ring counter needs a preset."

GUESS FIRST

In a binary count, when does bit QC toggle?

4-bit synchronous up counter

From the JK excitation table (0→0: 0x, 0→1: 1x, 1→0: x1, 1→1: x0), the manual's state table reduces to:

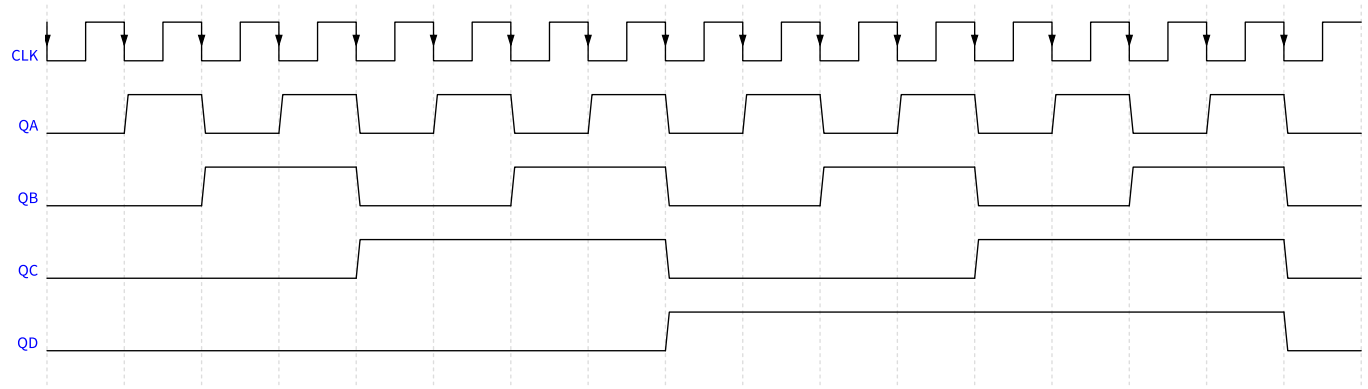
$$J_A = K_A = 1 \qquad J_B = K_B = Q_A \qquad J_C = K_C = Q_A Q_B \qquad J_D = K_D = Q_A Q_B Q_C$$

So each bit toggles exactly when all lower bits are 1. All clocks are common: that is what makes it **synchronous**.

Flip-flop	Pins (7473: FF1 = J 14, K 3, CLK 1, CLR' 2, Q 12; FF2 = J 7, K 10, CLK 5, CLR' 6, Q 9)
A = 7473 #1 FF1	J 14, K 3 → +5 V; **QA = 12**
B = 7473 #1 FF2	J 7, K 10 ← QA; **QB = 9**
C = 7473 #2 FF1	J 14, K 3 ← 7408-3 (QA → 1, QB → 2); **QC = 12**
D = 7473 #2 FF2	J 7, K 10 ← 7408-6 (7408-3 → 4, QC → 5); **QD = 9**
all four	CLK (pins 1, 5 on both ICs) ← clock; CLR' (pins 2, 6) ← clear switch; **VCC pin 4, GND pin 11**

Procedure:

1. Clear: CLR' = 0, then 1. The count shows DCBA = 0000.
2. Apply clock pulses. The 7473 changes on the **falling** edge.
3. The count goes 0000 → 0001 → … → 1111, and on the 16th pulse back to **0000**.



Expected waveforms: QA toggles every falling edge, QB every 2nd, QC every 4th, QD every 8th (divide-by-2, 4, 8, 16).

Ring and Johnson counters (two 7474s)

Name the four flip-flops Q3, Q2, Q1, Q0: IC1 FF1, IC1 FF2, IC2 FF1, IC2 FF2.

7474 pins: FF1 = CLR' 1, D 2, CLK 3, PR' 4, Q 5, Q' 6; FF2 = Q' 8, Q 9, PR' 10, CLK 11, D 12, CLR' 13.

	Ring counter	Johnson (twisted-ring) counter
shift chain	Q3 (IC1-5) → D (IC1-12); Q2 (IC1-9) → IC2-2; Q1 (IC2-5) → IC2-12	same
feedback	**Q0 (IC2-9) → D3 (IC1-2)**	**Q0' (IC2-8) → D3 (IC1-2)**
clear line goes to	**PR' of Q3 (IC1-4)** and CLR' of Q2, Q1, Q0 (IC1-13, IC2-1, IC2-13)	CLR' of all four (IC1-1, IC1-13, IC2-1, IC2-13)
tie HIGH	CLR' of Q3 (IC1-1), unused PR's	all PR's
start state	1000	0000
sequence	1000 → 0100 → 0010 → 0001 → 1000	0000 → 1000 → 1100 → 1110 → 1111 → 0111 → 0011 → 0001 → 0000
states (n FFs)	n = 4	2n = 8

TRAP: A RING COUNTER CLEARED TO 0000

The manual clears all four flip-flops. A ring counter at 0000 just circulates zeros forever. One flip-flop must be **preset** to 1, so the clear line goes to Q3's PR' instead of its CLR'. The Johnson counter starts fine from 0000.

Exp 12 · Shift registers (7474 ×2)

SISO / SIPO (Fig 12.2):

- Same chain as the ring counter, without feedback. Serial in → D3 (IC1-2); serial out = Q0 (IC2-9).
- All CLR' go to the clear line; all PR' are tied HIGH.

Enter 1, 1, 0, 1 (one bit per clock):

After clock	serial in	Q3 Q2 Q1 Q0
1	1	1 0 0 0
2	1	1 1 0 0
3	0	0 1 1 0
4	1	**1 0 1 1** (parallel out = 1011, the word entered)
5	0	0 1 0 1
6	0	0 0 1 0
7	0	0 0 0 1

Serial out (Q0) after clocks 4, 5, 6, 7 = **1, 1, 0, 1**: the first bit in comes out first.

PISO / PIPO (Fig 12.3): parallel load through the **preset** inputs.

- Each PR' is driven by NAND(PE, data bit) on a 7400:
- PE → 1, D → 2 → 3 → IC1-4;
- PE → 4, C → 5 → 6 → IC1-10;
- PE → 9, B → 10 → 8 → IC2-4;
- PE → 12, A → 13 → 11 → IC2-10.
- Parallel outputs go through a 7408 (each Q AND output-enable).

Procedure:

1. Clear (CLR = 0, PE = 0), then set CLR = 1.
2. Set D C B A on the switches.
3. PE = 1 presets the 1-bits, so the register holds DCBA. Then set PE = 0.
4. **PIPO:** output enable = 1, read W X Y Z.
5. **PISO:** clock with serial in = 0. The bits leave at Q0 one per clock.

WRITE IN YOUR RECORD

Result: the 4-bit synchronous counter counted 0000–1111 and returned to 0000 on the 16th pulse. The ring counter (preset 1000) cycled through 4 states and the Johnson counter through 8. SISO/SIPO: 1011 entered serially, read in parallel after 4 clocks and serially after 4–7 clocks. PISO/PIPO: parallel data loaded through preset and read out in parallel and serially.

Theory for the record: in a synchronous counter all flip-flops share the clock, and logic on J/K decides which toggle: each bit toggles when all lower bits are 1 (J = K = product of the lower Q's). This avoids the cumulative delay of ripple counters. A ring counter circulates a single 1 (n states, must be preset). A Johnson counter feeds back Q' (2n states). Shift registers move data one position per clock: SISO, SIPO, PISO and PIPO, according to how data enters and leaves.

CHECK 1

(a) Why is $J_D = K_D = Q_A Q_B Q_C$? (b) How many flip-flops does a mod-10 Johnson counter need? (c) After loading 1011 into the SIPO register, how many clocks until the register reads 0000?

Answers

Opening question. QC toggles whenever QA = QB = 1, i.e. after every 4th count. So $J_C = K_C = Q_A Q_B$. The ring counter needs a preset because a cleared ring (0000) has no 1 to circulate.

Check 1. (a) In binary counting, a bit flips exactly when every lower bit is 1 (…0111 → …1000). So D toggles only when A, B and C are all 1. (b) 2n = 10, so **5**. (c) Four more clocks with serial in = 0 (it reads 0000 after clock 8).

Exp 13 · FSM: overlapping sequence detector (1010, and 101), and an asynchronous circuit

Manual pp. 15–18 (second numbering) · ICs 7473, 7404, 7408, 7411, 7432, 7486 (and 7474 for the 101 version) · both detectors checked on 300 random input streams each

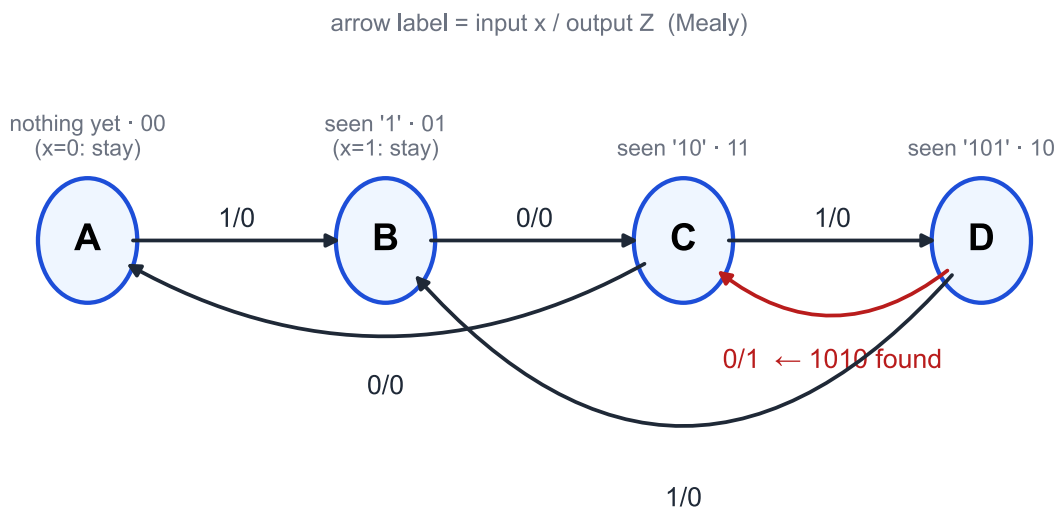
MANUAL EXP 13, AIM

Design a sequence detector that detects an overlapping sequence 1010 from an input sequence. (The list of experiments says **101**, so both are given.) Viva form: "Mealy or Moore? And what does *overlapping* change in the state diagram?"

GUESS FIRST

After 1010 is detected, which earlier state does the machine return to, and why not the start?

1010 detector (the manual's design)



Mealy state diagram. Each state remembers the longest useful suffix seen. After 1010, the last "10" is already the start of the next 1010, so the machine goes to C, not A.

State encoding (manual): A = 00, B = 01, C = 11, D = 10 (bits = flip-flop A, flip-flop B). With 7473 JK flip-flops, the K-maps give:

$J_A = BX'$ $K_A = BX' + B'X = B \oplus X$ $J_B = X + A$ $K_B = A$ $Z = AB'X'$

Signal	Wiring (7473: FF A = J 14, K 3, Q 12, Q' 13, CLK 1, CLR' 2; FF B = J 7, K 10, Q 9, Q' 8, CLK 5, CLR' 6)
X'	7404: X → 1 → 2
J_A	7408: QB (9) → 1, X' → 2 → 3 → 7473-14
K_A	7486: QB → 1, X → 2 → 3 → 7473-3
J_B	7432: X → 1, QA (12) → 2 → 3 → 7473-7
K_B	QA (12) → 7473-10
Z	7411: QA → 1, QB' (8) → 2, X' → 13 → **12 = Z**
clock, clear	pins 1, 5 ← pulser; pins 2, 6 ← clear; VCC 4, GND 11

Running it:

1. Clear to A.
2. For each bit: set X, **read Z first** (Mealy: Z depends on the present X), then pulse the clock.
3. Input 1, 0, 1, 0, 1, 0 gives Z = 0, 0, 0, **1**, 0, **1**. The second detection uses the overlap.

101 detector (the version in the experiment list)

States: S0 = nothing (00), S1 = seen "1" (01), S2 = seen "10" (10). With D flip-flops (7474), the logic shrinks to:

$$D_1 = Q_0X' \qquad D_0 = X \qquad Z = Q_1X$$

(JK form: $J_1 = Q_0X'$, $K_1 = 1$, $J_0 = X$, $K_0 = X'$.)

Wiring:

- 7474: FF1 holds Q1 (D 2, Q 5), FF2 holds Q0 (D 12, Q 9).
- X → 7474-12. 7408: Q0 (9) → 1, X' (7404) → 2 → 3 → 7474-2.
- Z: 7408: Q1 (5) → 4, X → 5 → **6**. CLK pins 3, 11; CLR' pins 1, 13 to clear; PR' 4, 10 HIGH.

Input 1, 0, 1, 0, 1 gives Z = 0, 0, **1**, 0, **1** (overlap: the final 1 of one "101" starts the next).

(ii) Fundamental-mode asynchronous circuit

Spec: Z follows X while Y = 1; Z holds while Y = 0. The manual reduces the six-state flow table to two states, giving

$$F^+ = XY + FY' \qquad Z = F$$

That is a gated D latch, with Y as the enable.

Wiring:

- 7408: X → 1, Y → 2 → 3 (XY).
- 7404: Y → 1 → 2 (Y').
- 7408: F → 4, Y' → 5 → 6 (FY').
- 7432: 7408-3 → 1, 7408-6 → 2 → **3 = F**, which is fed back to 7408-4. Z = F.

TRAP: THE LATCH CAN DROP OUT

With X = F = 1 and Y switching 1 → 0, the XY term turns off a moment before FY' turns on (the inverter delay). F can glitch to 0 and the latch loses its 1: a static-1 hazard. The cure is the consensus term **XF**: F^+ = XY + FY' + XF. It is the same function (checked), and hazard-free.

WRITE IN YOUR RECORD

Result: the overlapping 1010 Mealy detector (J_A = BX', K_A = B ⊕ X, J_B = X + A, K_B = A, Z = AB'X') produced Z = 1 at the 4th and 6th bits of 101010. The 101 detector ($D_1 = Q_0X'$, D_0 = X, $Z = Q_1X$) produced Z = 1 at the 3rd and 5th bits of 10101. The asynchronous circuit F^+ = XY + FY' transferred X to Z when Y = 1 and held it when Y = 0.

Theory for the record: a finite state machine has a finite set of states held in flip-flops. The next state depends on the present state and the inputs. In a Mealy machine the output depends on the state and the input; in a Moore machine on the state only. Design steps: state diagram → state table → state assignment → flip-flop excitation table → K-map minimisation → circuit. An overlapping detector returns, after a match, to the state for the longest suffix of the sequence that is also its prefix. A fundamental-mode asynchronous circuit has no clock: one input changes at a time and the circuit settles, and hazards must be removed with consensus terms.

CHECK 1

(a) Make the 1010 detector non-overlapping: where does the "0/1" arrow from D go? (b) Why must you read Z before pulsing the clock in a Mealy machine? (c) Moore version of 101: how many states?

Answers

Opening question. Mealy (Z = AB'X' uses the input X). After 1010 the machine goes to **C** ("10" seen), not A, because the trailing "10" can begin the next 1010. That is what *overlapping* means.

Check 1. (a) To **A** (start), so a match has to begin from scratch. (b) Z is a function of the present input. After the clock edge the state has moved on, and Z refers to the next bit. (c) **4**: S0, "1", "10", "101". The detection state is an extra state whose output is 1.

Exp 14 · TTL NAND and NOR gates (inside the chip)

In the experiment list, but the manual gives no pages for it · the circuit below is the standard 7400-type TTL NAND, simulated in LTspice (src/sim/ttl.py) with discrete transistors · theory: ECE2102 logic families

12 Shift registers → 13 FSM → **14 TTL**

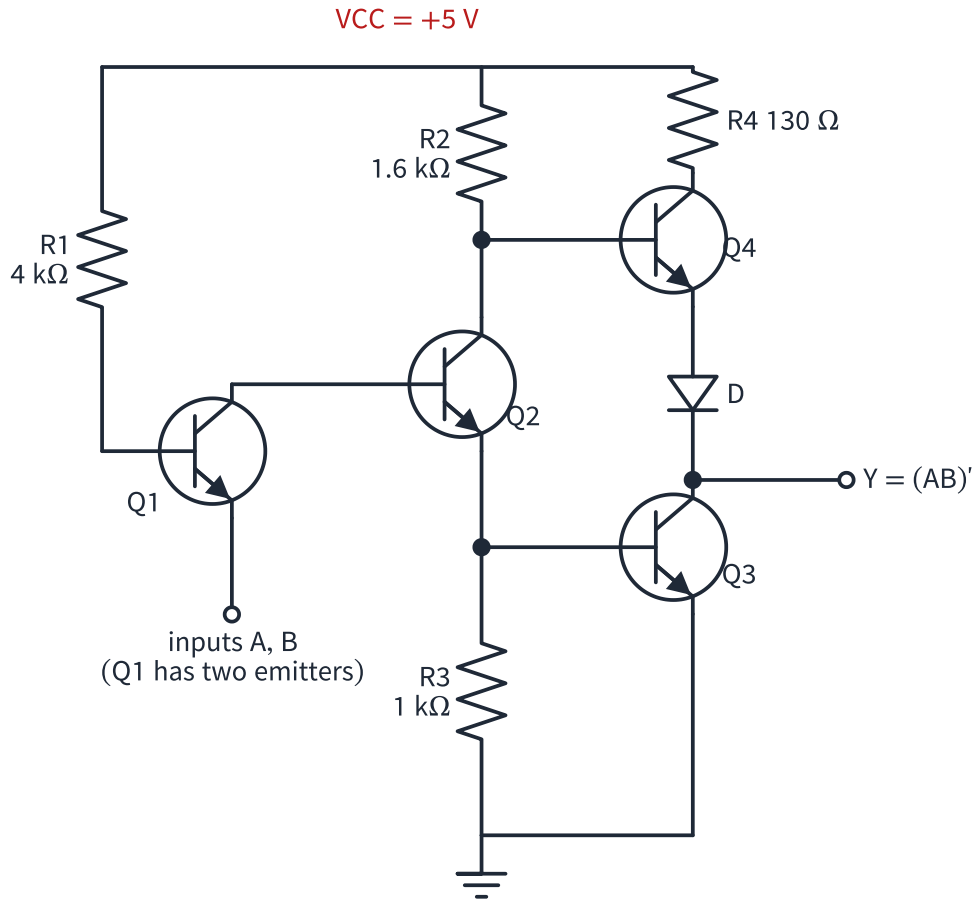
EXPERIMENT LIST, ITEM 14

Implement TTL logic for a NAND and a NOR gate. Viva form: "What does Q1's multi-emitter do, why is the output stage called a totem pole, and what is TTL's noise margin?"

GUESS FIRST

If any input is LOW, which way does Q1's base current go?

The TTL NAND



Standard TTL NAND: multi-emitter input (Q1), phase splitter (Q2), totem-pole output (Q4 on top, Q3 below). Build Q1 as two transistors with shared base and shared collector.

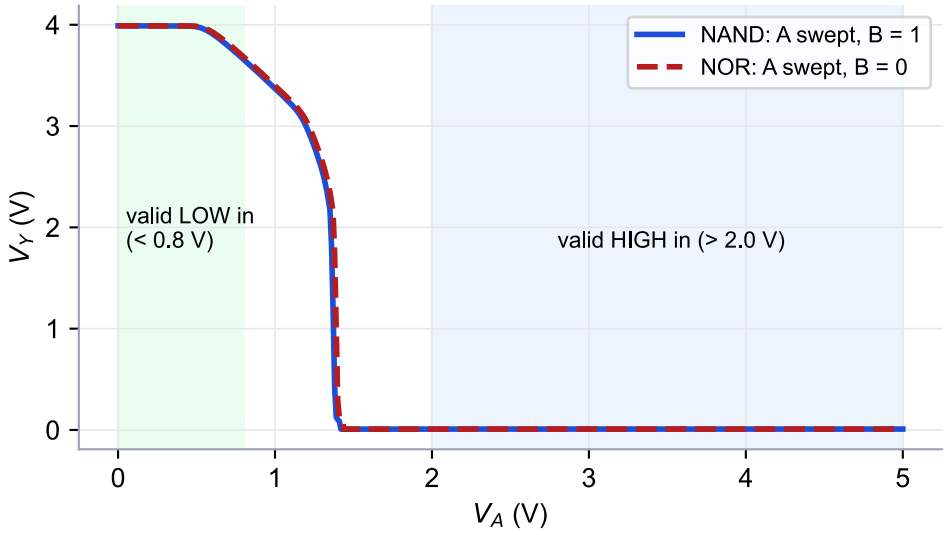
Inputs	What happens	Output
any input LOW	Q1's base current flows out through that emitter, so Q1 pulls Q2's base low. Q2 and Q3 are off; Q4 (via R2) and D conduct.	**HIGH** ≈ 5 − V_{BE4} − V_D ≈ 3.6 V when loaded (≈ 4 V lightly loaded)
all inputs HIGH	Q1's emitters are reverse-biased. R1's current flows through Q1's base–collector junction into Q2's base: Q2 and Q3 saturate. D keeps Q4 off.	**LOW** ≈ $V_{CE(sat)}$ ≈ 0.2 V

Totem pole: Q4 actively pulls the output up and Q3 actively pulls it down, stacked one above the other. That gives fast switching both ways. The 130 Ω resistor and the diode stop both transistors conducting hard at once.

TTL NOR: each input gets its own Q1 + Q2 pair. The two Q2s sit **in parallel** (shared collector and emitter), so **any** HIGH input turns Q3 on and pulls the output LOW. The totem pole is the same.

Results (LTspice, 2N2222 transistors, 1N4148 diode, 100 kΩ load)

A B	00	01	10	11
NAND V_Y	3.99 V	3.99 V	3.99 V	**0.01 V**
NOR V_Y	**3.99 V**	0.01 V	0.01 V	0.01 V



Transfer characteristic: the output switches near 1.37 V (NAND) / 1.39 V (NOR), inside TTL's undefined band of 0.8–2.0 V.

Standard TTL levels (74 series):

Parameter	Value
$V_{IL(max)}$	0.8 V
$V_{IH(min)}$	2.0 V
$V_{OL(max)}$	0.4 V
$V_{OH(min)}$	2.4 V
noise margins	NM_L = 0.8 − 0.4 = **0.4 V**; NM_H = 2.4 − 2.0 = **0.4 V**
fan-out	10
propagation delay	≈ 10 ns per gate

DO IT

LTspice (if done as simulation):

1. Draw the circuit with two NPNs for Q1 (bases tied, collectors tied, one emitter per input). Q2–Q4 = 2N2222; D = 1N4148. VCC = 5 V.
2. Inputs are DC sources of 0 or 5 V. Run `.op` for each of the four input pairs.
3. For the VTC, hold B high (NAND) or low (NOR) and run `.dc VA 0 5 0.01`.

Breadboard: the same circuit with BC547s and a 5 V supply. Inputs to 0 V / 5 V through the switches; output to an LED through about 330 Ω, or to the DMM.

WRITE IN YOUR RECORD

Result: the TTL NAND gave a HIGH output (≈ 4 V) for any LOW input and a LOW output (≈ 0 V) only when both inputs were HIGH. The TTL NOR gave a HIGH output only for 00. Both switched at ≈ 1.4 V.

Theory for the record: transistor-transistor logic uses a multi-emitter input transistor that performs the AND function. A phase-splitter transistor drives a totem-pole output: an active pull-up transistor with a diode, and an active pull-down transistor. Any LOW input diverts Q1's base current, turning off the phase splitter and the pull-down, so the output is HIGH. With all inputs HIGH, current flows through Q1's base–collector junction into the phase splitter, saturating the pull-down, so the output is LOW. A NOR uses parallel phase splitters. TTL works from 5 V, with $V_{IL} \le 0.8$ V, $V_{IH} \ge 2.0$ V, noise margin 0.4 V and fan-out 10. A floating input behaves as HIGH.

CHECK 1

(a) Why does a floating TTL input act as HIGH? (b) What would happen without the diode D? (c) Why can't two totem-pole outputs be wired together?

Answers

Opening question. A LOW input lets Q1's base current flow **out through that emitter** into the input source, which starves Q2. The output stage is a totem pole because Q4 and Q3 are stacked between VCC and ground. The noise margin is **0.4 V** each way.

Check 1. (a) With nothing pulling the emitter down, Q1's base–emitter junction has no path, just as if the input were HIGH. R1's current goes to Q2. (b) With the output LOW, Q4's base (Q2's collector at about 0.9 V) could still forward-bias Q4. Both Q4 and Q3 would conduct, wasting current and raising V_{OL}. D adds the extra voltage drop that keeps Q4 off. (c) If one output is HIGH and the other LOW, Q4 of one and Q3 of the other form a low-resistance path from VCC to ground. A large current flows and can damage them. Use open-collector gates for wired logic.

11 · Revision set: one question per idea

Answer from memory on paper, then check. A miss points back to its chapter, shown in brackets.

1. VCC and GND pins of: a 14-pin gate IC, a 16-pin MSI chip, the 7483, the 7473. *(00)*
2. What does a floating TTL input read as, and where do you tie unused AND and NOR inputs? *(00)*
3. Build OR from NANDs only, and AND from NORs only. Give the gate counts. *(Exp 1)*
4. Full adder S and C_{out}. Full subtractor D and B_o. *(Exp 2)*
5. How does one 7483 subtract, and what does $C_{out} = 0$ mean afterwards? *(Exp 2)*
6. BCD adder correction condition. *(Exp 2)*
7. Which form (SOP/POS) for NAND-only, and which for NOR-only? *(Exp 3)*
8. Binary → Gray equations; Gray → binary equations. *(Exp 4)*
9. Excess-3 of 6, and why XS-3 is self-complementing. *(Exp 4)*
10. 7485 cascade inputs for a single chip. *(Exp 5)*
11. Odd-parity generator for XYZ; what checker output 1 means. *(Exp 5)*
12. f = Σm(0,3,5,7,8,10,14) on an 8:1 MUX: the eight data inputs. *(Exp 6)*
13. 74138 as a 1:8 DEMUX: where does the data go? *(Exp 7)*
14. Why does the selected LED go dark on the NAND decoder? *(Exp 8)*
15. Race-around: cause and cure. *(Exp 9)*
16. 7474 and 7473: which edge, and which asynchronous inputs? *(Exp 10)*
17. D = ? to make a T flip-flop; S, R = ? to make a D from an SR. *(Exp 10)*
18. J and K for each stage of a 4-bit synchronous up counter. *(Exp 11)*
19. Ring vs Johnson: feedback, number of states, start state. *(Exp 11)*
20. Load 1011 serially: the register contents after each of the 4 clocks. *(Exp 12)*
21. Mealy vs Moore; what "overlapping" means for 1010. *(Exp 13)*
22. The 101 detector equations with D flip-flops. *(Exp 13)*
23. TTL NAND: the output stage's name, and why the diode is there. Noise margin. *(Exp 14)*

Answers

1. 14-pin: VCC 14, GND 7. 16-pin: 16, 8. **7483: VCC 5, GND 12. 7473: VCC 4, GND 11.**
2. HIGH. Unused AND/NAND inputs → +5 V or a used input; unused OR/NOR inputs → GND.
3. OR = NAND(NAND(A,A), NAND(B,B)), 3 gates. AND = NOR(NOR(A,A), NOR(B,B)), 3 gates.
4. $S = A \oplus B \oplus C_{in}$, $C_{out} = AB + BC_{in} + AC_{in}$. $D = A \oplus B \oplus B_i$, $B_o = A'B + (A \oplus B)'B_i$.
5. XOR each B bit with M = 1 and set $C_{in} = M = 1$, giving A + B' + 1. $C_{out} = 0$ means A < B; the result is in 2's complement.
6. $K = C_4 + S_3S_2 + S_3S_1$. When K = 1, add 0110.
7. NAND-only: SOP. NOR-only: POS.
8. $G3 = B3$, $G_i = B_{i+1} \oplus B_i$. $B3 = G3$, $B_i = B_{i+1} \oplus G_i$.
9. 6 + 3 = 9 = **1001**. Inverting XS-3(d) gives XS-3(9 − d).
10. I(A=B) = 1, I(A>B) = I(A<B) = 0.
11. $P = (X \oplus Y \oplus Z)'$. C = 1 means an even number of 1s arrived: an error.
12. Select ABC: I0 = D', I1 = D, I2 = D, I3 = D, I4 = D', I5 = D', I6 = 0, I7 = D'.
13. Into E1' (active LOW), with E2' = 0 and E3 = 1. The selected output follows D.
14. The outputs are active-LOW: the selected one goes to 0.
15. A level-triggered JK with J = K = 1 toggles repeatedly while CLK = 1. Cure: edge triggering or master-slave.
16. 7474: positive edge, PR' and CLR'. 7473: negative edge, CLR' only.
17. $D = T \oplus Q$. S = D, R = D'.
18. $J_A = K_A = 1$; $J_B = K_B = Q_A$; $J_C = K_C = Q_AQ_B$; $J_D = K_D = Q_AQ_BQ_C$.
19. Ring: Q0 → D3, n states, must be preset to 1000. Johnson: Q0' → D3, 2n states, starts from 0000.
20. 1000, 1100, 0110, 1011.
21. Mealy: output depends on state and input. Moore: output depends on state only. Overlapping: after a match, go to the state for "10", not the start.
22. $D_1 = Q_0X'$, $D_0 = X$, $Z = Q_1X$.
23. Totem pole. The diode keeps Q4 off when the output is LOW. Noise margin 0.4 V.

ECE2132 Data Structures & Algorithms Lab — Start here

Built from the 9 manual files you uploaded (Exp 1–8, 10–12; AY 2026-27). Every program in this pack was compiled with g++ in strict mode and run on the manual's own sample inputs. The output boxes are copied from those real runs.

C++ from zero → Pointers (1–2) → Arrays (3–4) → Matrices (5–6) → Sorting (7–8) → Stack (9) → Queue (10) → Linked lists (11–12) → Exam day

What the lab exam asks of you

You will be given one experiment (possibly with a small change, like the manual's ASSIGNMENT lines). To score, you need four things:

1. **Write the record part**: aim, short algorithm, time/space complexity, result.
2. **Type the program** from memory, compile it, and fix your own typos from the compiler's messages.
3. **Run it** on a sample input and show the right output.
4. **Answer viva questions**: why this line, what happens if…, what is the complexity.

Each experiment chapter gives you those four, in that order.

How to use each chapter

> **THE ONE RULE**
>
> **Guess before you read.** Each chapter opens with a question box. Answer it in your head or on paper *before* reading on, even if you're unsure. A wrong guess still helps: the right answer sticks better after you've tried. Then type the program yourself, without looking. Re-reading feels like studying but doesn't make you able to type it from scratch.

- **Code boxes** (light grey) contain the corrected program. Type it yourself; don't copy-paste.
- **Screen boxes** (black) show what the program printed in a real run, with the typed input included.
- **Red boxes** show real compiler errors from the manual's broken versions, so you learn to read them.
- **Check** questions close each chapter. Answers are at the end of that chapter.
- **"If the examiner changes the question"** sections hold the manual's ASSIGNMENT variants. Each one is tested.

Manual errors found (and fixed in this pack)

The manual's listings are images, and many experiments print the code twice (a PROGRAM listing and an IMPLEMENTATION screenshot) that **don't match**. Everything below was checked by compiling and running it.

Exp	Problem in the manual	Effect	Fix in this pack
2	RESULT paragraph copied from Exp 1	Wrong result statement	Own result written
3	No check on position or free space	Position 0 or 50 corrupts memory	Position 1…n+1 checked
4	Every option asks "Enter value:" and reads a number it never uses (listing only; the screenshot is right)	With the manual's own sample input, the program waits forever for a value	Prompt removed; deleted element printed
4	Algorithm step 5 says "insertion menu"	Text slip	—
5	Algorithm step 7 is a stray deletion line; explanation table rows are shifted by one	Text slips	Correct table in chapter
6	Fig. 3 caption "A[i][j] to T[i][j]"	Should be T[j][i]	—
7, 8	`int arr[n];` with `n` read at run time	Not standard C++. Visual Studio refuses it; g++ only allows it as an extension	`int arr[100];` + range check
8	Aim and explanation promise Bubble, Selection, Insertion, `copyArray()` and `clock()` timing, but the code has only Quick Sort	Half the experiment is missing	Full comparison program added
8	Heading "Characteristics of Insertion Sort" over quick sort points	Text slip	—
10	Figure captions say "…Operation on **Stack**"	Copied from a stack experiment	—
10	Screenshot copy: `enqueue(value);` outside its `if`, choice 3 prints "Program terminated", bare `else`	Doesn't compile	Listing version used, braces fixed
10	Array named `queue`	Clashes with `std::queue` if `<bits/stdc++.h>` is included	Renamed `queueArr`
11	Screenshot copy: traverse never moves `temp` (infinite loop), insert never links the node, menu text corrupted	Broken	Listing version used (it is correct)
12	Listing: `insert(intvalue)`, menu strings without closing quotes, `delete temp;` outside the block where `temp` exists, `main` cut off	Doesn't compile	Fixed and completed

> **EXPERIMENT 9 IS MISSING**
>
> No "Exp 9" file was uploaded. Exp 10's figure captions read "Enqueue Operation on Stack", so Exp 9 is almost certainly **Stack using arrays**. Chapter 8 of this pack covers it, written in the same style as the Queue experiment. It is **not** from your manual; check with a classmate whether Exp 9 really is Stack.

The compiler on your laptop

To practise at home you need a C++ compiler. One is now installed on this laptop: **g++ 16.2 (WinLibs MinGW)**. Chapter 1 shows how to use it. In the lab you'll use whichever editor is there (VS Code, Code::Blocks or Dev-C++). The programs in this pack run the same on all three.

1 · C++ from zero (just what the lab needs)

No C assumed. Only the pieces the 12 experiments actually use. New pieces (pointers, 2-D arrays, struct, new/delete) are taught in the experiment that first needs them.

Run a program → Program skeleton → Variables + cin/cout → if / else → Loops → Arrays → Functions → Reading errors

1.1 Running a program

A C++ program is a text file ending in **.cpp**. A **compiler** (g++) turns it into an **.exe** you can run. If the text breaks the language rules, the compiler refuses and prints **errors** with line numbers. Nothing runs until every error is fixed.

Where	How to compile + run
Code::Blocks	File → New → Empty file → save as `exp1.cpp` → **F9** (Build and run)
Dev-C++	File → New → Source File → save as `.cpp` → **F11** (Compile & Run)
VS Code	Open the folder, then in the terminal (Ctrl+`): `g++ exp1.cpp -o exp1` then `.\exp1`
This laptop	Any new terminal: `g++ -static exp1.cpp -o exp1` then `.\exp1`

SAVE AS .CPP, NOT .C

A `.c` file is compiled as C, and `cout`, `cin`, `new` and `bool &` all fail. On this laptop, always add `-static`: another program on the PATH carries an older C++ library, and without `-static` your `.exe` may exit silently.

1.2 The skeleton every experiment starts with

GUESS FIRST

Which line would you guess makes the program *start*, and what do you think `return 0;` reports, and to whom?

P01_HELLO.CPP

```cpp
#include <iostream>          // lets us use cout and cin
using namespace std;         // so we can write cout instead of std::cout

int main()                   // the program starts running here
{
    cout << "Hello, lab!" << endl;    // print, then go to a new line
    return 0;                         // 0 = "finished normally"
}
```

SCREEN (REAL RUN)

```
Hello, lab!
```

- `#include <iostream>` brings in **cout** (print) and **cin** (read). Every experiment has it.
- `using namespace std;` lets you write `cout` instead of `std::cout`. Every experiment has it.
- `int main() { ... }` is where running starts. Statements run **top to bottom**.
- Every statement ends with `;`. Blocks of statements live inside `{ }`.
- `endl` (or `"\n"` inside a string) moves to a new line.
- `return 0;` tells the operating system "finished normally".
- `//` starts a comment. The compiler ignores the rest of the line.

1.3 Variables, input, arithmetic, decisions

A **variable** is a named box that holds one value of a fixed **type**:

Type	Holds	Example
`int`	whole numbers	`int n = 5;`
`float`	numbers with decimals	`float x = 2.5;`
`char`	one character	`char c = 'A';`
`bool`	`true` / `false`	`bool done = false;`

P02_INPUT.CPP

```cpp
#include <iostream>
using namespace std;

int main()
{
    int a, b;                          // two whole-number boxes
    cout << "Enter two numbers: ";
    cin >> a >> b;                     // read two numbers typed by the user

    int sum = a + b;
    cout << "Sum = " << sum << endl;
    cout << "a / b = " << a / b << endl;      // int / int drops the fraction
    cout << "a % b = " << a % b << endl;      // % = remainder

    if (a > b)
        cout << "a is bigger" << endl;
    else if (a == b)                   // == compares, = stores
        cout << "equal" << endl;
    else
        cout << "b is bigger" << endl;

    return 0;
}
```

SCREEN (REAL RUN)

```
Enter two numbers: 17 5
Sum = 22
a / b = 3
a % b = 2
a is bigger
```

THE THREE CLASSIC BEGINNER SLIPS

1. `=` **stores**, `==` **compares**. `if (a = b)` compiles but is wrong.
2. `int / int` **throws away** the fraction: 17 / 5 = 3. `%` gives the remainder: 17 % 5 = 2.
3. `cin >> a >> b;` uses `>>` (data flows *into* a). `cout << x;` uses `<<` (data flows *out to* the screen).

`if (…) {…} else if (…) {…} else {…}`: only the first true branch runs. With a single statement the `{ }` are optional; with more than one they are **required**. Exp 10's manual screenshot breaks exactly this rule.

1.4 Loops and arrays

An **array** is a row of boxes of the same type, numbered from **0**. `int arr[5]` has `arr[0]` … `arr[4]`. There is **no** `arr[5]`; using it reads or writes someone else's memory, and C++ won't stop you.

P03_LOOPS_ARRAYS.CPP

```cpp
#include <iostream>
using namespace std;

int main()
{
    int arr[5] = {10, 20, 30, 40, 50};       // indexes 0..4

    cout << "for loop: ";
    for (int i = 0; i < 5; i++)              // start; keep going while true; step
        cout << arr[i] << " ";
    cout << endl;

    cout << "backwards with while: ";
    int i = 4;
    while (i >= 0)
    {
        cout << arr[i] << " ";
        i--;
    }
    cout << endl;

    int choice;
    do                                       // body runs at least once
    {
        cout << "Enter 0 to stop: ";
        cin >> choice;
    } while (choice != 0);

    return 0;
}
```

SCREEN (REAL RUN)

```
for loop: 10 20 30 40 50
backwards with while: 50 40 30 20 10
Enter 0 to stop: 4
Enter 0 to stop: 0
```

Loop	Use it when	In the lab
`for (int i = 0; i < n; i++)`	you know how many times	walking an array (every experiment)
`while (condition)`	you stop on a condition	insertion sort's shifting, linked-list walking
`do { } while (condition);`	body must run **at least once**	menus (Exp 10–12), circular list traversal

`i++` means "add 1 to i"; `i--` means "subtract 1".

> **CHECK 1.1**
> What does this print? `for (int i = 4; i > 0; i--) cout << i;`

1.5 Functions

A **function** is a named piece of code you can call. From Exp 5 on, every experiment splits its work into functions (`inputMatrix` , `insertionSort` , `enqueue` , ...).

P04_FUNCTIONS.CPP

```cpp
#include <iostream>
using namespace std;

int square(int x)                  // takes an int, gives back an int
{
    return x * x;
}

void printArray(int arr[], int n)  // void = gives nothing back
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

void doubleAll(int arr[], int n)   // arrays are NOT copied: changes are seen by main
{
    for (int i = 0; i < n; i++)
        arr[i] = arr[i] * 2;
}

void tryToChange(int x)            // plain int IS copied: main's variable is untouched
{
    x = 99;
    cout << "inside tryToChange: " << x << endl;
}

void reallyChange(int &x)          // & = reference: x IS main's variable
{
    x = 99;
}

int main()
{
    cout << "square(7) = " << square(7) << endl;

    int a[3] = {1, 2, 3};
```

```
    doubleAll(a, 3);
    printArray(a, 3);

    int v = 5;
    tryToChange(v);
    cout << "after tryToChange: " << v << endl;
    reallyChange(v);
    cout << "after reallyChange: " << v << endl;
    return 0;
}
```

SCREEN (REAL RUN)

```
square(7) = 49
2 4 6
inside tryToChange: 99
after tryToChange: 5
after reallyChange: 99
```

Read the result carefully. It explains three things you'll meet again:

- `int square(int x)`: the first `int` is what the function *gives back* via `return`; `void` means it gives nothing back.
- **An array parameter is not copied.** The function works on the caller's array, so `doubleAll` changed `a`. That's why `insertionSort(arr, n)` sorts `main`'s array.
- **A plain `int` parameter is a copy**, so `tryToChange` changed only its own copy. Add `&` (`int &x`) and the parameter *is* the caller's variable. Exp 8's `swapElements(int &a, int &b)` needs this.

> **ORDER RULE**
> A function must appear **above** the place that calls it. Write helper functions first and `main()` last, as every experiment in the manual does.

1.6 Reading compiler errors (exam-day survival)

The compiler reports `file:LINE:COLUMN: error: message`. **Fix the first error first**: one missing `;` can produce ten errors below it. Here is the manual's own Exp 12 listing, compiled:

G++ SAYS

```
m12_listing.cpp:62:17: error: missing terminating " character
   62 |     {   cout << "\n1. Insert
      |                 ^
m12_listing.cpp:13:6: error: variable or field 'insert' declared void
   13 | void insert(intvalue)
      |      ^~~~~
m12_listing.cpp:13:13: error: 'intvalue' was not declared in this scope
   13 | void insert(intvalue)
      |             ^~~~~~~~
m12_listing.cpp:46:15: error: unused variable 'temp' [-Werror=unused-variable]
   46 |     {   Node* temp = head;
      |               ^~~~
m12_listing.cpp:55:16: error: 'temp' was not declared in this scope; did you mean 'tm'?
   55 |     }   delete temp;
      |                ^~~~
m12_listing.cpp:62:14: error: no match for 'operator<<' (operand types are 'std::ostream' {aka
'std::basic_ostream<char>'} and 'std::ostream' {aka 'std::basic_ostream<char>'})
   62 |     {   cout << "\n1. Insert
      |         ~~~~ ^~
```

Message	Usual cause
`expected ';' before …`	missing `;` on the line **above**
`'x' was not declared in this scope`	typo in a name, or a variable used outside the `{ }` it was made in
`missing terminating " character`	a string without its closing `"`
`expected '}' at end of input`	a `{` was never closed
`'else' without a previous 'if'`	a statement squeezed between `}` and `else`
`ISO C++ forbids variable length array`	`int arr[n]` with `n` read at run time; use `int arr[100]`

Answers

Check 1.1: `4321`. It starts at 4, prints while i > 0, and stops once i becomes 0.

2 · Exp 1 + 2 — Pointers and pointer arithmetic

Manual Exp 1: "implement Pointers". Exp 2: "implement Pointer Arithmetic". Both have no user input, so they are the shortest programs in the lab.

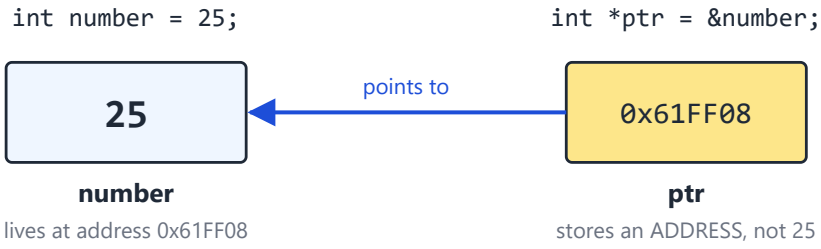
Address → Pointer variable → & and * → Pointer + 1 → Arrays are pointers → (later) linked lists

2.1 The idea

Every variable lives in memory at a numbered location: its **address**. A **pointer** is a variable whose value *is* an address.

GUESS FIRST

`int number = 25;` and `int *ptr = &number;`. If you print `ptr`, do you see 25? If you print `*ptr`?



Exp 1's variables. The address 0x61FF08 is the manual's example; yours will differ on every run.

Symbol	Read it as	In Exp 1
`int *ptr;`	"ptr is a pointer to an int"	declares the pointer
`&number`	"address of number"	`ptr = &number;`
`*ptr`	"value at the address in ptr"	prints 25

`*` MEANS TWO DIFFERENT THINGS

In a **declaration** (`int *ptr;`), `*` means "this is a pointer". In an **expression** (`cout << *ptr;`), `*` means "go to that address" (dereference).

2.2 Exp 1 program (manual's code was correct)

E01_POINTERS.CPP

```cpp
#include <iostream>
using namespace std;

int main()
{
    int number = 25;
    int *ptr;              // ptr can hold the address of an int

    ptr = &number;         // & = "address of"

    cout << "Value of number = " << number << endl;
    cout << "Address of number = " << &number << endl;
    cout << "Pointer stores = " << ptr << endl;
    cout << "Value using pointer = " << *ptr << endl;   // * = "value at"

    return 0;
}
```

SCREEN (REAL RUN)

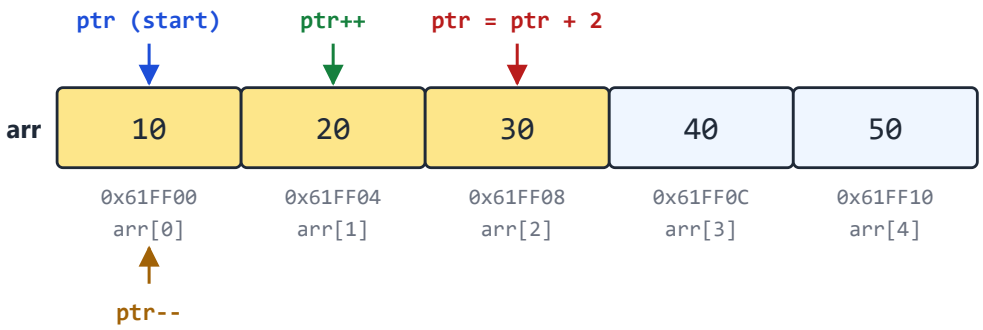
```
Value of number = 25
Address of number = 0xbb8cdff8c4
Pointer stores = 0xbb8cdff8c4
Value using pointer = 25
```

Lines 2 and 3 of the output always match each other; the actual number changes on every run and every PC. The manual says so too ("memory address varies").

2.3 Exp 2: pointer arithmetic

> **GUESS FIRST**
> `ptr` points at `arr[0]`, which is at address ...00. After `ptr++`, is the address ...01 or ...04? Why?

`ptr + 1` doesn't add 1 byte. It moves to the **next int**, which is `sizeof(int)` = 4 bytes further. That only works because an array's elements sit **side by side** in memory.



+1 on an int pointer = +4 bytes (sizeof(int)), i.e. the NEXT element

The manual's Figure 1, redrawn: each move is one whole element.

E02_POINTER_ARITHMETIC.CPP

```cpp
#include <iostream>
using namespace std;

int main()
{
    int arr[5] = {10, 20, 30, 40, 50};
    int *ptr = arr;                  // array name = address of arr[0]

    cout << "Initial Address : " << ptr << endl;
    cout << "Value : " << *ptr << endl;

    ptr++;                           // moves 1 int forward (4 bytes)
    cout << "\nAfter Increment" << endl;
    cout << "Address : " << ptr << endl;
    cout << "Value : " << *ptr << endl;

    ptr--;                           // moves 1 int back
    cout << "\nAfter Decrement" << endl;
    cout << "Address : " << ptr << endl;
    cout << "Value : " << *ptr << endl;

    ptr = ptr + 2;                   // moves 2 ints forward (8 bytes)
    cout << "\nAfter Adding 2" << endl;
    cout << "Address : " << ptr << endl;
    cout << "Value : " << *ptr << endl;

    return 0;
}
```

SCREEN (REAL RUN)

```
Initial Address : 0x45c67ffa90
Value : 10

After Increment
Address : 0x45c67ffa94
Value : 20

After Decrement
Address : 0x45c67ffa90
Value : 10

After Adding 2
Address : 0x45c67ffa98
Value : 30
```

The real run shows exactly the pattern the manual predicts: ...700 → ...704 → ...700 → ...708, values 10 → 20 → 10 → 30.

TWO FACTS THE VIVA LOVES

- `int *ptr = arr;` works because **an array's name is the address of its first element**: `arr == &arr[0]`.
- `*(ptr + i)` is the same as `arr[i]`. Square brackets are pointer arithmetic in disguise.

WRITE IN YOUR RECORD

Aim (Exp 1): To store the address of a variable in a pointer and access the variable through the pointer.

Algorithm: declare `int number = 25` → declare `int *ptr` → `ptr = &number` → print number, `&number`, `ptr`, `*ptr`.

Aim (Exp 2): To perform increment, decrement and addition on a pointer to an integer array.

Algorithm: `ptr = arr` → print → `ptr++` → print → `ptr--` → print → `ptr = ptr + 2` → print.

Complexity: time O(1), space O(1) for both.

Result (Exp 1): The pointer stored the address of `number` (same value as `&number`), and `*ptr` gave 25.

Result (Exp 2): Each increment moved the pointer by 4 bytes (one int): values 10, 20, 10, 30 at addresses base, +4, base, +8.

2.4 Viva questions

Question	Answer
What is a pointer?	A variable that stores the memory address of another variable.
What does `&` do? `*`?	`&x` gives x's address; `*p` gives the value stored at the address in p.
Why does `ptr++` add 4, not 1?	Pointer arithmetic moves in units of the pointed-to type; an int is 4 bytes here.
What would `ptr++` add for a `char *`? A `double *`?	1 byte; 8 bytes.
What is a NULL pointer?	A pointer that points nowhere (`ptr = NULL;`). Linked lists use it to mark the end.
What happens if you dereference an uninitialised pointer?	Undefined behaviour: it points to a random address, and the program may crash.
Where are pointers used?	Dynamic memory (`new`), linked lists, trees, graphs, passing by reference, arrays.

CHECK 2.1

`int a[4] = {5, 6, 7, 8}; int *p = a + 3;`. What is `*p`? What is `*(p - 2)`?

CHECK 2.2

Without looking, write the four lines of Exp 1 that declare `number`, declare `ptr`, assign it, and print the value through the pointer.

Answers

Check 2.1: `*p` = a[3] = **8**; `*(p - 2)` = a[1] = **6**.

Check 2.2: `int number = 25;` · `int *ptr;` · `ptr = &number;` · `cout << *ptr;`. If you wrote `ptr = number` (no `&`), the compiler rejects it: you can't store an int in a pointer.

3 · Exp 3 — Insert into an array (beginning, any position, end)

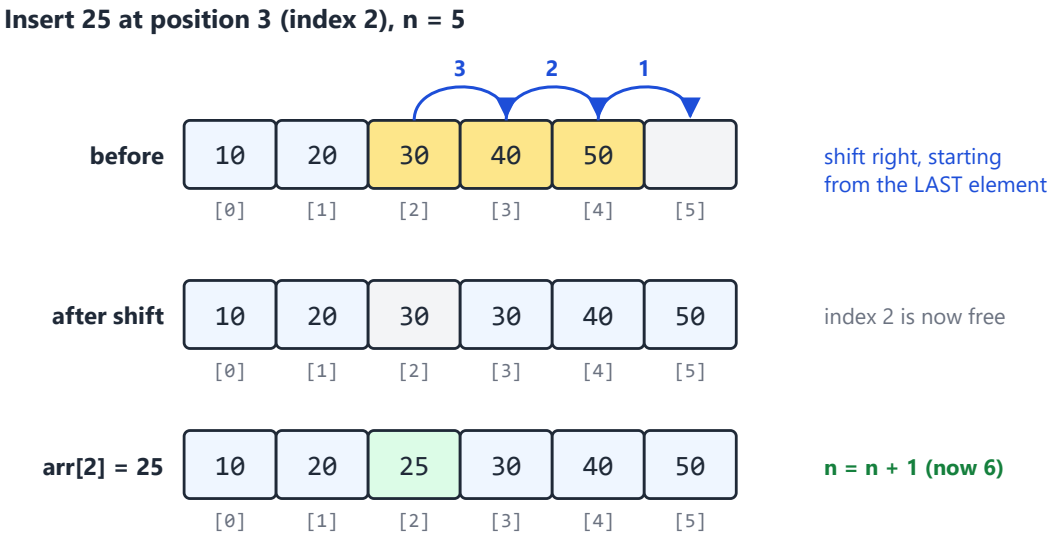
The first "data structure" experiment. The skill here, shifting elements in a loop, comes back in Exp 4 and inside insertion sort (Exp 7).

Array = fixed boxes → **Make a gap by shifting right** → Drop the value in → n = n + 1

3.1 The idea

An array's elements sit side by side and can't move by themselves. To put a value at index 2 you must first **make a gap** there by shifting everything from index 2 onward one place to the right.

> **GUESS FIRST**
> To shift `10 20 30 40 50` right from index 2, should the loop copy **30 first** or **50 first**? Try both on paper and see what goes wrong.



Insert 25 at position 3. The arrows are numbered in the order the loop does them.

If you copy 30 first, `arr[3] = arr[2]` overwrites 40 before 40 has moved: you get `10 20 30 30 30 30`. So the loop must run **from the end down**: `for (i = n; i >= pos; i--) arr[i] = arr[i-1];`.

> **POSITION VS INDEX**
> The program asks for a **position** counted from 1 (what humans say), but C++ indexes from 0. Position 3 = `arr[2]`, hence `arr[pos - 1] = value;`. Valid positions for insertion are **1 to n+1** (n+1 = "after the last one").

3.2 The program (corrected)

The manual's logic was right. This version adds the two checks it was missing: free space (`n < 100`) and a valid position.

E03_ARRAY_INSERT.CPP

```cpp
#include <iostream>
using namespace std;

int main()
{
    int arr[100], n, value, pos, choice;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter array elements:\n";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    if (n >= 100)                           // FIX: no room left for one more
    {
        cout << "Array is full. Insertion not possible.";
        return 0;
    }

    cout << "\n1. Insert at Beginning";
    cout << "\n2. Insert at Any Position";
    cout << "\n3. Insert at End";
    cout << "\nEnter your choice: ";
    cin >> choice;

    if (choice == 1)
    {
        cout << "Enter value: ";
        cin >> value;

        for (int i = n; i > 0; i--)         // shift right, last element first
            arr[i] = arr[i - 1];

        arr[0] = value;
        n++;
    }
    else if (choice == 2)
    {
        cout << "Enter Position: ";
        cin >> pos;

        if (pos < 1 || pos > n + 1)         // FIX: positions 1..n+1 are valid
        {
            cout << "Invalid position.";
            return 0;
        }

        cout << "Enter Value: ";
        cin >> value;

        for (int i = n; i >= pos; i--)      // shift right from the end down to pos
            arr[i] = arr[i - 1];

        arr[pos - 1] = value;               // position 3 = index 2
        n++;
    }
    else if (choice == 3)
    {
```

```
        cout << "Enter Value: ";
        cin >> value;

        arr[n] = value;                    // first free slot is index n
        n++;
    }
    else
    {
        cout << "Invalid choice.";
        return 0;
    }

    cout << "\nUpdated Array:\n";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}
```

The manual's three sample inputs, run for real:

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Insert at Beginning
2. Insert at Any Position
3. Insert at End
Enter your choice: 1
Enter value: 5

Updated Array:
5 10 20 30 40 50
```

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Insert at Beginning
2. Insert at Any Position
3. Insert at End
Enter your choice: 2
Enter Position: 3
Enter Value: 25

Updated Array:
10 20 25 30 40 50
```

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Insert at Beginning
2. Insert at Any Position
3. Insert at End
```

```
Enter your choice: 3
Enter Value: 60

Updated Array:
10 20 30 40 50 60
```

The added checks working (position 7 in a 3-element array):

```
SCREEN (REAL RUN)
Enter number of elements: 3
Enter array elements:
1 2 3

1. Insert at Beginning
2. Insert at Any Position
3. Insert at End
Enter your choice: 2
Enter Position: 7
Invalid position.
```

Case	Loop	Why
Beginning	`for (i = n; i > 0; i--) arr[i] = arr[i-1];` then `arr[0] = value;`	every element moves
Position	`for (i = n; i >= pos; i--) arr[i] = arr[i-1];` then `arr[pos-1] = value;`	elements from index pos−1 onward move
End	`arr[n] = value;`	first free slot is index n; nothing moves

All three end with `n++`: the array now holds one more element.

WRITE IN YOUR RECORD

Aim: To insert an element into an array at the beginning, at a given position, and at the end.

Algorithm: read n and the elements → read choice → *beginning:* shift all right (from the end), `arr[0] = value` → *position:* check 1 ≤ pos ≤ n+1, shift right from the end down to index pos−1, `arr[pos-1] = value` → *end:* `arr[n] = value` → `n++` → print.

Complexity: beginning O(n), position O(n) (worst case), end O(1). Space O(1) extra.

Result: 10 20 30 40 50 became 5 10 20 30 40 50 (beginning), 10 20 25 30 40 50 (25 at position 3), 10 20 30 40 50 60 (end).

MISTAKES THAT COST MARKS

- Forgetting `n++`: the new last element never prints.
- Shifting left-to-right: duplicates the value (see the guess above).
- `arr[pos] = value` instead of `arr[pos-1]`: everything lands one place too far right.

3.3 Viva questions

Question	Answer
Why is insertion at the beginning O(n)?	All n elements must shift one place right.
Why is insertion at the end O(1)?	No shifting: write at index n (if there is space).
What is the drawback of arrays for insertion?	Fixed size, and shifting costs O(n). A linked list inserts by changing pointers.
Why `int arr[100]` and not `int arr[n]`?	Array sizes must be constants in standard C++; 100 is the maximum capacity, n is how many are in use.
What if n is already 100?	Overflow: there's no room. The program must refuse (our first check).

3.4 If the examiner changes the question

The manual's ASSIGNMENT list. Each function is tested (10 checks, all pass) and is written to drop into the program above. Each returns the new n.

Insert at a position (reused by the next two):

```
int insertAt(int arr[], int n, int pos, int value)   // pos is 1-based
{
    if (n >= CAP || pos < 1 || pos > n + 1)
        return n;                                    // no space / bad position
    for (int i = n; i >= pos; i--)
        arr[i] = arr[i - 1];
    arr[pos - 1] = value;
    return n + 1;
}
```

Insert multiple elements:

```
// Insert k values (vals[0..k-1]) at the end, stopping if the array fills up
int insertMultiple(int arr[], int n, int vals[], int k)
{
    for (int i = 0; i < k; i++)
    {
        if (n >= CAP)
        {
            cout << "Array full, " << k - i << " value(s) not inserted" << endl;
            break;
        }
        arr[n] = vals[i];
        n++;
    }
    return n;
}
```

Insert into a sorted array, keeping it sorted. This is the core move of insertion sort (Exp 7):

```
// arr is sorted ascending; keep it sorted
int insertSorted(int arr[], int n, int value)
{
    if (n >= CAP)
        return n;
    int i = n - 1;
    while (i >= 0 && arr[i] > value)     // shift bigger ones right (like insertion sort)
    {
        arr[i + 1] = arr[i];
        i--;
    }
    arr[i + 1] = value;
    return n + 1;
}
```

Insert only if space is available:

```
int insertIfSpace(int arr[], int n, int value)
{
    if (n >= CAP)
    {
        cout << "No space available. Insertion not possible." << endl;
        return n;
    }
    arr[n] = value;                      // space exists: put it at the end
    return n + 1;
}
```

Insert before / after a given element: search for the element, then reuse `insertAt`.

```
// Insert value just before the first occurrence of target
int insertBefore(int arr[], int n, int target, int value)
{
    for (int i = 0; i < n; i++)
        if (arr[i] == target)
            return insertAt(arr, n, i + 1, value);   // index i = position i+1
    cout << target << " not found" << endl;
    return n;
}
```

```
int insertAfter(int arr[], int n, int target, int value)
{
    for (int i = 0; i < n; i++)
        if (arr[i] == target)
            return insertAt(arr, n, i + 2, value);   // one place further right
    cout << target << " not found" << endl;
    return n;
}
```

In `main` you call these like `n = insertSorted(arr, n, value);`.

> **CHECK 3.1**
> `arr` = `4 8 15`, n = 3. Trace the "any position" branch for pos = 2, value = 6: which assignments run, in order, and what is the final array?

CHECK 3.2

Is `pos = 4` valid for this n = 3 array? Is `pos = 5` ?

Answers

Check 3.1: i = 3: `arr[3] = arr[2]` (15); i = 2: `arr[2] = arr[1]` (8); the loop stops (i = 1 < pos). Then `arr[1] = 6`, n = 4 → **4 6 8 15**.

Check 3.2: pos = 4 = n+1 is valid (insert at the end). pos = 5 is invalid: it would leave a gap at index 3.

4 · Exp 4 — Delete from an array (beginning, any position, end)

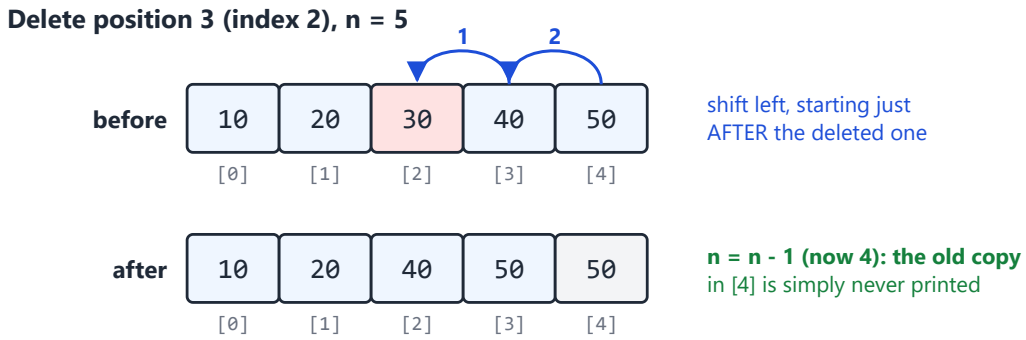
The mirror image of Exp 3: close the gap by shifting **left**, then n = n − 1.

Pick the element → **Shift the rest left** → n = n − 1 → print 0..n−1

4.1 The idea

GUESS FIRST

In Exp 3 the shift loop had to run from the **end** backwards. For deletion (shift left), which end must the loop start from, and why?



Delete position 3: shift from just after the gap, moving forwards.

When shifting **left**, start right after the deleted element and move **forward**: `arr[i] = arr[i+1]` for i = pos−1 … n−2. Going the other way would copy 50 over 40 before 40 has moved. **Rule of thumb: always copy the element that is about to be overwritten first.**

The last element is never erased; after `n--` the program simply never prints index n. That's why delete-at-end is just `n--`: O(1), no shifting.

4.2 The manual's bug

The PROGRAM listing (manual page 9) asks **"Enter value:"** in all three branches and reads a number it never uses. Deleting needs only a *position*. With the manual's own Sample Input 1 (`5`, `10 20 30 40 50`, `1`) that listing stops at "Enter value:" and waits for input the sample never gives. The IMPLEMENTATION screenshot (page 11) has no such line. This pack uses the screenshot's logic, plus a position check, and prints which element was deleted.

4.3 The program (corrected)

E04_ARRAY_DELETE.CPP

```
#include <iostream>
using namespace std;

int main()
```

```
{
    int arr[100], n, pos, choice;           // FIX: no "value" - deletion needs no value

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter array elements:\n";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    if (n <= 0)                             // FIX: nothing to delete
    {
        cout << "Array is empty. Deletion not possible.";
        return 0;
    }

    cout << "\n1. Delete at Beginning";
    cout << "\n2. Delete at Any Position";
    cout << "\n3. Delete at End";
    cout << "\nEnter your choice: ";
    cin >> choice;

    if (choice == 1)
    {
        cout << "Deleted element: " << arr[0];
        for (int i = 0; i < n - 1; i++)     // shift left, first element first
            arr[i] = arr[i + 1];
        n--;
    }
    else if (choice == 2)
    {
        cout << "Enter Position: ";
        cin >> pos;

        if (pos < 1 || pos > n)             // FIX: positions 1..n are valid
        {
            cout << "Invalid position.";
            return 0;
        }

        cout << "Deleted element: " << arr[pos - 1];
        for (int i = pos - 1; i < n - 1; i++)
            arr[i] = arr[i + 1];
        n--;
    }
    else if (choice == 3)
    {
        cout << "Deleted element: " << arr[n - 1];
        n--;                                // just forget the last one
    }
    else
    {
        cout << "Invalid choice.";
        return 0;
    }

    cout << "\nUpdated Array:\n";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
```

```
    return 0;
}
```

The manual's three sample inputs:

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Delete at Beginning
2. Delete at Any Position
3. Delete at End
Enter your choice: 1
Deleted element: 10
Updated Array:
20 30 40 50
```

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Delete at Beginning
2. Delete at Any Position
3. Delete at End
Enter your choice: 2
Enter Position: 3
Deleted element: 30
Updated Array:
10 20 40 50
```

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Delete at Beginning
2. Delete at Any Position
3. Delete at End
Enter your choice: 3
Deleted element: 50
Updated Array:
10 20 30 40
```

A bad position is refused:

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Delete at Beginning
2. Delete at Any Position
3. Delete at End
Enter your choice: 2
```

```
Enter Position: 6
Invalid position.
```

Case	Loop	Then
Beginning	`for (i = 0; i < n-1; i++) arr[i] = arr[i+1];`	`n--`
Position	`for (i = pos-1; i < n-1; i++) arr[i] = arr[i+1];`	`n--`
End	(none)	`n--`

Valid deletion positions are **1 to n** (not n+1: there's nothing there to delete). With n = 0 there's nothing to delete at all: **underflow**.

> **WRITE IN YOUR RECORD**
>
> **Aim:** To delete an element from an array at the beginning, at a given position, and at the end.
>
> **Algorithm:** read n and the elements → read choice → *beginning:* shift elements 1…n−1 one place left → *position:* check 1 ≤ pos ≤ n, shift elements after pos−1 one place left → *end:* nothing to shift → `n--` → print.
>
> **Complexity:** beginning O(n), position O(n) (worst case), end O(1). Space O(1).
>
> **Result:** 10 20 30 40 50 became 20 30 40 50 (beginning), 10 20 40 50 (position 3, deleted 30), 10 20 30 40 (end).

4.4 Viva questions

Question	Answer
Why does deletion at the end need no shifting?	Reducing n hides the last element; no gap is created.
Is the deleted value really erased from memory?	No. It stays in the slot until overwritten; the program just stops counting it.
What is underflow here?	Trying to delete from an empty array (n = 0).
Insertion shifts from the end, deletion from the front. Why?	So no element is overwritten before it has been copied.
Time complexity of deleting a value you first have to search for?	O(n): the search is O(n) and the shift is O(n), so O(n) overall.

4.5 If the examiner changes the question

Beginning, position and end are the experiment itself. The other two items:

Delete a given value (search, then shift):

```c
// Search for value; if found, delete it (first occurrence) and shift left
int deleteByValue(int arr[], int n, int value)
{
    int pos = -1;
    for (int i = 0; i < n; i++)          // linear search
```

```
        if (arr[i] == value)
        {
            pos = i;
            break;
        }

    if (pos == -1)
    {
        cout << value << " not found in array" << endl;
        return n;
    }
    for (int i = pos; i < n - 1; i++)
        arr[i] = arr[i + 1];
    return n - 1;
}
```

Menu-driven insertion and deletion on one array. A full program combining Exp 3 and Exp 4, with a repeating menu (the `do … while` pattern from Exp 10):

E04B_ARRAY_MENU.CPP

```
// Exp 4 assignment: menu-driven insertion AND deletion on one array
#include <iostream>
using namespace std;

int arr[100], n = 0;

void display()
{
    if (n == 0)
        cout << "Array is empty." << endl;
    else
    {
        cout << "Array: ";
        for (int i = 0; i < n; i++)
            cout << arr[i] << " ";
        cout << endl;
    }
}

void insertAt(int pos, int value)            // pos is 1-based
{
    if (n >= 100)
        cout << "Array is full." << endl;
    else if (pos < 1 || pos > n + 1)
        cout << "Invalid position." << endl;
    else
    {
        for (int i = n; i >= pos; i--)
            arr[i] = arr[i - 1];
        arr[pos - 1] = value;
        n++;
    }
}

void deleteAt(int pos)
{
    if (n == 0)
        cout << "Array is empty." << endl;
    else if (pos < 1 || pos > n)
```

```
        cout << "Invalid position." << endl;
    else
    {
        cout << "Deleted element: " << arr[pos - 1] << endl;
        for (int i = pos - 1; i < n - 1; i++)
            arr[i] = arr[i + 1];
        n--;
    }
}

int main()
{
    int choice, pos, value;
    do
    {
        cout << "\n1. Insert  2. Delete  3. Display  4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter position and value: ";
            cin >> pos >> value;
            insertAt(pos, value);
        }
        else if (choice == 2)
        {
            cout << "Enter position: ";
            cin >> pos;
            deleteAt(pos);
        }
        else if (choice == 3)
            display();
        else if (choice != 4)
            cout << "Invalid choice!" << endl;
    } while (choice != 4);

    return 0;
}
```

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Insert  2. Delete  3. Display  4. Exit
Enter your choice: 1
Enter position and value: 1 10
Enter your choice: 1
Enter position and value: 2 30
Enter your choice: 1
Enter position and value: 2 20
Enter your choice: 3
Array: 10 20 30
Enter your choice: 2
Enter position: 1
Deleted element: 10
Enter your choice: 3
Array: 20 30
Enter your choice: 2
Enter position: 5
Invalid position.
Enter your choice: 4
```

CHECK 4.1

`arr` = `7 3 9 1`, n = 4. Delete position 2. Write the assignments the loop performs, the final n, and the printed array.

CHECK 4.2

Someone writes the position loop as `for (i = pos-1; i < n; i++) arr[i] = arr[i+1];`. What goes wrong?

Answers

Check 4.1: i = 1: `arr[1] = arr[2]` (9); i = 2: `arr[2] = arr[3]` (1); stop at i = 3. n = 3 → **7 9 1**.

Check 4.2: On the last pass (i = n−1) it reads `arr[n]`, one past the data: junk (or past the end of the array if n = 100). The result still prints correctly, because n−1 elements are printed, but it reads memory it shouldn't. The correct bound is `i < n-1`.

5 · Exp 5 + 6 — Matrix multiplication and transpose (modular programming)

Two experiments, one pattern: three functions (input, operate, display) plus a main() that calls them. New C++ here: 2-D arrays and passing them to functions.

2-D array → Pass to a function → inputMatrix → multiply / transpose → displayMatrix

5.1 New C++: 2-D arrays

`int A[10][10];` is a grid of 10 rows × 10 columns; `A[i][j]` is row i, column j (both from 0). Like `arr[100]`, the size is the **maximum**: the user's 2 × 3 matrix uses only the top-left corner.

Walking a grid takes **two nested loops**: the outer loop picks the row, the inner loop walks along it.

```
for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
        cin >> A[i][j];
```

WHY THE PARAMETER IS `INT MATRIX[][10]`

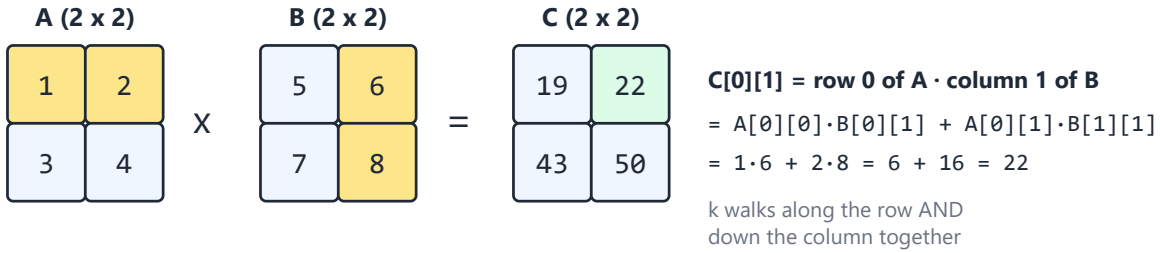
When passing a 2-D array, the **number of columns must be written** and must match the declaration (`[10]`). Only the first size may be left empty. `int matrix[][]` doesn't compile.

Modular programming means splitting the program into functions that each do one job: `inputMatrix()`, `multiplyMatrix()` / `transposeMatrix()`, `displayMatrix()`, with `main()` only reading sizes and calling them. The manual asks for exactly this structure, so use these exact function names.

5.2 Exp 5: multiplication

GUESS FIRST

A is 2 × **3** and B is **3** × 2. Can you multiply A × B? What order is the answer? Can you multiply B × A?



Rule: (m x n) times (n x p) gives (m x p). The two inner numbers must match: c1 == r2.

Each answer cell is one row of A "dotted" with one column of B.

$$C[i][j] = A[i][0]\cdot B[0][j] + A[i][1]\cdot B[1][j] + \ldots + A[i][c1-1]\cdot B[c1-1][j]$$

That sum is the **third loop** (k). Three nested loops: i over rows of A, j over columns of B, k along the shared dimension. Hence **$O(n^3)$** for n × n matrices.

E05_MATRIX_MULTIPLY.CPP

```cpp
#include <iostream>
using namespace std;

// Function to input matrix elements
void inputMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            cin >> matrix[i][j];
}

// Function to multiply two matrices: C = A x B
void multiplyMatrix(int A[][10], int B[][10], int C[][10], int r1, int c1, int c2)
{
    for (int i = 0; i < r1; i++)              // each row of A
    {
        for (int j = 0; j < c2; j++)          // each column of B
        {
            C[i][j] = 0;
            for (int k = 0; k < c1; k++)      // walk along row i and column j
                C[i][j] = C[i][j] + A[i][k] * B[k][j];
        }
    }
}

// Function to display matrix
void displayMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
            cout << matrix[i][j] << " ";
        cout << endl;
    }
}

int main()
{
    int A[10][10], B[10][10], C[10][10];
    int r1, c1, r2, c2;

    cout << "Enter rows and columns of first matrix: ";
    cin >> r1 >> c1;

    cout << "Enter rows and columns of second matrix: ";
    cin >> r2 >> c2;

    // FIX: arrays are 10 x 10, so larger sizes would overflow
    if (r1 < 1 || c1 < 1 || r2 < 1 || c2 < 1 || r1 > 10 || c1 > 10 || r2 > 10 || c2 > 10)
    {
        cout << "Sizes must be between 1 and 10.";
        return 0;
    }

    // Check multiplication condition
    if (c1 != r2)
```

```
    {
        cout << "Matrix multiplication not possible.";
        return 0;
    }

    cout << "Enter elements of first matrix:\n";
    inputMatrix(A, r1, c1);

    cout << "Enter elements of second matrix:\n";
    inputMatrix(B, r2, c2);

    multiplyMatrix(A, B, C, r1, c1, c2);

    cout << "\nResultant Matrix:\n";
    displayMatrix(C, r1, c2);

    return 0;
}
```

The manual's sample (2 × 2), and a non-square case worked by hand (row 1 of the answer: 1·7 + 2·9 + 3·11 = 58):

SCREEN (REAL RUN)

```
Enter rows and columns of first matrix: 2 2
Enter rows and columns of second matrix: 2 2
Enter elements of first matrix:
1 2
3 4
Enter elements of second matrix:
5 6
7 8

Resultant Matrix:
19 22
43 50
```

SCREEN (REAL RUN)

```
Enter rows and columns of first matrix: 2 3
Enter rows and columns of second matrix: 3 2
Enter elements of first matrix:
1 2 3
4 5 6
Enter elements of second matrix:
7 8
9 10
11 12

Resultant Matrix:
58 64
139 154
```

SCREEN (REAL RUN)

```
Enter rows and columns of first matrix: 2 3
Enter rows and columns of second matrix: 2 3
Matrix multiplication not possible.
```

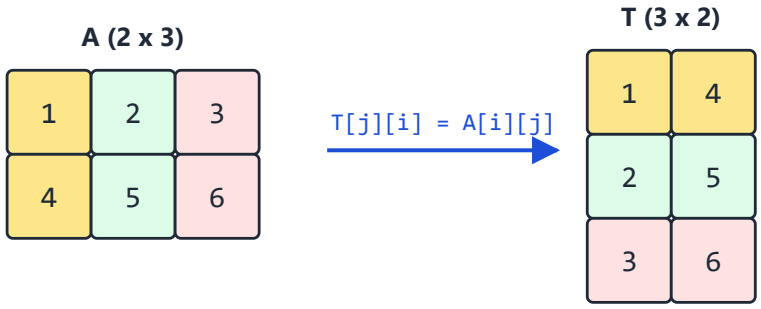
THE LINE PEOPLE FORGET

`C[i][j] = 0;` before the k-loop. Without it, C starts with garbage and the sum is wrong. It must sit **inside** the j-loop, so each cell starts from 0.

5.3 Exp 6: transpose

GUESS FIRST

A is 2 × 3. What is the order of its transpose, and where does A[0][2] go?



Column j of A becomes row j of T (each colour = one column of A → one row of T)

Rows become columns: T[j][i] = A[i][j].

E06_MATRIX_TRANSPOSE.CPP

```cpp
#include <iostream>
using namespace std;

// Function to input matrix elements
void inputMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            cin >> matrix[i][j];
}

// Function to find transpose of matrix
void transposeMatrix(int A[][10], int T[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            T[j][i] = A[i][j];              // row i becomes column i
}

// Function to display matrix
void displayMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
            cout << matrix[i][j] << " ";
        cout << endl;
    }
}
```

```
int main()
{
    int A[10][10], T[10][10];
    int rows, cols;

    cout << "Enter number of rows and columns of matrix: ";
    cin >> rows >> cols;

    if (rows < 1 || cols < 1 || rows > 10 || cols > 10)   // FIX: array is 10 x 10
    {
        cout << "Sizes must be between 1 and 10.";
        return 0;
    }

    cout << "Enter elements of matrix:\n";
    inputMatrix(A, rows, cols);

    transposeMatrix(A, T, rows, cols);

    cout << "\nOriginal Matrix:\n";
    displayMatrix(A, rows, cols);

    cout << "\nTranspose Matrix:\n";
    displayMatrix(T, cols, rows);            // rows and columns swap

    return 0;
}
```

SCREEN (REAL RUN)

```
Enter number of rows and columns of matrix: 2 3
Enter elements of matrix:
1 2 3
4 5 6

Original Matrix:
1 2 3
4 5 6

Transpose Matrix:
1 4
2 5
3 6
```

Note the call `displayMatrix(T, cols, rows)`: the transpose has the sizes **swapped**. Passing `(T, rows, cols)` prints the wrong shape.

WRITE IN YOUR RECORD — EXP 5

Aim: To multiply two matrices using separate functions for input, multiplication and display.

Algorithm: read r1, c1, r2, c2 → if c1 ≠ r2, print "not possible" and stop → `inputMatrix(A)`, `inputMatrix(B)` → `multiplyMatrix`: for each i, j set C[i][j] = 0, then for k = 0…c1−1 add A[i][k]·B[k][j] → `displayMatrix(C, r1, c2)`.

Complexity: time O(r1·c1·c2), i.e. $O(n^3)$ for square matrices; space O(r1·c2) for C.

Result: [[1,2],[3,4]] × [[5,6],[7,8]] = [[19,22],[43,50]].

WRITE IN YOUR RECORD — EXP 6

Aim: To find the transpose of a matrix using separate functions for input, transpose and display.

Algorithm: read rows, cols → `inputMatrix(A)` → `transposeMatrix`: for each i, j set T[j][i] = A[i][j] → display A (rows × cols) and T (cols × rows).

Complexity: time O(m·n); space O(m·n) for T.

Result: [[1,2,3],[4,5,6]] (2 × 3) → [[1,4],[2,5],[3,6]] (3 × 2).

5.4 Viva questions

Question	Answer
Condition for A × B?	Columns of A = rows of B (c1 == r2). Result is r1 × c2.
Is A × B = B × A?	Generally no; matrix multiplication isn't commutative (B × A may not even be defined).
Why three loops?	Two pick the answer cell (i, j); the third (k) adds up the row·column products.
Why `int matrix[][10]`?	The compiler needs the column count to find `matrix[i][j]` in memory (row i starts i·10 elements in).
Advantage of modular programming?	Each function is small and testable; `inputMatrix` is reused for A and B; easier to debug and modify.
Can a square matrix be transposed in place?	Yes: swap A[i][j] with A[j][i] for j > i only (the upper triangle). A rectangular one can't, because its shape changes.
What is a symmetric matrix?	A square matrix equal to its transpose (A[i][j] = A[j][i]).

5.5 If the examiner changes the question

"Multiply two 3 × 3 matrices", "check whether they can be multiplied" and "transpose of a 3 × 3" are all the programs above (type 3 3 as the sizes). The new ones:

Add / subtract (same order needed: r1 == r2 and c1 == c2, checked in main):

```c
// Both matrices must have the SAME order (rows x cols)
void addMatrix(int A[][10], int B[][10], int S[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            S[i][j] = A[i][j] + B[i][j];
}
```

```c
void subtractMatrix(int A[][10], int B[][10], int D[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            D[i][j] = A[i][j] - B[i][j];
}
```

Symmetric or not:

```
// Symmetric = square AND A[i][j] == A[j][i] everywhere (A equals its transpose)
bool isSymmetric(int A[][10], int rows, int cols)
{
    if (rows != cols)
        return false;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            if (A[i][j] != A[j][i])
                return false;
    return true;
}
```

In `main`: `if (isSymmetric(A, rows, cols)) cout << "Symmetric"; else cout << "Not symmetric";`

> **CHECK 5.1**
> A = [[2, 0], [1, 3]], B = [[1, 4], [5, 2]]. Compute C = A × B by hand, cell by cell.

> **CHECK 5.2**
> In `multiplyMatrix`, someone writes `for (int k = 0; k < c2; k++)`. For 2 × 3 times 3 × 2 matrices, what goes wrong?

Answers

Check 5.1: C[0][0] = 2·1 + 0·5 = 2; C[0][1] = 2·4 + 0·2 = 8; C[1][0] = 1·1 + 3·5 = 16; C[1][1] = 1·4 + 3·2 = 10 → **[[2, 8], [16, 10]]**.

Check 5.2: k runs only to 1 (c2 = 2) instead of 2, so the third product A[i][2]·B[2][j] is never added and every cell is wrong. k must run over the **shared** size, c1 (= r2).

6 · Exp 7 — Insertion sort (and its performance)

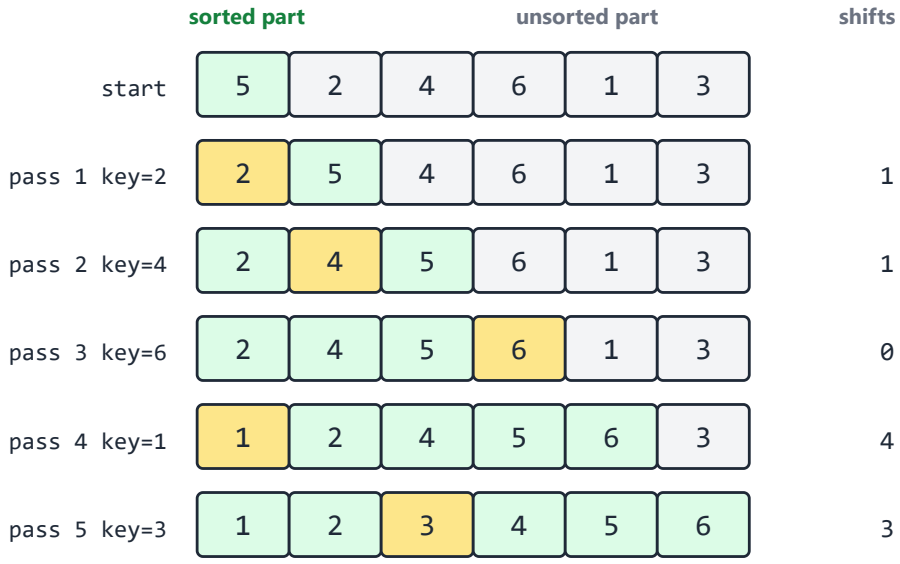
You already know the core move from Exp 3's "insert into a sorted array". Insertion sort just does it for every element in turn.

Sorted part / unsorted part → Pick key → **Shift bigger ones right** → Drop key in gap → Repeat

6.1 The idea

Like sorting playing cards in your hand: the left part is already sorted. Pick up the next card (the **key**), slide every bigger card one place right, and drop the key into the gap.

> **GUESS FIRST**
>
> `5 2 4 6 1 3`. How many single-place shifts does insertion sort need in total? (Count them pass by pass.) And how many for the already-sorted `1 2 3 4 5 6`?



yellow = where the key landed; total shifts = 9

The manual's example, pass by pass. Green = sorted part; yellow = where the key landed.

```
int key = arr[i];
int j = i - 1;
while (j >= 0 && arr[j] > key)   // bigger than key? move it right
{
    arr[j + 1] = arr[j];
    j--;
}
arr[j + 1] = key;                // j stopped one left of the gap
```

WHY `ARR[J + 1] = KEY` AND NOT `ARR[J]`

The loop stops when `arr[j]` is **not** bigger than the key (or j = −1). The gap is one to the right of that element, at j + 1.

Why `j >= 0` comes **first**: when the key is the smallest (key = 1 in pass 4), j reaches −1. `&&` stops at the first false, so `arr[-1]` is never read. Swap the order and the program reads outside the array.

6.2 The program (corrected)

The manual's sort was correct. The only fix: `int arr[n]` (a size read at run time) isn't standard C++, and strict compilers reject it:

G++ SAYS

```
m07_vla.cpp:25:9: error: ISO C++ forbids variable length array 'arr' [-Wvla]
   25 |     int arr[n];
      |         ^~~
```

So the array is `int arr[100]` with a range check on n.

E07_INSERTION_SORT.CPP

```cpp
#include <iostream>
using namespace std;

// Function to perform Insertion Sort
void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];                    // card picked up
        int j = i - 1;

        // Shift elements greater than key one position to the right
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;                    // drop the card in the gap
    }
}

// Function to display the array
void displayArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main()
{
    int n;
    int arr[100];                            // FIX: was int arr[n] (not standard C++)
```

```
    cout << "Enter number of elements: ";
    cin >> n;

    if (n < 1 || n > 100)
    {
        cout << "n must be between 1 and 100.";
        return 0;
    }

    cout << "Enter elements of the array:" << endl;
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    cout << "Original Array: ";
    displayArray(arr, n);

    insertionSort(arr, n);

    cout << "Sorted Array: ";
    displayArray(arr, n);

    return 0;
}
```

SCREEN (REAL RUN)

```
Enter number of elements: 6
Enter elements of the array:
5 2 4 6 1 3
Original Array: 5 2 4 6 1 3
Sorted Array: 1 2 3 4 5 6
```

Duplicates and negatives work too (the strict `>` keeps equal elements in their original order, which is why the sort is **stable**):

SCREEN (REAL RUN)

```
Enter number of elements: 7
Enter elements of the array:
3 -1 3 0 9 -1 2
Original Array: 3 -1 3 0 9 -1 2
Sorted Array: -1 -1 0 2 3 3 9
```

6.3 Performance analysis (the "analyse" half of the aim)

These counts come from the tested counting version (section 6.5):

Input (n = 6)	Comparisons	Shifts	Case
1 2 3 4 5 6 (sorted)	5 = n − 1	0	**best**: O(n)
5 2 4 6 1 3 (manual)	12	9	average: $O(n^2)$
6 5 4 3 2 1 (reversed)	15 = n(n−1)/2	15	**worst**: $O(n^2)$

- **Best case O(n):** each key is compared once with its left neighbour, and stops.
- **Worst case O(n^2):** key i is compared with and shifted past all i elements before it: 1 + 2 + … + (n−1) = n(n−1)/2.
- **Space O(1):** sorts inside the same array using only `key` and `j`. It's **in-place**.

WRITE IN YOUR RECORD

Aim: To sort an integer array in ascending order using insertion sort and analyse its performance.

Algorithm: for i = 1 to n−1: key = arr[i], j = i−1; while j ≥ 0 and arr[j] > key: arr[j+1] = arr[j], j−−; then arr[j+1] = key. Print the array.

Complexity: best O(n) (sorted input), average and worst O(n^2) (reverse-sorted input: n(n−1)/2 comparisons); space O(1).

Result: 5 2 4 6 1 3 → 1 2 3 4 5 6 in 5 passes (9 shifts). Insertion sort is in-place, stable, and fast for small or nearly sorted data.

6.4 Viva questions

Question	Answer
Why does the outer loop start at i = 1?	A single element (index 0) is already "sorted".
Best, average, worst case?	O(n), O(n^2), O(n^2).
What input gives the best case? The worst?	Already sorted; reverse sorted.
Is it stable? In-place?	Both yes: `>` (not `>=`) keeps equal keys in order; only O(1) extra memory.
When is insertion sort a good choice?	Small arrays, nearly-sorted data, or data arriving one item at a time ("online").
Why save arr[i] in key?	The first shift overwrites arr[i]; without the copy the value would be lost.
How many passes for n elements?	n − 1.

6.5 If the examiner changes the question

"Separate functions for input, sorting and display" is the program above (add an `inputArray` function if asked). The rest:

Descending order (one character changes):

```
// Descending: the ONLY change is  arr[j] > key  ->  arr[j] < key
void insertionSortDesc(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] < key)
        {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

Count comparisons and shifts. Used for the table in 6.3, and covers "compare performance for sorted, reverse-sorted and random arrays": run it on all three and print the counts.

```
// Counts comparisons (each arr[j] > key test) and shifts (each arr[j+1] = arr[j])
void insertionSortCount(int arr[], int n, int &comparisons, int &shifts)
{
    comparisons = 0;
    shifts = 0;
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0)
        {
            comparisons++;
            if (arr[j] > key)
            {
                arr[j + 1] = arr[j];
                shifts++;
                j--;
            }
            else
                break;
        }
        arr[j + 1] = key;
    }
}
```

Call: `int c, s; insertionSortCount(arr, n, c, s); cout << "Comparisons = " << c << ", Shifts = " << s;`

Floating-point numbers:

```c
// Floating-point version: change the type of the array and of key - nothing else
void insertionSortFloat(float arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        float key = arr[i];
        int j = i - 1;                      // j is an index: stays int
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

Position of an element after sorting:

```c
// After sorting, find where x ended up (1-based position, -1 if absent)
int findPosition(int arr[], int n, int x)
{
    for (int i = 0; i < n; i++)
        if (arr[i] == x)
            return i + 1;
    return -1;
}
```

CHECK 6.1

Sort `4 3 2 10 12 1 5 6` by hand, pass by pass. Write the array after each pass.

CHECK 6.2

Change one character in the while condition to sort in **descending** order. Which character, and what does it become?

Answers

Check 6.1: key 3 → `3 4 2 10 12 1 5 6`; key 2 → `2 3 4 10 12 1 5 6`; key 10 → no change; key 12 → no change; key 1 → `1 2 3 4 10 12 5 6`; key 5 → `1 2 3 4 5 10 12 6`; key 6 → `1 2 3 4 5 6 10 12`. Seven passes (n − 1).

Check 6.2: `arr[j] > key` → `arr[j] < key`: move **smaller** elements right, so bigger ones end up first.

7 · Exp 8 — Quick sort, and comparison with other sorts

The hardest program in the lab: recursion + partition. Learn partition first; once that's solid, quickSort itself is 6 lines.

Pick pivot (last) → **Partition: small left, big right** → Pivot is now final → Recurse on left part → Recurse on right part

7.1 The idea: divide and conquer

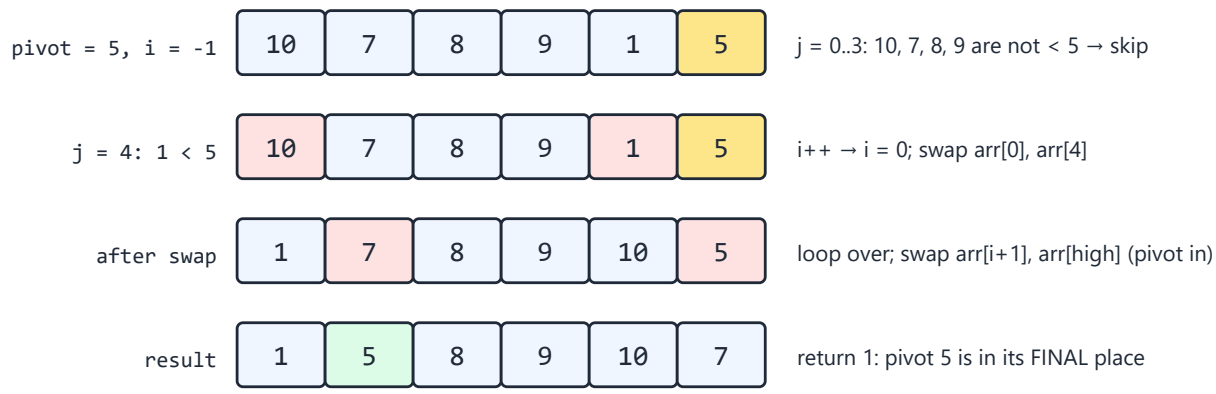
1. **Pick a pivot.** This program uses the **last** element.
2. **Partition:** rearrange so everything smaller than the pivot is on its left and everything else on its right. The pivot is then in its **final sorted position**.
3. **Recurse:** quick-sort the left part and the right part the same way. A part of 0 or 1 element is already sorted, and the recursion stops there.

7.2 Partition, step by step (learn this cold)

GUESS FIRST

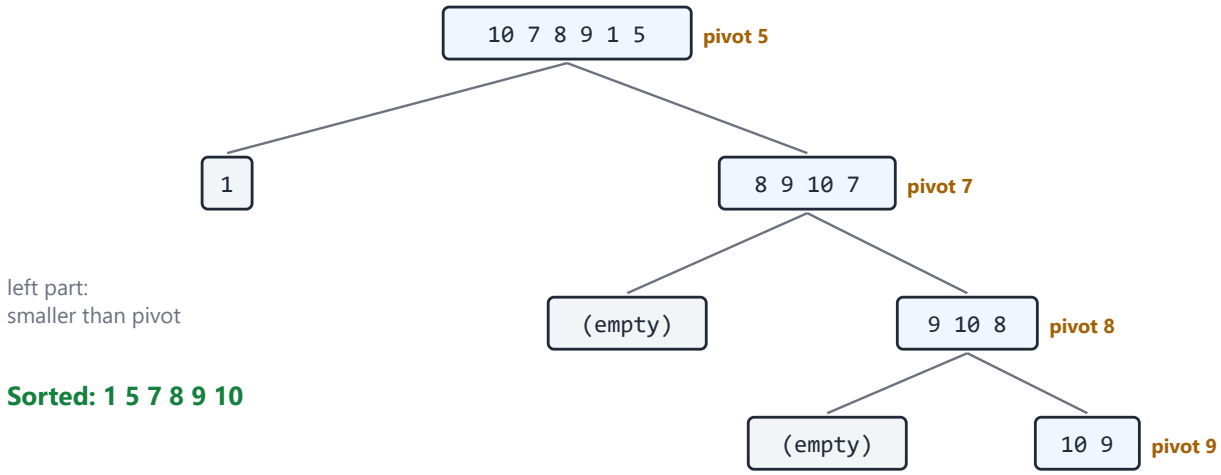
In `partitionArray`, the variable `i` starts at `low - 1`. What does `i` keep track of? (Hint: watch where small elements get swapped to.)

```
int pivot = arr[high];
int i = low - 1;                // last index of the "smaller than pivot" zone
for (int j = low; j < high; j++)
    if (arr[j] < pivot)         // small one found: grow the zone, swap it in
    {
        i++;
        swapElements(arr[i], arr[j]);
    }
swapElements(arr[i + 1], arr[high]);   // pivot goes right after the zone
return i + 1;                          // pivot's final index
```



yellow = pivot, red = the two cells being swapped, green = pivot placed

First partition of the manual's example 10 7 8 9 1 5. Same result as the manual's "Pass 1".



The whole recursion. Each box is one call to partition; the manual's Passes 1–4 are these four pivots.

WHAT `INT &A` DOES IN SWAPELEMENTS

`swapElements(arr[i], arr[j])` must change the array itself. `int &a` makes `a` **another name** for `arr[i]` (a reference, chapter 1.5). With plain `int a`, the function would swap two copies and the array would not change.

7.3 The program (corrected)

Same VLA fix as Exp 7 (`int arr[100]`). Partition and recursion are exactly the manual's.

E08_QUICK_SORT.CPP

```cpp
#include <iostream>
using namespace std;

// Function to swap two elements (& = work on the caller's variables)
void swapElements(int &a, int &b)
{
    int temp = a;
    a = b;
    b = temp;
}

// Function to partition the array around the last element
int partitionArray(int arr[], int low, int high)
{
    int pivot = arr[high];                  // last element as pivot
    int i = low - 1;                        // end of the "smaller than pivot" zone

    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swapElements(arr[i], arr[j]);
        }
    }
    swapElements(arr[i + 1], arr[high]);    // pivot goes just after the small zone
    return i + 1;                           // pivot's final index
}
```

```cpp
// Function to implement Quick Sort
void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partitionArray(arr, low, high);
        quickSort(arr, low, pi - 1);         // sort left part
        quickSort(arr, pi + 1, high);        // sort right part
    }
}

// Function to display the array
void displayArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main()
{
    int n;
    int arr[100];                            // FIX: was int arr[n] (not standard C++)

    cout << "Enter number of elements: ";
    cin >> n;

    if (n < 1 || n > 100)
    {
        cout << "n must be between 1 and 100.";
        return 0;
    }

    cout << "Enter elements of the array:" << endl;
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    cout << "Original Array: ";
    displayArray(arr, n);

    quickSort(arr, 0, n - 1);

    cout << "Sorted Array using Quick Sort: ";
    displayArray(arr, n);

    return 0;
}
```

SCREEN (REAL RUN)

```
Enter number of elements: 6
Enter elements of the array:
10 7 8 9 1 5
Original Array: 10 7 8 9 1 5
Sorted Array using Quick Sort: 1 5 7 8 9 10
```

With duplicates and negatives:

SCREEN (REAL RUN)

```
Enter number of elements: 8
Enter elements of the array:
4 4 1 9 0 4 -3 7
Original Array: 4 4 1 9 0 4 -3 7
Sorted Array using Quick Sort: -3 0 1 4 4 4 7 9
```

7.4 The missing half: comparing with other sorts

The aim says "compare its performance with other sorting techniques". The manual's algorithm (steps 11–12) and explanation table list `bubbleSort()`, `selectionSort()`, `insertionSort()`, `copyArray()` and `clock()`, but **none of them are in the printed code**. Here is the full program:

- Every sort gets its **own copy** of the same input (`copyArray`). Sorting the already-sorted output again wouldn't be a fair test.
- Timing **6 numbers** gives 0 ms for everything, so part 2 times 10 000 random numbers. `srand(1)` makes the "random" data the same on every run.
- `clock()` counts processor ticks; `(end - start) / CLOCKS_PER_SEC` converts to seconds, and × 1000 to milliseconds.

E08_SORT_COMPARE.CPP

```cpp
#include <iostream>
#include <cstdlib>     // rand, srand
#include <ctime>       // clock, CLOCKS_PER_SEC
using namespace std;

const int BIG = 10000;                      // size used for the timing test

void swapElements(int &a, int &b)
{
    int temp = a;
    a = b;
    b = temp;
}

int partitionArray(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
        if (arr[j] < pivot)
        {
            i++;
            swapElements(arr[i], arr[j]);
        }
    swapElements(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partitionArray(arr, low, high);
        quickSort(arr, low, pi - 1);
```

```
        quickSort(arr, pi + 1, high);
    }
}

// Bubble Sort: neighbours swap until the largest "bubbles" to the end
void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        bool swapped = false;
        for (int j = 0; j < n - 1 - i; j++)
            if (arr[j] > arr[j + 1])
            {
                swapElements(arr[j], arr[j + 1]);
                swapped = true;
            }
        if (!swapped)                       // no swap in a pass = already sorted
            break;
    }
}

// Selection Sort: pick the smallest of the rest, put it at position i
void selectionSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;
        for (int j = i + 1; j < n; j++)
            if (arr[j] < arr[minIndex])
                minIndex = j;
        swapElements(arr[i], arr[minIndex]);
    }
}

void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

void copyArray(int source[], int dest[], int n)
{
    for (int i = 0; i < n; i++)
        dest[i] = source[i];
}

void displayArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}
```

```cpp
int original[BIG], work[BIG];               // global: too big for the stack

int main()
{
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    if (n < 1 || n > 100)
    {
        cout << "n must be between 1 and 100.";
        return 0;
    }

    cout << "Enter elements of the array:" << endl;
    for (int i = 0; i < n; i++)
        cin >> original[i];

    // Part 1: every algorithm sorts the SAME input (each gets its own copy)
    copyArray(original, work, n);  quickSort(work, 0, n - 1);
    cout << "Quick Sort     : ";  displayArray(work, n);
    copyArray(original, work, n);  bubbleSort(work, n);
    cout << "Bubble Sort    : ";  displayArray(work, n);
    copyArray(original, work, n);  selectionSort(work, n);
    cout << "Selection Sort : ";  displayArray(work, n);
    copyArray(original, work, n);  insertionSort(work, n);
    cout << "Insertion Sort : ";  displayArray(work, n);

    // Part 2: timing needs a big array - 6 numbers sort in ~0 ms
    srand(1);                                // fixed seed: same "random" data every run
    for (int i = 0; i < BIG; i++)
        original[i] = rand() % 100000;

    cout << "\nTime to sort " << BIG << " random numbers:" << endl;
    clock_t start, end;

    copyArray(original, work, BIG);
    start = clock();  quickSort(work, 0, BIG - 1);  end = clock();
    cout << "Quick Sort     : " << 1000.0 * (end - start) / CLOCKS_PER_SEC << " ms" << endl;

    copyArray(original, work, BIG);
    start = clock();  bubbleSort(work, BIG);  end = clock();
    cout << "Bubble Sort    : " << 1000.0 * (end - start) / CLOCKS_PER_SEC << " ms" << endl;

    copyArray(original, work, BIG);
    start = clock();  selectionSort(work, BIG);  end = clock();
    cout << "Selection Sort : " << 1000.0 * (end - start) / CLOCKS_PER_SEC << " ms" << endl;

    copyArray(original, work, BIG);
    start = clock();  insertionSort(work, BIG);  end = clock();
    cout << "Insertion Sort : " << 1000.0 * (end - start) / CLOCKS_PER_SEC << " ms" << endl;

    return 0;
}
```

SCREEN (REAL RUN)

```
Enter number of elements: 6
Enter elements of the array:
10 7 8 9 1 5
```

```
Quick Sort     : 1 5 7 8 9 10
Bubble Sort    : 1 5 7 8 9 10
Selection Sort : 1 5 7 8 9 10
Insertion Sort : 1 5 7 8 9 10

Time to sort 10000 random numbers:
Quick Sort     : 1 ms
Bubble Sort    : 114 ms
Selection Sort : 53 ms
Insertion Sort : 37 ms
```

The times vary from PC to PC, but the **ranking** is what the experiment shows: quick sort is far faster than the three $O(n^2)$ sorts on large random input.

Algorithm	Best	Average	Worst	Extra space	Stable?
Quick sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n) avg (recursion)	No
Bubble sort (with early exit)	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No
Insertion sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Merge sort (assignment)	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes

QUICK SORT'S WORST CASE, IN THIS PROGRAM

With the **last element as pivot**, an **already sorted** array is the worst case: every pivot is the largest, so each partition peels off just one element. That gives $n(n-1)/2$ comparisons (15 for $n = 6$; tested in 7.6). Fix: pick the middle (or a random) element as pivot.

WRITE IN YOUR RECORD

Aim: To sort an integer array using quick sort and compare its performance with bubble, selection and insertion sort.

Algorithm: quickSort(low, high): if low < high → pi = partition(low, high) → quickSort(low, pi−1) → quickSort(pi+1, high). partition: pivot = arr[high], i = low−1; for j = low…high−1: if arr[j] < pivot, i++ and swap arr[i], arr[j]; finally swap arr[i+1], arr[high]; return i+1. Comparison: copy the same input for each sort, time each with clock().

Complexity: best/average O(n log n), worst $O(n^2)$ (sorted input with last-element pivot); space O(log n) average for the recursion, O(n) worst.

Result: 10 7 8 9 1 5 → 1 5 7 8 9 10. All four sorts gave the same output. On 10 000 random numbers quick sort took about 1 ms; bubble, selection and insertion took tens to over a hundred ms (bubble slowest).

7.5 Viva questions

Question	Answer
What is divide and conquer?	Split into smaller subproblems, solve each (recursively), combine. Quick sort splits by a pivot.
What is a pivot?	The element used to split; after partition it sits at its final sorted index.
Why `low < high` in quickSort?	A part with 0 or 1 element is sorted. This is the recursion's stopping condition (base case).
Why O(n log n) on average?	Each level of recursion does O(n) partition work, and balanced splits give about $\log_2 n$ levels.
Worst case and when?	$O(n^2)$, when the pivot is always the smallest or largest (sorted / reverse-sorted input with last-element pivot).
Space complexity?	In-place, but the recursion stack uses O(log n) on average and O(n) in the worst case.
Is quick sort stable?	No. Long-distance swaps can reorder equal elements.
Why copy the array before each sort in the comparison?	So every algorithm sorts the same unsorted input; a fair test.
Why does `swapElements` use `&`?	To swap the actual array elements, not copies.

7.6 If the examiner changes the question

"Separate functions for input, partitioning, sorting and display" is the program above. The rest (7 checks, all pass):

Descending order (one character in partition):

```
// Descending: the ONLY change is  arr[j] < pivot  ->  arr[j] > pivot
int partitionDesc(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
        if (arr[j] > pivot)
        {
            i++;
            swapElements(arr[i], arr[j]);
        }
    swapElements(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSortDesc(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partitionDesc(arr, low, high);
        quickSortDesc(arr, low, pi - 1);
```

```
        quickSortDesc(arr, pi + 1, high);
    }
}
```

First element as pivot: swap it to the end, then partition exactly as before (`quickSort` stays the same, just calling `partitionFirst`):

```
// First element as pivot: swap it to the end, then partition exactly as before
int partitionFirst(int arr[], int low, int high)
{
    swapElements(arr[low], arr[high]);
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
        if (arr[j] < pivot)
        {
            i++;
            swapElements(arr[i], arr[j]);
        }
    swapElements(arr[i + 1], arr[high]);
    return i + 1;
}
```

Middle element as pivot (the worst-case fix):

```
// Middle element as pivot: same trick (sorted input no longer gives the worst case)
int partitionMiddle(int arr[], int low, int high)
{
    int mid = low + (high - low) / 2;
    swapElements(arr[mid], arr[high]);
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
        if (arr[j] < pivot)
        {
            i++;
            swapElements(arr[i], arr[j]);
        }
    swapElements(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSortMiddle(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partitionMiddle(arr, low, high);
        quickSortMiddle(arr, low, pi - 1);
        quickSortMiddle(arr, pi + 1, high);
    }
}
```

Count comparisons and swaps. On the manual's example this gives 11 comparisons and 5 swaps; on a sorted array of 6, 15 comparisons (the worst case). Run it on sorted, reverse-sorted and random input for the "compare performance" item.

```
// Global counters: every function can add to them; reset to 0 before sorting
int comparisons = 0, swaps = 0;

int partitionCount(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
    {
        comparisons++;
        if (arr[j] < pivot)
        {
            i++;
            swapElements(arr[i], arr[j]);
            swaps++;
        }
    }
    swapElements(arr[i + 1], arr[high]);
    swaps++;
    return i + 1;
}

void quickSortCount(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partitionCount(arr, low, high);
        quickSortCount(arr, low, pi - 1);
        quickSortCount(arr, pi + 1, high);
    }
}
```

Merge sort (for "compare quick sort with merge sort"; time it with `clock()` as in 7.4):

```
// Merge Sort: split in half, sort each half, merge the two sorted halves
void merge(int arr[], int low, int mid, int high)
{
    int temp[100];                          // holds the merged result
    int i = low, j = mid + 1, k = 0;
    while (i <= mid && j <= high)
    {
        if (arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }
    while (i <= mid)                        // leftovers of the left half
        temp[k++] = arr[i++];
    while (j <= high)                       // leftovers of the right half
        temp[k++] = arr[j++];
    for (int t = 0; t < k; t++)             // copy back
        arr[low + t] = temp[t];
}

void mergeSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int mid = (low + high) / 2;
```

```
        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);
        merge(arr, low, mid, high);
    }
}
```

CHECK 7.1

Partition `4 9 2 7 5` (pivot 5). Track i and the array after each swap. What index does partition return?

CHECK 7.2

Without looking, write `quickSort(int arr[], int low, int high)`. Six lines.

Answers

Check 7.1: pivot 5, i = −1. j = 0: 4 < 5 → i = 0, swap(arr[0], arr[0]) → `4 9 2 7 5`. j = 1: 9 no. j = 2: 2 < 5 → i = 1, swap(arr[1], arr[2]) → `4 2 9 7 5`. j = 3: 7 no. End: swap(arr[2], arr[4]) → `4 2 5 7 9`. Returns **2**.

Check 7.2: `void quickSort(int arr[], int low, int high) { if (low < high) { int pi = partitionArray(arr, low, high); quickSort(arr, low, pi - 1); quickSort(arr, pi + 1, high); } }`

8 · Exp 9 (probable) — Stack using an array

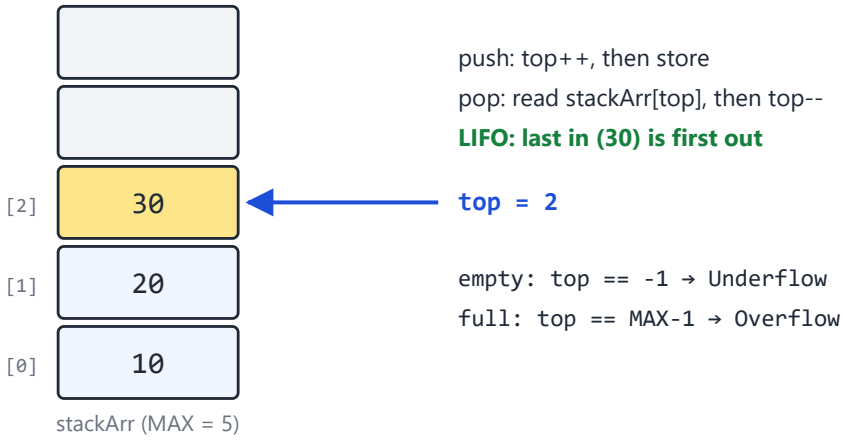
Not in your uploaded files. No Exp 9 file arrived, but Exp 10's figures are captioned "Enqueue Operation on Stack", which points to a stack experiment just before it. This chapter is written in exactly the style of Exp 10, so if Exp 9 is Stack, you're covered. Confirm with a classmate.

One open end (top) → push: top++, store → pop: read, top-- → LIFO

8.1 The idea

A **stack** is a pile: you add and remove only at the **top**. The last item pushed is the first popped: **LIFO** (Last In, First Out). Examples: undo history, the browser back button, function calls (the "call stack" quick sort's recursion uses).

> **GUESS FIRST**
> Push 10, 20, 30, then pop twice. Which values come out, in what order, and what is left?



An array stack after pushing 10, 20, 30. top is the index of the top element; −1 means empty.

8.2 The program

New C++ here (also used in Exp 10):

- `#define MAX 5`: the compiler replaces every `MAX` with 5. Change it in one place to resize.
- **Global variables** (declared above all functions): `stackArr` and `top` are shared by `push`, `pop`, `peek`, `display` and `main`.
- **Menu loop:** `do { show menu; read choice; act } while (choice != 5);` runs until the user picks Exit.

E09_STACK.CPP

```
#include <iostream>
using namespace std;

#define MAX 5
int stackArr[MAX];
int top = -1;                                   // -1 = empty
```

```
void push(int value)
{
    if (top == MAX - 1)
        cout << "Stack Overflow!" << endl;
    else
    {
        top++;
        stackArr[top] = value;
        cout << value << " pushed into stack." << endl;
    }
}

void pop()
{
    if (top == -1)
        cout << "Stack Underflow!" << endl;
    else
    {
        cout << stackArr[top] << " popped from stack." << endl;
        top--;
    }
}

void peek()
{
    if (top == -1)
        cout << "Stack is empty." << endl;
    else
        cout << "Top element is " << stackArr[top] << endl;
}

void display()
{
    if (top == -1)
        cout << "Stack is empty." << endl;
    else
    {
        cout << "Stack elements (top to bottom):" << endl;
        for (int i = top; i >= 0; i--)
            cout << stackArr[i] << " ";
        cout << endl;
    }
}

int main()
{
    int choice, value;

    do
    {
        cout << "\n1. Push" << endl;
        cout << "2. Pop" << endl;
        cout << "3. Peek" << endl;
        cout << "4. Display" << endl;
        cout << "5. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to push: ";
```

```
            cin >> value;
            push(value);
        }
        else if (choice == 2)
            pop();
        else if (choice == 3)
            peek();
        else if (choice == 4)
            display();
        else if (choice == 5)
            cout << "Program terminated." << endl;
        else
            cout << "Invalid choice!" << endl;
    } while (choice != 5);

    return 0;
}
```

Push 10, 20, 30 → display → peek → pop all three → pop again (underflow) → exit:

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 10
10 pushed into stack.
Enter your choice: 1
Enter value to push: 20
20 pushed into stack.
Enter your choice: 1
Enter value to push: 30
30 pushed into stack.
Enter your choice: 4
Stack elements (top to bottom):
30 20 10
Enter your choice: 3
Top element is 30
Enter your choice: 2
30 popped from stack.
Enter your choice: 4
Stack elements (top to bottom):
20 10
Enter your choice: 2
20 popped from stack.
Enter your choice: 2
10 popped from stack.
Enter your choice: 2
Stack Underflow!
Enter your choice: 5
Program terminated.
```

Overflow test (6 pushes into MAX = 5):

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Push
2. Pop
```

```
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 1
1 pushed into stack.
Enter your choice: 1
Enter value to push: 2
2 pushed into stack.
Enter your choice: 1
Enter value to push: 3
3 pushed into stack.
Enter your choice: 1
Enter value to push: 4
4 pushed into stack.
Enter your choice: 1
Enter value to push: 5
5 pushed into stack.
Enter your choice: 1
Enter value to push: 6
Stack Overflow!
Enter your choice: 5
Program terminated.
```

WRITE IN YOUR RECORD

Aim: To implement a stack using an array with push, pop, peek and display operations.

Algorithm: top = −1. *Push:* if top == MAX−1 → overflow; else top++, stackArr[top] = value. *Pop:* if top == −1 → underflow; else print stackArr[top], top−−. *Peek:* print stackArr[top]. *Display:* print from top down to 0.

Complexity: push, pop, peek O(1); display O(n); space O(MAX).

Result: Pushed 10, 20, 30; popped 30, 20, 10 (LIFO). Overflow on the 6th push and underflow on popping an empty stack were detected.

8.3 Viva questions

Question	Answer
What is LIFO?	Last In, First Out: the most recently added item is removed first.
Overflow / underflow?	Push onto a full stack (top == MAX−1) / pop from an empty stack (top == −1).
Why does top start at −1?	So the first push makes it 0, the first array index. −1 = empty.
Stack vs. queue?	Stack: one end, LIFO. Queue: insert at rear, delete at front, FIFO.
Applications?	Function calls and recursion, undo, expression evaluation (postfix), bracket matching, backtracking.
Does pop erase the value?	No, it only moves top down; the old value is overwritten by the next push.

CHECK 8.1

MAX = 3. Operations: push 4, push 7, pop, push 9, push 2, push 6. What does each print? What is top at the end?

Answers

Check 8.1: 4 pushed (top 0), 7 pushed (top 1), pop prints 7 (top 0), 9 pushed (top 1), 2 pushed (top 2), push 6 → **Stack Overflow!** (top == MAX−1 = 2). top ends at **2**, and the stack is 4 9 2 (bottom → top).

9 · Exp 10 — Queue using an array (enqueue, dequeue)

A queue is a line at a ticket counter: join at the back (rear), leave from the front. Two indexes, front and rear, do all the work.

Two ends → enqueue: rear++ → dequeue: front++ → FIFO → linear-queue flaw → circular queue

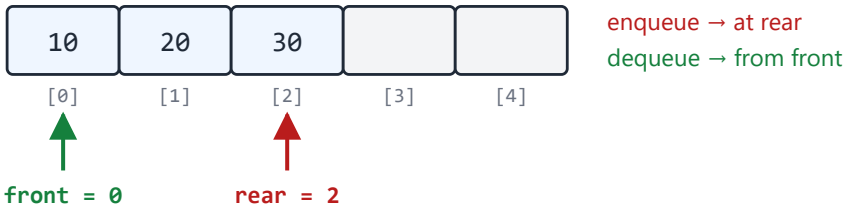
9.1 The idea

A **queue** inserts at the **rear** and removes from the **front**: **FIFO** (First In, First Out). `front` and `rear` both start at **−1** (empty).

GUESS FIRST

MAX = 5. Enqueue 1, 2, 3, 4, 5, dequeue twice, then enqueue 6. There are 2 empty slots. Does the program accept 6?

After enqueue 10, 20, 30 (MAX = 5)



After 1 dequeue:



The manual's example: enqueue 10, 20, 30, then one dequeue.

Operation	Check first	Then
enqueue(v)	`rear == MAX-1` → Overflow	if `front == -1` set `front = 0`; `rear++`; `queueArr[rear] = v`
dequeue()	`front == -1` → Underflow	print `queueArr[front]`; if `front == rear` (last one) reset both to −1, else `front++`
display()	`front == -1` → empty	print `queueArr[front]` … `queueArr[rear]`

WHY RESET TO −1 WHEN THE LAST ELEMENT LEAVES

After dequeuing the only element, front would pass rear, which is confusing. Resetting both to −1 means "empty" is always `front == -1`, and the next enqueue starts again from index 0.

9.2 The program (corrected)

The manual's **listing** (page 5) is right; its **screenshot** (page 9) doesn't compile. In the screenshot, `enqueue(value);` sits after the `if`'s closing brace, so the next `else` has no `if`; choice 3 prints "Program terminated" instead of displaying; and there's a bare `else` with no statement:

G++ SAYS

```
m10_screenshot.cpp:23:9: error: 'else' without a previous 'if'
   23 |         else if (choice == 2)
      |         ^~~~
m10_screenshot.cpp:25:9: error: expected '}' before 'else'
   25 |         else if (choice == 3)
      |         ^~~~
m10_screenshot.cpp:28:5: error: expected primary-expression before '}' token
   28 |     } while (choice != 4);
      |     ^
```

Two more fixes: the array is renamed from `queue` to `queueArr` (with `#include <bits/stdc++.h>`, `queue` clashes with the library's `std::queue`), and the dead `|| front > rear` test is dropped (after the reset rule it can never be true).

E10_QUEUE.CPP

```cpp
#include <iostream>
using namespace std;

#define MAX 5
int queueArr[MAX];         // FIX: renamed from "queue" - clashes with std::queue
int front = -1;
int rear = -1;

void enqueue(int value)
{
    if (rear == MAX - 1)
    {
        cout << "Queue Overflow!" << endl;
    }
    else
    {
        if (front == -1)                    // first element
            front = 0;
        rear++;
        queueArr[rear] = value;
        cout << value << " inserted into queue." << endl;
    }
}

void dequeue()
{
    if (front == -1)
    {
        cout << "Queue Underflow!" << endl;
    }
    else
    {
        cout << queueArr[front] << " deleted from queue." << endl;
        if (front == rear)                  // that was the last element
        {
```

```
            front = -1;
            rear = -1;
        }
        else
        {
            front++;
        }
    }
}

void display()
{
    if (front == -1)
    {
        cout << "Queue is empty." << endl;
    }
    else
    {
        cout << "Queue elements are:" << endl;
        for (int i = front; i <= rear; i++)
            cout << queueArr[i] << " ";
        cout << endl;
    }
}

int main()
{
    int choice, value;

    do
    {
        cout << "\n1. Enqueue" << endl;
        cout << "2. Dequeue" << endl;
        cout << "3. Display" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to enqueue: ";
            cin >> value;
            enqueue(value);
        }
        else if (choice == 2)
        {
            dequeue();
        }
        else if (choice == 3)
        {
            display();
        }
        else if (choice == 4)
        {
            cout << "Program terminated." << endl;
        }
        else
        {
            cout << "Invalid choice!" << endl;
        }
    } while (choice != 4);
```

```
    return 0;
}
```

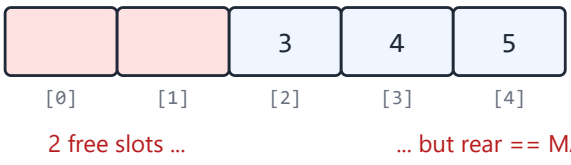
The manual's Sample Inputs 1 and 2 in one run:

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 10
10 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 20
20 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 30
30 inserted into queue.
Enter your choice: 3
Queue elements are:
10 20 30
Enter your choice: 2
10 deleted from queue.
Enter your choice: 3
Queue elements are:
20 30
Enter your choice: 4
Program terminated.
```

9.3 The linear queue's flaw (a favourite viva question)

Linear queue flaw: enqueue 1..5, dequeue twice, enqueue 7



so the program prints "Queue Overflow!" (verified run). The circular queue fixes this.

Slots freed by dequeue are never reused, because rear only moves forward.

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Queue Underflow!
Enter your choice: 1
Enter value to enqueue: 1
1 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 2
2 inserted into queue.
```

```
Enter your choice: 1
Enter value to enqueue: 3
3 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 4
4 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 5
5 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 6
Queue Overflow!
Enter your choice: 2
1 deleted from queue.
Enter your choice: 2
2 deleted from queue.
Enter your choice: 1
Enter value to enqueue: 7
Queue Overflow!
Enter your choice: 3
Queue elements are:
3 4 5
Enter your choice: 4
Program terminated.
```

Underflow on the empty queue; then 1–5 fill it; 6 overflows (correct, it's full); two dequeues free slots 0 and 1, **yet 7 still overflows**, because rear is still MAX−1. The fix is the **circular queue** (9.5).

WRITE IN YOUR RECORD

Aim: To implement a queue using an array and perform enqueue, dequeue and display operations.

Algorithm: front = rear = −1. *Enqueue:* if rear == MAX−1 → overflow; else (if front == −1, front = 0), rear++, queue[rear] = value. *Dequeue:* if front == −1 → underflow; else print queue[front]; if front == rear, front = rear = −1, else front++. *Display:* print queue[front…rear].

Complexity: enqueue O(1), dequeue O(1), display O(n); space O(MAX).

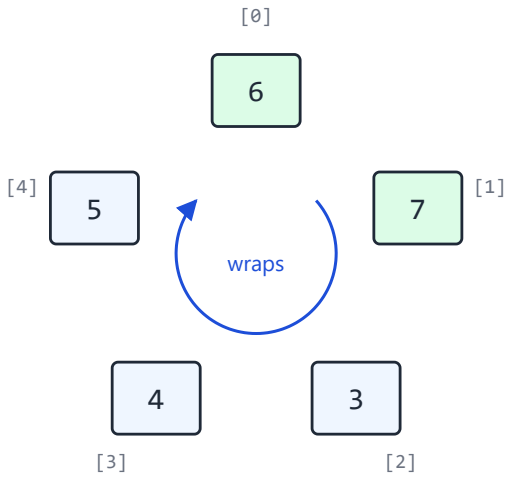
Result: Enqueued 10, 20, 30 (display 10 20 30); dequeue removed 10 (display 20 30), confirming FIFO. Overflow and underflow were detected.

9.4 Viva questions

Question	Answer
What is FIFO?	First In, First Out: elements leave in the order they arrived.
Where is insertion / deletion done?	Insertion at the rear, deletion at the front.
Overflow / underflow conditions?	rear == MAX−1 / front == −1.
Drawback of a linear array queue?	Freed slots at the front are never reused, so it can report overflow while slots are empty.
How does a circular queue fix it?	Indexes wrap: `rear = (rear + 1) % MAX`, so slot 0 is reused after slot MAX−1.
Applications?	CPU and printer scheduling, keyboard buffers, network packets, breadth-first search.
Why are enqueue and dequeue O(1)?	Each touches one index; nothing is shifted (unlike array deletion in Exp 4).

9.5 If the examiner changes the question

Circular queue (full program). The only changes from Exp 10: wrap with `% MAX`, and "full" becomes `(rear + 1) % MAX == front`.



front = 2 (value 3)
rear = 1 (value 7)

rear = (rear + 1) % MAX
4 → (4+1) % 5 = 0

full when (rear+1) % MAX == front

green = new values stored in the slots the linear queue wasted

After enqueue 1–5, dequeue 1 and 2, enqueue 6 and 7: the new values reuse slots 0 and 1.

E10B_CIRCULAR_QUEUE.CPP

```cpp
// Exp 10 assignment: Circular Queue using an array.
// Same as the linear queue, except rear and front wrap around with  % MAX.
#include <iostream>
using namespace std;

#define MAX 5
int cq[MAX];
int front = -1, rear = -1;
```

```
bool isEmpty() { return front == -1; }
bool isFull()  { return (rear + 1) % MAX == front; }   // next slot after rear is front

void enqueue(int value)
{
    if (isFull())
    {
        cout << "Queue Overflow!" << endl;
        return;
    }
    if (isEmpty())
        front = 0;
    rear = (rear + 1) % MAX;                 // 4 -> 0 : wrap around
    cq[rear] = value;
    cout << value << " inserted into queue." << endl;
}

void dequeue()
{
    if (isEmpty())
    {
        cout << "Queue Underflow!" << endl;
        return;
    }
    cout << cq[front] << " deleted from queue." << endl;
    if (front == rear)                       // last element removed
        front = rear = -1;
    else
        front = (front + 1) % MAX;
}

void display()
{
    if (isEmpty())
    {
        cout << "Queue is empty." << endl;
        return;
    }
    cout << "Queue elements are:" << endl;
    int i = front;
    while (true)
    {
        cout << cq[i] << " ";
        if (i == rear)
            break;
        i = (i + 1) % MAX;
    }
    cout << endl;
}

int main()
{
    int choice, value;
    do
    {
        cout << "\n1. Enqueue" << endl;
        cout << "2. Dequeue" << endl;
        cout << "3. Display" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
```

```
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to enqueue: ";
            cin >> value;
            enqueue(value);
        }
        else if (choice == 2)
            dequeue();
        else if (choice == 3)
            display();
        else if (choice == 4)
            cout << "Program terminated." << endl;
        else
            cout << "Invalid choice!" << endl;
    } while (choice != 4);

    return 0;
}
```

The same sequence that defeated the linear queue. 6 and 7 are now accepted:

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 1
1 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 2
2 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 3
3 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 4
4 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 5
5 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 6
Queue Overflow!
Enter your choice: 2
1 deleted from queue.
Enter your choice: 2
2 deleted from queue.
Enter your choice: 1
Enter value to enqueue: 6
6 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 7
7 inserted into queue.
Enter your choice: 3
Queue elements are:
3 4 5 6 7
Enter your choice: 1
```

```
Enter value to enqueue: 8
Queue Overflow!
Enter your choice: 4
Program terminated.
```

Linear vs. circular queue ("WAP to compare"): run both programs on the input above. The linear one reports overflow with free slots; the circular one stores 6 and 7. Both are O(1) per operation.

Peek, isEmpty / isFull, show front and rear (add to Exp 10's program and to the menu):

```
void peek()
{
    if (front == -1)
        cout << "Queue is empty." << endl;
    else
        cout << "Front element is " << queueArr[front] << endl;   // look, don't remove
}
```

```
bool isEmpty() { return front == -1; }
bool isFull()  { return rear == MAX - 1; }
```

```
void showFrontRear()
{
    if (front == -1)
        cout << "Queue is empty." << endl;
    else
        cout << "Front = " << queueArr[front] << ", Rear = " << queueArr[rear] << endl;
}
```

Reverse the queue (swap the two ends, move inwards):

```
// Reverse the elements between front and rear: swap the two ends, move inwards
void reverseQueue()
{
    if (front == -1)
        return;
    int i = front, j = rear;
    while (i < j)
    {
        int temp = queueArr[i];
        queueArr[i] = queueArr[j];
        queueArr[j] = temp;
        i++;
        j--;
    }
}
```

Queue using a linked list (read chapter 10 first; no MAX, so no overflow):

```
// Queue using a linked list: no fixed size, so no overflow.
// Enqueue at the rear end of the list, dequeue from the front end.
struct QNode
{
    int data;
    QNode *next;
};
```

```
QNode *qFront = NULL, *qRear = NULL;

void enqueueLL(int value)
{
    QNode *newNode = new QNode;
    newNode->data = value;
    newNode->next = NULL;
    if (qRear == NULL)                      // empty queue
        qFront = qRear = newNode;
    else
    {
        qRear->next = newNode;
        qRear = newNode;
    }
}

int dequeueLL()                             // returns -1 if empty
{
    if (qFront == NULL)
    {
        cout << "Queue Underflow!" << endl;
        return -1;
    }
    QNode *temp = qFront;
    int value = temp->data;
    qFront = qFront->next;
    if (qFront == NULL)                     // queue became empty
        qRear = NULL;
    delete temp;
    return value;
}
```

Queue for CPU scheduling (FCFS). Processes are served in arrival order. Tested: bursts 24, 3, 3 give waiting times 0, 24, 27, average 17.

```
// CPU scheduling (First Come First Served): processes wait in a queue in arrival order.
// Waiting time of a process = sum of burst times of everyone before it.
float fcfs(int burst[], int n)
{
    int waiting = 0, totalWait = 0;
    cout << "Process\tBurst\tWaiting\tTurnaround" << endl;     // \t = tab
    for (int i = 0; i < n; i++)             // dequeue in FIFO order
    {
        int turnaround = waiting + burst[i];
        cout << "P" << i + 1 << "\t" << burst[i] << "\t" << waiting << "\t" << turnaround << endl;
        totalWait += waiting;
        waiting += burst[i];
    }
    cout << "Average waiting time = " << (float)totalWait / n << endl;
    return (float)totalWait / n;
}
```

CHECK 9.1

MAX = 4, linear queue. enqueue 5, enqueue 6, dequeue, dequeue, enqueue 7. What are front and rear after each step?

CHECK 9.2

Circular queue, MAX = 4, front = 2, rear = 3. Enqueue 9: which index is it stored at? Is the queue full now?

Answers

Check 9.1: enqueue 5 → front 0, rear 0. enqueue 6 → 0, 1. dequeue (5) → 1, 1. dequeue (6): front == rear → reset → −1, −1. enqueue 7 → 0, 0.

Check 9.2: rear = (3 + 1) % 4 = **0**, so 9 goes to index 0. Full? (0 + 1) % 4 = 1 ≠ front (2), so **not full**: index 1 is still free.

10 · Exp 11 — Singly linked list (insert, delete, traverse)

The pointers from Exp 1 finally pay off. New C++: struct, new / delete, and the arrow operator ->. Read 10.1 slowly; everything after builds on it.

struct Node → new Node → head → insert at end → delete from beginning → traverse until NULL

10.1 New C++: struct, new, delete, ->

A **struct** bundles several variables into one new type:

```
struct Node
{
    int data;      // the value
    Node *next;    // address of the next node (NULL if none)
};
```

- `new Node` asks for memory for one Node **while the program runs** and gives back its **address**: `Node *newNode = new Node;`
- `p->data` means "the `data` field of the node that `p` points to". It is shorthand for `(*p).data`.
- `delete p;` gives that node's memory back. Every `new` should have a matching `delete`.
- `NULL` is the "points nowhere" address: the end of the list, or an empty list (`head == NULL`).

GUESS FIRST

An array keeps its elements side by side. How does a linked list know where its second element is, if the nodes can be anywhere in memory?

HEAD → 10 → 20 → 30 → NULL



Insert 40 at end: walk temp to the node whose next is NULL, then

```
temp->next = newNode;
```

Delete from beginning:

```
temp = head; head = head->next; delete temp;
```

Order matters: move head BEFORE delete, or the rest of the list is lost.

The manual's list after inserting 10, 20, 30. Each node stores the address of the next.

The list is reached only through `head`, the address of the first node. Lose `head` and the whole list is lost.

10.2 The three operations

Operation	Steps	Time
insert at end	make node (next = NULL) → if `head == NULL`, head = node → else walk `temp` until `temp->next == NULL`, then `temp->next = newNode`	O(n) (the walk)
delete from beginning	if empty → message → else `temp = head`, `head = head->next`, `delete temp`	O(1)
traverse	`temp = head`; while `temp != NULL`: print `temp->data`, `temp = temp->next`	O(n)

THE TWO CLASSIC LINKED-LIST BUGS

1. **Forgetting `temp = temp->next;`** in traverse: `temp` never moves, so the loop never ends. The manual's IMPLEMENTATION screenshot (page 9) has exactly this bug, and its insert never links the new node in. Its **listing** (page 6) is correct; use that.
2. **`delete head;` before moving head**: the second node's address was inside the deleted node, so the rest of the list is lost. Always save the node in `temp`, move `head`, *then* `delete temp`.

10.3 The program

The manual's listing was correct (one line was cut off at the page edge: `cout << temp->data << " deleted from list." << endl;`).

E11_SINGLY_LINKED_LIST.CPP

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *next;
};

Node *head = NULL;                          // empty list

// Insert at the end
void insert(int value)
{
    Node *newNode = new Node;               // ask for memory for one node
    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL)
    {
        head = newNode;
    }
    else
    {
        Node *temp = head;
        while (temp->next != NULL)          // walk to the last node
            temp = temp->next;
```

```
        temp->next = newNode;
    }
    cout << value << " inserted into list." << endl;
}

// Delete from the beginning
void deleteNode()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else
    {
        Node *temp = head;
        cout << temp->data << " deleted from list." << endl;
        head = head->next;                  // move head first...
        delete temp;                        // ...then free the old first node
    }
}

void traverse()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else
    {
        Node *temp = head;
        cout << "Linked List: ";
        while (temp != NULL)
        {
            cout << temp->data << " ";
            temp = temp->next;              // without this line: infinite loop
        }
        cout << endl;
    }
}

int main()
{
    int choice, value;

    do
    {
        cout << "\n1. Insert" << endl;
        cout << "2. Delete" << endl;
        cout << "3. Traverse" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to insert: ";
            cin >> value;
            insert(value);
        }
        else if (choice == 2)
        {
```

```
            deleteNode();
        }
        else if (choice == 3)
        {
            traverse();
        }
        else if (choice == 4)
        {
            cout << "Program terminated." << endl;
        }
        else
        {
            cout << "Invalid choice!" << endl;
        }
    } while (choice != 4);

    return 0;
}
```

The manual's Sample Inputs 1 and 2 in one run:

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: 1
Enter value to insert: 10
10 inserted into list.
Enter your choice: 1
Enter value to insert: 20
20 inserted into list.
Enter your choice: 1
Enter value to insert: 30
30 inserted into list.
Enter your choice: 3
Linked List: 10 20 30
Enter your choice: 2
10 deleted from list.
Enter your choice: 3
Linked List: 20 30
Enter your choice: 4
Program terminated.
```

Edge cases (delete and traverse on an empty list, delete the only node):

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: 2
List is empty.
Enter your choice: 3
List is empty.
Enter your choice: 1
Enter value to insert: 5
5 inserted into list.
Enter your choice: 2
```

```
5 deleted from list.
Enter your choice: 3
List is empty.
Enter your choice: 4
Program terminated.
```

WRITE IN YOUR RECORD

Aim: To implement a singly linked list and perform insertion, deletion and traversal.

Algorithm: head = NULL. *Insert (end):* create node (data = value, next = NULL); if head == NULL, head = node; else move temp to the last node (temp->next == NULL) and set temp->next = node. *Delete (beginning):* if head == NULL → "empty"; else temp = head, head = head->next, delete temp. *Traverse:* temp = head; while temp ≠ NULL print temp->data, temp = temp->next.

Complexity: insert at end O(n), delete at beginning O(1), traverse O(n); space O(n) for n nodes.

Result: Inserted 10, 20, 30 (list: 10 20 30); deleting from the beginning removed 10 (list: 20 30).

10.4 Viva questions

Question	Answer
Linked list vs. array?	List: nodes anywhere in memory, size grows/shrinks, insert/delete by changing pointers, but no direct `a[i]` access (must walk) and an extra pointer per node. Array: contiguous, fixed size, O(1) indexing, O(n) insert/delete.
What does the last node's next hold?	NULL.
What is head?	A pointer to the first node; `head == NULL` means the list is empty.
Why is insert at end O(n)?	You must walk from head to the last node. (Keeping a `tail` pointer makes it O(1).)
Why is delete at beginning O(1)?	Only head changes; no walking.
What is `->`?	Access a field through a pointer: `p->data` = `(*p).data`.
Why `delete temp`?	Return the node's memory; otherwise it leaks.
Can you traverse backwards?	Not in a singly linked list (only next pointers). A doubly linked list adds `prev`.

10.5 If the examiner changes the question

Insert at end and delete from beginning are the experiment. The rest (9 checks, all pass), each a function to add above `main()` and to the menu:

Insert at the beginning (O(1)):

```
void insertBegin(int value)
{
    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = head;                    // new node points to the old first node
    head = newNode;                          // and becomes the first node
}
```

Delete from the end. Stop at the second-last node:

```
void deleteEnd()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
        return;
    }
    if (head->next == NULL)                  // only one node
    {
        cout << head->data << " deleted from list." << endl;
        delete head;
        head = NULL;
        return;
    }
    Node *temp = head;
    while (temp->next->next != NULL)         // stop at the SECOND-last node
        temp = temp->next;
    cout << temp->next->data << " deleted from list." << endl;
    delete temp->next;
    temp->next = NULL;                       // it is now the last node
}
```

Search (returns the position) and **count nodes**:

```
int search(int key)                          // 1-based position, -1 if absent
{
    int pos = 1;
    Node *temp = head;
    while (temp != NULL)
    {
        if (temp->data == key)
            return pos;
        temp = temp->next;
        pos++;
    }
    return -1;
}
```

```
int countNodes()
{
    int count = 0;
    for (Node *temp = head; temp != NULL; temp = temp->next)
        count++;
    return count;
}
```

Display in reverse (recursion: the deepest call prints first):

```
// Recursion: first print the REST of the list in reverse, then this node
void displayReverse(Node *p)
{
    if (p == NULL)
        return;
    displayReverse(p->next);
    cout << p->data << " ";
}
// call it as:  displayReverse(head);
```

Insert at a position:

```
void insertAtPos(int pos, int value)         // pos 1 = new first node
{
    if (pos < 1)
    {
        cout << "Invalid position." << endl;
        return;
    }
    if (pos == 1)
    {
        insertBegin(value);
        return;
    }
    Node *temp = head;
    for (int i = 1; i < pos - 1 && temp != NULL; i++)   // go to node pos-1
        temp = temp->next;
    if (temp == NULL)
    {
        cout << "Invalid position." << endl;
        return;
    }
    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = temp->next;              // 1) new node points ahead
    temp->next = newNode;                    // 2) then link it in (order matters!)
}
```

Delete at a position:

```
void deleteAtPos(int pos)
{
    if (head == NULL || pos < 1)
    {
        cout << "Invalid position." << endl;
        return;
    }
    Node *del;
    if (pos == 1)
    {
        del = head;
        head = head->next;
    }
    else
    {
        Node *temp = head;
        for (int i = 1; i < pos - 1 && temp != NULL; i++)   // go to node pos-1
            temp = temp->next;
```

```
        if (temp == NULL || temp->next == NULL)
        {
            cout << "Invalid position." << endl;
            return;
        }
        del = temp->next;
        temp->next = del->next;              // bypass the node
    }
    cout << del->data << " deleted from list." << endl;
    delete del;
}
```

CHECK 10.1

For inserting a new node after node `temp`, these two lines come in some order: `temp->next = newNode;` and `newNode->next = temp->next;`. Which must come first, and what happens if you swap them?

CHECK 10.2

List: head → 4 → 8 → NULL. Trace `deleteNode()` then `insert(5)`. Draw the list after each.

Answers

Check 10.1: `newNode->next = temp->next;` **first**. If `temp->next = newNode;` runs first, the old `temp->next` address is overwritten, so `newNode->next` would point to newNode itself and the rest of the list is lost.

Check 10.2: delete: temp = (4), head = (8), delete temp → head → 8 → NULL. insert(5): walk to 8 (next is NULL), link → head → 8 → 5 → NULL.

11 · Exp 12 — Circular linked list (insert, delete, traverse)

Exp 11 with one change: the last node points back to head instead of NULL. Every "== NULL" test becomes "== head".

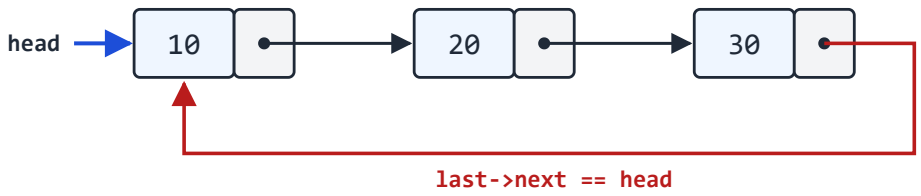
Exp 11 list → **last node points to head** → find last: next == head → traverse: do-while → delete first: fix the last node

11.1 The idea

GUESS FIRST

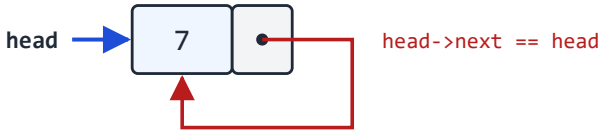
In a circular list there is no NULL to stop at. If traverse used `while (temp != head)` with `temp = head` at the start, what would it print?

Circular: the last node points back to head (never NULL)



Traverse with do { ... } while (temp != head); - a plain while (temp != head) would stop before printing anything.

The manual's list after inserting 10, 20, 30.



A single node points to itself. The program must treat this case separately.

	Singly (Exp 11)	Circular (Exp 12)
last node's next	NULL	head
"is this the last node?"	`temp->next == NULL`	`temp->next == head`
traverse loop	`while (temp != NULL)`	`do { … } while (temp != head);`
delete from beginning	O(1)	**O(n)**: the last node must be found and re-pointed to the new head

The answer to the guess: **nothing**. temp starts *equal* to head, so a `while` loop exits before its first pass. A `do … while` prints first and tests afterwards.

11.2 The program (corrected)

The manual's PROGRAM listing doesn't compile:

G++ SAYS

```
m12_listing.cpp:62:17: error: missing terminating " character
   62 |     {   cout << "\n1. Insert
      |                 ^
m12_listing.cpp:13:6: error: variable or field 'insert' declared void
   13 | void insert(intvalue)
      |      ^~~~~~
m12_listing.cpp:13:13: error: 'intvalue' was not declared in this scope
   13 | void insert(intvalue)
      |             ^~~~~~~~
m12_listing.cpp:46:15: error: unused variable 'temp' [-Werror=unused-variable]
   46 |     {   Node* temp = head;
      |               ^~~~
m12_listing.cpp:55:16: error: 'temp' was not declared in this scope; did you mean 'tm'?
   55 |     }   delete temp;
      |                ^~~~
m12_listing.cpp:62:14: error: no match for 'operator<<' (operand types are 'std::ostream' {aka
'std::basic_ostream<char>'} and 'std::ostream' {aka 'std::basic_ostream<char>'})
   62 |     {   cout << "\n1. Insert
      |         ~~~~ ^~
```

The four problems: `insert(intvalue)` (missing space); the menu `cout` lines are missing their closing `" << endl;` ; `delete temp;` sits after the closing `}` of the block where `temp` was declared; and `main()` is cut off at the page end. The IMPLEMENTATION screenshot (page 7) is correct and complete; this pack follows it.

E12_CIRCULAR_LINKED_LIST.CPP

```cpp
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *next;
};

Node *head = NULL;

// Insert at the end
void insert(int value)                    // FIX: manual printed "intvalue"
{
    Node *newNode = new Node;
    newNode->data = value;

    if (head == NULL)
    {
        head = newNode;
        newNode->next = head;             // one node points to itself
    }
    else
    {
        Node *temp = head;
```

```
        while (temp->next != head)          // last node is the one pointing to head
            temp = temp->next;
        temp->next = newNode;
        newNode->next = head;               // close the circle
    }
    cout << value << " inserted into list." << endl;
}

// Delete from the beginning
void deleteNode()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else if (head->next == head)            // only one node
    {
        cout << head->data << " deleted from list." << endl;
        delete head;
        head = NULL;
    }
    else
    {
        Node *temp = head;
        Node *last = head;
        while (last->next != head)          // find the last node
            last = last->next;

        cout << head->data << " deleted from list." << endl;
        head = head->next;
        last->next = head;                  // last now points to the new head
        delete temp;                        // FIX: was outside this block (temp unknown there)
    }
}

void traverse()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else
    {
        Node *temp = head;
        cout << "Circular Linked List: ";
        do                                  // do-while: print head before testing
        {
            cout << temp->data << " ";
            temp = temp->next;
        } while (temp != head);
        cout << endl;
    }
}

int main()
{
    int choice, value;

    do
    {
        cout << "\n1. Insert" << endl;       // FIX: manual's menu strings were unclosed
```

```
        cout << "2. Delete" << endl;
        cout << "3. Traverse" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to insert: ";
            cin >> value;
            insert(value);
        }
        else if (choice == 2)
        {
            deleteNode();
        }
        else if (choice == 3)
        {
            traverse();
        }
        else if (choice == 4)
        {
            cout << "Program terminated." << endl;
        }
        else
        {
            cout << "Invalid choice!" << endl;
        }
    } while (choice != 4);

    return 0;
}
```

The manual's sample run, reproduced exactly:

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: 1
Enter value to insert: 10
10 inserted into list.
Enter your choice: 1
Enter value to insert: 20
20 inserted into list.
Enter your choice: 1
Enter value to insert: 30
30 inserted into list.
Enter your choice: 3
Circular Linked List: 10 20 30
Enter your choice: 2
10 deleted from list.
Enter your choice: 3
Circular Linked List: 20 30
Enter your choice: 4
Program terminated.
```

Deleting the only node (the `head->next == head` branch), then deleting from an empty list:

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: 1
Enter value to insert: 7
7 inserted into list.
Enter your choice: 2
7 deleted from list.
Enter your choice: 3
List is empty.
Enter your choice: 2
List is empty.
Enter your choice: 4
Program terminated.
```

DELETE-FROM-BEGINNING, THREE CASES

1. **Empty** (`head == NULL`): print a message.
2. **One node** (`head->next == head`): delete it, set `head = NULL` . Without this case, the general code would set head to the deleted node itself.
3. **General:** find `last` (its next is head), move `head = head->next` , set `last->next = head` , then `delete temp` .

WRITE IN YOUR RECORD

Aim: To implement a circular linked list and perform insertion, deletion and traversal.

Algorithm: *Insert (end):* create node; if head == NULL: head = node, node->next = head; else move temp until temp->next == head, temp->next = node, node->next = head. *Delete (beginning):* if empty → message; if one node → delete it, head = NULL; else find last, temp = head, head = head->next, last->next = head, delete temp. *Traverse:* temp = head; do { print temp->data; temp = temp->next } while (temp ≠ head).

Complexity: insert at end O(n), delete at beginning O(n) (finding the last node), traverse O(n); space O(n), auxiliary O(1).

Result: Inserted 10, 20, 30 (list: 10 20 30); deleted 10 (list: 20 30). The last node always pointed back to head.

11.3 Viva questions

Question	Answer
What is a circular linked list?	A linked list whose last node points to the first node instead of NULL.
How do you detect the last node?	`temp->next == head`.
Why `do-while` for traversal?	The stop condition (`temp == head`) is already true at the start; do-while prints head first.
Why is delete-from-beginning O(n) here but O(1) in Exp 11?	The last node's next must be updated to the new head, and finding it takes a walk. (A `tail` pointer would make it O(1).)
One-node list: what is head->next?	head itself.
Advantages?	Any node can reach every other node; natural for round-robin scheduling, multiplayer turns, repeating playlists.
Risk?	Infinite loops, if a traversal's stop condition is wrong.
Types?	Circular singly linked (next only) and circular doubly linked (next and prev).

11.4 If the examiner changes the question

The same assignment list as Exp 11, adapted so the circle stays closed (10 checks, all pass, each including "is it still circular?").

Insert at the beginning. The last node must point to the new head:

```
// Like insert at end, but afterwards head moves to the new node
void insertBegin(int value)
{
    Node *newNode = new Node;
    newNode->data = value;
    if (head == NULL)
    {
        head = newNode;
        newNode->next = head;
        return;
    }
    Node *last = head;
    while (last->next != head)              // the last node must point to the new first node
        last = last->next;
    newNode->next = head;
    last->next = newNode;
    head = newNode;
}
```

Delete from the end:

```
void deleteEnd()
{
    if (head == NULL)
    {
```

```
        cout << "List is empty." << endl;
        return;
    }
    if (head->next == head)                 // only one node
    {
        cout << head->data << " deleted from list." << endl;
        delete head;
        head = NULL;
        return;
    }
    Node *temp = head;
    while (temp->next->next != head)        // stop at the second-last node
        temp = temp->next;
    cout << temp->next->data << " deleted from list." << endl;
    delete temp->next;
    temp->next = head;                      // new last node closes the circle
}
```

Search and count (do-while again):

```
int search(int key)                         // 1-based position, -1 if absent
{
    if (head == NULL)
        return -1;
    Node *temp = head;
    int pos = 1;
    do
    {
        if (temp->data == key)
            return pos;
        temp = temp->next;
        pos++;
    } while (temp != head);
    return -1;
}
```

```
int countNodes()
{
    if (head == NULL)
        return 0;
    int count = 0;
    Node *temp = head;
    do
    {
        count++;
        temp = temp->next;
    } while (temp != head);
    return count;
}
```

Display in reverse (recursion that stops when the next node is head):

```
// Recursion that stops when it comes back round to head
void reverseFrom(Node *p)
{
    if (p->next != head)
        reverseFrom(p->next);
    cout << p->data << " ";
}

void displayReverse()
{
    if (head == NULL)
        cout << "List is empty.";
    else
        reverseFrom(head);
    cout << endl;
}
```

Insert / delete at a position:

```
void insertAtPos(int pos, int value)         // pos 1 = new first node
{
    int n = countNodes();
    if (pos < 1 || pos > n + 1)
    {
        cout << "Invalid position." << endl;
        return;
    }
    if (pos == 1)
    {
        insertBegin(value);
        return;
    }
    Node *temp = head;
    for (int i = 1; i < pos - 1; i++)        // go to node pos-1
        temp = temp->next;
    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = temp->next;              // for pos = n+1 this is head: circle stays closed
    temp->next = newNode;
}
```

```
void deleteAtPos(int pos)
{
    int n = countNodes();
    if (pos < 1 || pos > n)
    {
        cout << "Invalid position." << endl;
        return;
    }
    if (pos == 1)                            // same as Exp 12's deleteNode()
    {
        Node *last = head;
        while (last->next != head)
            last = last->next;
        Node *del = head;
        cout << del->data << " deleted from list." << endl;
        if (n == 1)
            head = NULL;
        else
```

```
        {
            head = head->next;
            last->next = head;
        }
        delete del;
        return;
    }
    Node *temp = head;
    for (int i = 1; i < pos - 1; i++)
        temp = temp->next;
    Node *del = temp->next;
    temp->next = del->next;
    cout << del->data << " deleted from list." << endl;
    delete del;
}
```

CHECK 11.1

Circular list head → 3 → 6 → 9 → (back to 3). Run delete-from-beginning by hand: what are `last`, the new head, and `last->next` afterwards?

CHECK 11.2

Why does `insert` in Exp 12 need `newNode->next = head;` even in the `head == NULL` branch?

Answers

Check 11.1: last = node 9; head moves to node 6; `last->next = head` → 9 now points to 6; node 3 is deleted. List: 6 → 9 → (back to 6).

Check 11.2: A lone node is both first and last, so its next must point to head (itself) to keep the list circular. If it were left unset, the next traversal or insert would follow a garbage address.

12 · Exam day — procedure and one-page summary

Read this the night before and once more outside the lab. It doesn't replace typing the programs from memory (chapter 13).

12.1 The 6-step routine

1. **Read the question twice.** Is it the plain experiment or an ASSIGNMENT twist (descending order, insert at beginning, circular queue…)? Find the matching "If the examiner changes the question" section in your head.
2. **Write the record first** (aim, algorithm, complexity). It takes 5 minutes, earns marks even if the code fights you, and plans your program.
3. **Type the skeleton, compile it, then grow it.** `#include <iostream>` / `using namespace std;` / `int main() { return 0; }`. Build once (F9 / F11) so you know the setup works. Then add one function at a time and compile after each.
4. **Fix errors top-down.** Fix only the first error and recompile (chapter 1.6).
5. **Run the manual's sample input** and check the output against the expected answer below.
6. **Test one edge case** (empty list, bad position, overflow). Examiners like seeing it handled.

12.2 Expected outputs (the manual's sample inputs)

Exp	Input	Output
1	—	Value = 25, address = pointer value, value via pointer = 25
2	—	10 → 20 → 10 → 30; addresses base, +4, base, +8
3	10 20 30 40 50; begin 5 / pos 3 val 25 / end 60	5 10 20 30 40 50 / 10 20 25 30 40 50 / 10 20 30 40 50 60
4	10 20 30 40 50; begin / pos 3 / end	20 30 40 50 / 10 20 40 50 / 10 20 30 40
5	[[1,2],[3,4]] × [[5,6],[7,8]]	19 22 / 43 50
6	[[1,2,3],[4,5,6]]	1 4 / 2 5 / 3 6
7	5 2 4 6 1 3	1 2 3 4 5 6
8	10 7 8 9 1 5	1 5 7 8 9 10
9	push 10 20 30, pop	pops 30 (LIFO)
10	enqueue 10 20 30, dequeue	display 10 20 30; deletes 10; then 20 30
11	insert 10 20 30, delete	10 20 30; deletes 10; then 20 30
12	insert 10 20 30, delete	10 20 30; deletes 10; then 20 30

12.3 Complexity cheat sheet

Operation	Time	Space
Pointer access / arithmetic (Exp 1–2)	O(1)	O(1)
Array insert / delete at beginning or position (Exp 3–4)	O(n)	O(1)
Array insert / delete at end	O(1)	O(1)
Matrix multiply (r1·c1·c2) / n × n	$O(n^3)$	$O(n^2)$
Transpose m × n	O(m·n)	O(m·n)
Insertion sort: best / avg / worst	O(n) / $O(n^2)$ / $O(n^2)$	O(1)
Quick sort: best / avg / worst	O(n log n) / O(n log n) / $O(n^2)$	O(log n) avg, O(n) worst
Bubble / selection sort (worst)	$O(n^2)$	O(1)
Merge sort (all cases)	O(n log n)	O(n)
Stack push / pop / peek	O(1)	O(MAX)
Queue enqueue / dequeue; display	O(1); O(n)	O(MAX)
Singly list: insert end / delete begin / traverse	O(n) / O(1) / O(n)	O(n)
Circular list: insert end / delete begin / traverse	O(n) / O(n) / O(n)	O(n)

12.4 The lines that decide each program

Exp	The line(s) that matter
1	`ptr = &number;` · `*ptr`
2	`int *ptr = arr;` · `ptr++` moves one int (4 bytes)
3	`for (i = n; i >= pos; i--) arr[i] = arr[i-1];` · `arr[pos-1] = value;` · `n++`
4	`for (i = pos-1; i < n-1; i++) arr[i] = arr[i+1];` · `n--`
5	`C[i][j] = 0;` then `C[i][j] += A[i][k] * B[k][j];` for k < c1 · check `c1 == r2`
6	`T[j][i] = A[i][j];` · display `(T, cols, rows)`
7	`while (j >= 0 && arr[j] > key) { arr[j+1] = arr[j]; j--; }` · `arr[j+1] = key;`
8	`if (arr[j] < pivot) { i++; swap(arr[i], arr[j]); }` · `swap(arr[i+1], arr[high]); return i+1;`
9	full: `top == MAX-1` · empty: `top == -1`
10	full: `rear == MAX-1` · empty: `front == -1` · `if (front == rear) front = rear = -1;`
11	`while (temp->next != NULL)` · `head = head->next; delete temp;` · `temp = temp->next;`
12	`while (temp->next != head)` · `do { … } while (temp != head);` · `last->next = head;`

12.5 General viva questions

Question	Answer
What is a data structure?	A way of organising data in memory so operations on it are efficient (array, stack, queue, linked list, tree, graph).
Linear vs. non-linear?	Linear: elements in a sequence (array, stack, queue, list). Non-linear: hierarchy or network (tree, graph).
Static vs. dynamic?	Static: size fixed at compile time (array). Dynamic: grows at run time with `new` (linked list).
What does O(n) mean?	The running time grows in proportion to n (the input size), ignoring constants.
Time vs. space complexity?	Time = how the number of steps grows with n; space = how much extra memory grows with n.
Algorithm vs. program?	An algorithm is the step-by-step method (language-independent); a program is that method written in a language.
What is recursion?	A function calling itself on a smaller problem, with a base case that stops it (quick sort, merge sort, reverse display).
Why `using namespace std;`?	To use `cout`, `cin`, `endl` without writing `std::` each time.

13 · Revision set (the spaced re-test)

One question per key idea. Do this on paper, without the pack, a day or more after finishing the chapters, and again two or three days later. Typing the programs from memory is the real test.

> **HOW TO USE IT**
>
> Answer everything first, then mark with the answers at the end. Anything you missed: re-read that chapter section, then **type that program from scratch** the same day. Don't just re-read: being able to type it without looking is what earns marks.

Part A — type from memory (the actual exam skill)

Close the pack, open your editor, and type each program until it compiles and gives the expected output (chapter 12.2). Time yourself; aim for under 15 minutes each.

1. Exp 3 (insert, all three cases)
2. Exp 4 (delete, all three cases)
3. Exp 5 (matrix multiply with the three functions)
4. Exp 7 (insertion sort)
5. Exp 8 (quick sort with partition)
6. Exp 10 (queue with menu)
7. Exp 11 (singly linked list with menu)
8. Exp 12 (circular linked list with menu)

Part B — short questions

1. `int x = 9; int *p = &x; *p = 4;`. What is x now?
2. `int a[5]`; `int *p = a;`. Write the address of `a[3]` using p, and its value.
3. Insert at position 2 in `3 6 9` (n = 3), value 4. Final array?
4. Why does the insertion shift loop run from the end, but the deletion shift loop from the front?
5. Valid positions for insertion in an n-element array? For deletion?
6. A is 3 × 2, B is 2 × 4. Is A × B possible? Order of the result? Number of multiplications?
7. Transpose of a 4 × 1 matrix: what order?
8. Insertion sort on `3 1 2`: write the array after each pass.
9. Best-case input for insertion sort, and its time complexity?
10. Partition `7 2 9 4` with the last element as pivot. Final array and returned index?
11. Which input makes this program's quick sort $O(n^2)$?
12. Why does `swapElements` take `int &a, int &b`?
13. Linear queue, MAX = 3: enqueue 1, 2, 3, dequeue, enqueue 4. What happens, and why?
14. Circular queue condition for "full"?
15. Stack: push 1, 2, 3, pop, push 4, pop, pop. Which values come out?
16. In a singly linked list, how do you know you're at the last node? In a circular one?
17. What goes wrong if traverse forgets `temp = temp->next;`?
18. In delete-from-beginning, what goes wrong with `delete head; head = head->next;`?
19. Why is delete-from-beginning O(1) in a singly list but O(n) in the circular list (as programmed)?
20. Why is `int arr[n];` (n read with cin) a problem?
21. What does `p->data` mean, written another way?
22. Give the stability (stable or not) of insertion sort and of quick sort.

Answers

1. **4**: `*p = 4` writes through the pointer into x.
2. Address `p + 3`; value `*(p + 3)` (same as `a[3]`).
3. **3 4 6 9**.
4. In each direction, the element about to be overwritten must be copied first: shifting right, the last element moves first; shifting left, the one just after the gap moves first.
5. Insertion: 1 … n+1. Deletion: 1 … n.
6. Yes (2 = 2). Result 3 × 4. Multiplications: 3 · 4 · 2 = **24**.
7. **1 × 4**.
8. Pass 1 (key 1): `1 3 2`. Pass 2 (key 2): `1 2 3`.
9. An already sorted array: **O(n)** (n − 1 comparisons).
10. pivot 4, i = −1. j = 0: 7 no. j = 1: 2 < 4 → i = 0, swap(arr[0], arr[1]) → `2 7 9 4`. j = 2: 9 no. Final swap(arr[1], arr[3]) → `2 4 9 7`, returns **1**.
11. An already sorted (or reverse-sorted) array: the last-element pivot is always the largest (or smallest).
12. So the function swaps the real array elements, not copies.
13. **Overflow** on enqueue 4: rear is still MAX − 1 even though slot 0 was freed. This is the linear-queue flaw.
14. `(rear + 1) % MAX == front`.
15. **3, 4, 2** (LIFO); 1 remains.
16. Singly: `temp->next == NULL`. Circular: `temp->next == head`.
17. `temp` never moves, so the loop never ends (infinite loop).
18. After `delete head`, reading `head->next` uses freed memory; the rest of the list is lost or the program crashes. Save `temp = head`, move head, then delete temp.
19. In the circular list the last node must be re-pointed to the new head, and finding it means walking the whole list.
20. It's a variable-length array: not standard C++. Visual Studio rejects it and strict g++ errors on it. Use a fixed maximum like `int arr[100]`.
21. `(*p).data`.
22. Insertion sort: **stable**. Quick sort: **not stable**.